

HATEM MAHBOULI

Modélisation de programmes C en expressions régulières

Mémoire présenté
à la Faculté des études supérieures de l'Université Laval
dans le cadre du programme de maîtrise en informatique
pour l'obtention du grade de Maître ès sciences (M.Sc.)

FACULTÉ DES SCIENCES ET DE GÉNIE
UNIVERSITÉ LAVAL
QUÉBEC

2011

Résumé

L'analyse statique des programmes est une technique de vérification qui permet de statuer si un programme est conforme à une propriété donnée. Pour l'atteinte de cet objectif, il faudrait disposer d'une abstraction du programme à vérifier et d'une définition des propriétés. Dans la mesure où l'outil de vérification prend place dans un cadre algébrique, la définition des propriétés ainsi que la modélisation du programme sont représentées sous la forme d'expressions régulières.

Ce mémoire traite en premier lieu de la traduction des programmes écrits en langage C vers une abstraction sous forme d'expressions régulières. La méthode de traduction proposée, ainsi que les différentes étapes de transformations y sont exposées. Les premiers chapitres du présent mémoire énoncent les connaissances élémentaires de la théorie des langages ainsi que de la compilation des programmes. Un bref aperçu des différentes méthodes de vérification des programmes est présenté comme une mise en contexte. Finalement, la dernière partie concerne la traduction des programmes ainsi que la description de l'outil de traduction qui a été réalisé.

Avant-propos

Je tiens à remercier mon directeur de recherche, le professeur Béchir Ktari pour son soutien et son dévouement, les précieux conseils qu'il m'a donnés et sa patience.

Je présente également mes vifs remerciements aux professeurs Nadia Tawbi et Mohammed Mejri, pour avoir accepté de lire et d'évaluer ce travail.

Je présente toute ma gratitude et mes profonds remerciements à mes parents pour leurs conseils et leur soutien quotidien, je leur suis aussi reconnaissant de m'avoir supporté et encouragé à travers toutes ces longues années d'études.

Je remercie également tous mes collègues du groupe LSFM, spécialement mes collègues de bureau pour les nombreuses et enrichissantes discussions que nous avons eu. Ainsi, je remercie tous mes amis pour leurs encouragements et leur soutien, même aux petites heures du matin.

*À mes parents,
à mes frères et soeurs.*

*"If you want to keep a secret, you must
also hide it from yourself."
George Orwell (1984)*

Table des matières

Résumé	ii
Avant-propos	iii
Table des matières	v
Liste des tableaux	viii
Table des figures	ix
1 Introduction	1
2 Théorie des langages	4
2.1 Notions de langages	4
2.2 Automates finis	6
2.2.1 Diagramme de transitions	6
2.2.2 Table de transitions	7
2.2.3 Automates finis déterministes	8
2.2.4 Automates finis non déterministes	10
2.2.5 Limite des automates finis	12
2.3 Automates à pile	13
2.4 Grammaires de phrases	15
2.4.1 Hiérarchie de Chomsky	16
2.4.2 Grammaire générale (type 0)	17
2.4.3 Grammaires contextuelles (type 1)	17
2.4.4 Grammaires hors-contexte (type 2)	18
2.4.5 Grammaires régulières (type 3)	20
2.4.6 Grammaire à choix finis (type 4)	22
2.5 Expressions régulières	22
2.5.1 Union de deux langages	22
2.5.2 Concaténation de deux langages	23
2.5.3 Fermeture d'un langage	23
2.5.4 Expressions régulières	24

2.6	Algèbre de Kleene	25
2.7	Conclusion	27
3	Principes et techniques de compilation	28
3.1	Compilation de programmes	28
3.2	Analyse lexicale	31
3.3	Analyse syntaxique	32
3.3.1	Analyse syntaxique descendante	33
3.3.2	Analyse syntaxique ascendante	36
3.4	Constructeurs d'analyseurs syntaxiques	39
3.4.1	Lex/Yacc	40
3.4.2	ANTLR	41
3.4.3	GoldParser	42
3.4.4	Comparaison des pseudo-langages de description de grammaire	43
3.5	Conclusion	47
4	Vérification de programmes	48
4.1	Analyse dynamique de programmes	48
4.2	Analyse statique de programmes	50
4.2.1	TAL	50
4.2.2	PCC	54
4.2.3	Analyse par évaluation de modèle	56
4.3	Conclusion	59
5	Technique de traduction	60
5.1	Approche algébrique	60
5.2	Transcription de programmes	62
5.2.1	Réécriture de programmes	62
5.2.2	Transformation de programmes	64
5.2.3	Transformation des fonctions	65
5.2.4	Transformation des expressions	67
5.2.5	Transformation des instructions	76
5.2.6	Exemple de traduction	85
5.3	Modélisation en automate fini déterministe	89
5.3.1	Principes de modélisation	90
5.3.2	Élimination des ϵ -transitions	93
5.3.3	Table des transitions	94
5.3.4	Exemple de modélisation	94
5.4	Modélisation en expressions régulières	97
5.5	Conclusion	98

6	Implantation	99
6.1	Application Web pour l'analyse de programmes	100
6.2	Modélisation de code C	103
6.3	Conclusion	107
7	Conclusion	109
	Bibliographie	112
A	Étoile d'une matrice de dimension $n \times n$, $n \geq 2$	116
B	Grammaire du langage C	117

Liste des tableaux

3.1	Grammaire des expressions arithmétiques en GoldParser.	45
3.2	Syntaxe de la grammaire utilisant Yacc.	45
3.3	Syntaxe de la grammaire utilisant ANTLR.	46
5.1	Définition d'une fonction en langage C.	65
5.2	Syntaxe simplifiée de définition d'une fonction.	66
5.3	Programme de calcul du factoriel.	67
5.4	Réécriture du programme de calcul du factoriel.	68
5.5	Syntaxe abstraite des expressions du langage C.	70
5.6	Syntaxe du langage $\mathcal{L}_I$	77
5.7	Syntaxe du langage $\mathcal{L}'_I$	78
5.8	Fonction <i>BigIfThenElse</i>	83
5.9	Programme p à transformer.	85
5.10	Programme p après transformation.	89
5.11	Programme p après transformation.	95
7.1	Exemple de code C.	110

Table des figures

2.1	Diagramme de transitions du langage L_2	6
2.2	Diagramme de transitions du langage L'_2	7
2.3	Diagramme de transitions sur l'alphabet $\{a, b\}$	8
2.4	Diagramme de transitions de l'automate A_2	12
2.5	Digramme de transition de l'automate A_3	14
3.1	Étapes de compilation	29
3.2	Diagramme de flux de Lex/Yacc.	40
3.3	Les deux composants et le fichier partagé.	42
3.4	Étapes d'analyse d'un code source.	44
4.1	Techniques de surveillance dynamique du code.	49
4.2	Syntaxe de TAL.	52
4.3	Sémantique opérationnelle de TAL.	53
4.4	Architecture de PCC.	54
4.5	Schématisation de l'approche du Model Checking.	57
5.1	Approche algébrique à la vérification de modèle.	61
5.2	Modèle du programme p	96
5.3	Flot de contrôle du programme p	96
6.1	Application Web pour l'analyse de programmes.	101
6.2	Outils de modélisation - onglet "Source".	104
6.3	Outils de modélisation - onglet "Traduction".	105
6.4	Outils de modélisation - onglet "Automate" sans ϵ -réduction.	106
6.5	Outils de modélisation - onglet "Automate" avec ϵ -réduction.	107
6.6	Outils de modélisation - onglet "Expression".	107

Chapitre 1

Introduction

Dans le livre *1984* de George Orwell, l'auteur fait référence à des télécrans qui sont placés dans tous les foyers et dans tous les espaces publics dans le but d'imposer à la population une pensée unique et totalitaire. Sans pour autant pousser la paranoïa au point de se demander si notre société tend vers l'environnement sorti de l'imagination de l'auteur, on ne peut s'empêcher d'y songer quand on constate que de nos jours : il y un au moins un ordinateur par foyer, il y des écrans d'informations partout (dans les grandes avenues, les centres commerciaux etc.), de plus en plus de gens sont connectés à Internet en permanence, nous avons en moyenne un équipement informatique que nous transportant sur nous (smartphone, notebook, lecteur mp3, etc.).

Cette dépendance envers l'informatique connaît une expansion planétaire, et touche toutes les classes et toutes les générations. Il est assez étonnant de voir la simplicité avec laquelle des enfants de 4 ans manipulent des tablettes informatiques (c.f iPad), alors que certains aînés n'assimilent pas encore l'usage d'une souris. Dans un tel contexte, il est légitime de se poser des questions sur les risques que peuvent apporter toutes ces technologies si elles sont utilisées à des fins malhonnêtes. En effet, de nos jours il est plus que banal de prendre le contrôle d'un ordinateur qui se trouve à des milliers de kilomètres de distance, c'est fort accommodant lorsqu'il s'agit de récupérer ses propres données alors qu'on est en voyage, mais qu'en serait-il si une tierce personne pouvait y avoir accès ? Quels sont les gages que nous pouvons avoir vis-à-vis des logiciels que nous utilisons instinctivement chaque jour ? Qu'est-ce qui peut nous garantir l'intimité de la discussion que nous avons avec nos proches à l'autre bout du monde ?

C'est à de telles questions que la recherche dans le domaine de la sécurité informatique oeuvre pour trouver des réponses. La complexité des problèmes que pose la sécurité informatique fait qu'il n'est pas évident de trouver des solutions intrinsèques. Il existe

plusieurs sous domaine de sécurité informatique. Par exemple la *cryptographie* assure la transmission de manière sécuritaire des données confidentielles. La *certification des logiciels* garantit la conformité d'un programme par rapport aux critères de sécurité en générant un certificat valide. La *vérification des programmes* quant à elle, permet de s'assurer du fonctionnement "correct" des programmes.

L'*analyse statique*, une technique de vérification des programmes, présente la particularité de déceler les éventuelles non-conformités d'un programme avant même son exécution. L'étude du code source du programme en question permet de dénombrer les risques encourus lors de son exécution. Cette approche est intéressante dans la mesure où aucun préjudice n'est apporté, et que les programmes suspects sont évincés avant même leur compilation. C'est précisément dans ce cadre de recherche que se situe le travail présenté dans ce mémoire. Il s'agit d'effectuer une précompilation du code source du programme, pour obtenir une expression formelle, qui sera analysée par des méthodes algébriques dans le but de découvrir d'éventuelles non-conformités.

Objectifs

L'abstraction des programmes est un point de départ pour plusieurs techniques de vérification formelles [1], le cadre algébrique dans lequel se situe le travail présenté dans [31] traite effectivement un programme sous sa forme abstraite d'une expression régulière de l'algèbre de Kleene. Notre objectif consiste donc à concrétiser l'analyse des programmes par une méthode algébrique en proposant une abstraction des programmes écrits en langage C. La méthode présentée dans ce document permet de partir d'un programme concret, écrit avec le formalisme ANSI C, et d'aboutir à une formulation d'une expression régulière représentative du programme, permettant ainsi son analyse par la méthode susmentionnée. Les transformations à apporter consistent à simplifier le programme en éliminant les structures complexes de la syntaxe du langage C, telles que les imbrications multiples, les structures itératives pour ne garder qu'une version linéaire du programme, basée sur des instructions de saut et des branchements conditionnels. Cette forme intermédiaire permet de dégager le diagramme de flot de contrôle du programme et ainsi abstraire le comportement sous forme d'un automate fini composé seulement des appels de fonction et des branchements originaux du programme. Finalement, une version de la table des transitions de ce diagramme est traitée par la méthode décrite par Kozen [28], permet d'aboutir à la formulation sous forme d'expressions régulières du programme initial.

Structure du mémoire

Dans le chapitre 2 nous introduisons les bases fondamentales de la théorie des langages, cette partie présente les notions essentielles à connaître pour pouvoir caractériser un langage de programmation. De plus, nous présentons des éléments importants pour la modélisation d'un langage à l'aide d'un automate fini déterministe, ainsi qu'une description des expressions régulières et leur apport dans la méthode utilisée. Le chapitre 3 présente les principes de la compilation, processus par lequel il est possible de transformer un programme écrit en langage de haut niveau sous la forme d'une structure exploitable pour en tirer le comportement intentionné par le programmeur. Nous présentons aussi dans ce chapitre un survol des outils disponibles pour l'automatisation de la tâche de compilation ainsi que des exemples concrets utilisés lors de l'élaboration de l'outil d'abstraction développé. Nous abordons les approches existantes en la matière de vérification des programmes dans le chapitre 4, et nous détaillons l'analyse par évaluation de modèle, approche de laquelle est inspirée l'analyse algébrique. Le chapitre 5 présente une description détaillée du processus de modélisation des programmes écrits en langage C. Ce chapitre couvre les deux principales étapes de transformation, à savoir la simplification du code source du programme, et la modélisation des différentes instructions sous forme d'expressions régulières de l'algèbre de Kleene. Le chapitre 6 présente les différentes illustrations représentatives du fonctionnement de l'outil développé, et de la transformation apportée sur un programme pour aboutir à la forme d'une expression régulière de l'algèbre de Kleene. Finalement, le chapitre 7 fait un retour récapitulatif du contenu de ce mémoire et présente les perspectives futures pour les améliorations à apporter à la version de l'outil actuel.

Chapitre 2

Théorie des langages

Avant de parler de programme informatique et de compilation, il est primordial d'appréhender les notions de la théorie des langages. Effectivement, le seul moyen d'inculquer à un ordinateur un comportement particulier, c'est de recourir à un programme informatique permettant de décrire la démarche à entreprendre.

Dans ce chapitre nous présentons un survol des notions fondamentales de la théorie des langages, nécessaires pour la compréhension ultérieure de la définition du langage C.

Notons qu'une grande partie des définitions et des concepts présentés dans ce chapitre sont tirés de [14] et, dans une moindre mesure, de [18].

2.1 Notions de langages

Tout langage naturel, qu'il soit complexe ou rudimentaire, est formé de la combinaison de lettres, de mots et de phrases. Un mot est un ensemble de lettres disposées côte à côte ; une phrase est un ensemble de mots séparés par des blancs ; et un texte est un ensemble de phrases dont la composition donne le sens global du document. Cependant, il existe des règles bien spécifiques à chaque langage qui permettent de déterminer si un mot appartient au langage ou non, si l'agencement des mots constitue une phrase cohérente ou pas, et si les phrases constituent un discours rationnel.

La théorie des langages dans le domaine informatique s'inspire directement des principes du langage naturel, les notions d'alphabet, de mot et de phrase y représentent les mêmes concepts.

2.1 Définition. Un *alphabet* est un ensemble fini, non vide, de symboles. On le dénote généralement par Σ . $\square$

La séquence vide, représentée par ϵ , peut être aussi un élément de l'alphabet ; par contre, elle ne contient aucun symbole. Ainsi la concaténation de la séquence vide et d'une séquence non vide a aura pour résultat la séquence a :

$$\epsilon a = a \epsilon = a$$

L'ensemble regroupant toutes les séquences finies de symboles d'un alphabet Σ est un ensemble infini dénombrable noté Σ^* . Le symbole $*$ représente la fonction de fermeture qui, appliquée à un ensemble non vide fini ou infini, permet d'obtenir un autre ensemble qui est infini.

2.2 Définition. Soit Σ un alphabet. On dénote par Σ^* l'ensemble de toutes les séquences finies de symboles de Σ . $\square$

Autrement dit, l'ensemble Σ^* contient tous les mots de taille finie qu'il est possible de former avec les éléments de l'alphabet Σ . Prenons comme exemple un alphabet contenant trois éléments $\Sigma = \{a, b, c\}$, alors

$$\Sigma^* = \{\epsilon, a, b, c, aa, ab, ac, \dots, aaa, aab, \dots, aaaa, aaab, \dots\}$$

Nous pouvons remarquer que la séquence vide appartient à l'ensemble Σ^* même si elle ne fait pas partie des éléments de Σ .

Nous pouvons énoncer maintenant la définition d'un langage :

2.3 Définition. Un *langage* sur un alphabet Σ est un sous-ensemble de l'ensemble Σ^* . $\square$

Ainsi, en prenant en considération l'alphabet $\Sigma = \{a, b, c\}$, nous pouvons affirmer que $L_1 = \{a, b, c, aa, bb, cc\}$ et $L_2 = \{\epsilon, a, aa, aaa, aaaa, \dots\}$ sont des langages sur Σ puisqu'ils sont des sous-ensembles de Σ^* . Il serait utile de signaler que quelque soit le langage, les mots qu'il contient sont toujours de longueur finie, bien que le langage peut être infini. De plus, la séquence vide peut faire partie des éléments d'un langage comme elle peut en être omise.

La façon la plus naïve de définir un langage est d'énumérer tous les mots qu'il contient. Par contre, cette approche n'est pas toujours concevable surtout dans le cas de langages infinis. La représentation la plus courante d'un langage consiste en sa modélisation à travers les automates ou encore les expressions régulières.

2.2 Automates finis

Un langage, tel que défini plus haut, est un ensemble de mots constitués chacun de la concaténation des éléments d'un alphabet. De ce fait, pour représenter un langage de manière efficace, il suffit de définir un modèle qui permet de déterminer si un mot appartient au langage ou pas. Ainsi, au lieu d'énumérer tous les mots du langage, nous nous assurons de l'appartenance d'un mot au langage en le confrontant au *diagramme de transitions* représentant ce langage.

2.2.1 Diagramme de transitions

2.4 Définition. Un diagramme de transitions est une collection finie d'états, représentés sous forme de cercles, et de transitions, représentées sous forme d'arcs dirigés, tels que :

- chaque état peut être étiqueté d'une étiquette unique ;
- un et un seul *état initial*, représenté par un cercle avec une petite flèche ($\rightarrow$) ;
- un ou plusieurs *états finaux*, représentés par des cercles doubles (il peut ne pas y avoir d'état final) ;
- chaque arc relie deux états (pas nécessairement différents) ;
- chaque arc est étiqueté avec un ou plusieurs symboles d'un alphabet.

□

Pour illustrer cette définition, reprenons le langage $L_2 = \{\epsilon, a, aa, aaa, aaaa, \dots\}$ défini sur l'alphabet $\Sigma = \{a, b, c\}$. Il est constitué de tous les mots qu'il est possible de former à partir du symbole a . Il est évident que l'on ne peut pas énumérer la totalité des mots de ce langage. La représentation de ce langage par un diagramme de transitions est composée d'un seul état, à la fois initial et final, et d'une seule transition étiquetée avec le symbole a .

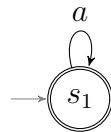


FIGURE 2.1 – Diagramme de transitions du langage L_2

Tous les mots du langage L_2 sont acceptés par ce diagramme de transition et seuls les mots du langage L_2 . En effet, pour chaque symbole a constituant un mot, le diagramme se trouve dans un état final, ce qui implique l'acceptation du mot. La présence de la

séquence vide parmi l'ensemble des mots du langage justifie le fait que l'état initial est en même temps un état final.

Prenons un autre exemple ; soit le langage $L'_2 = \{a, aa, aaa, aaaa, \dots\}$ qui représente le même ensemble de mots que L_2 diminué de la séquence vide. Le diagramme de transitions de ce langage est le suivant :

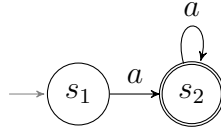


FIGURE 2.2 – Diagramme de transitions du langage L'_2 .

Nous pouvons observer que dans ce cas, l'état initial et l'état final sont dissociés. Selon le langage que représente un diagramme de transitions, ce dernier peut avoir plusieurs propriétés différentes. Nous présentons la notion d'un diagramme *complètement défini* et celle d'un diagramme *non ambigu* pour finalement définir la propriété d'un *diagramme déterministe*.

2.5 Définition. Un diagramme de transitions sur un alphabet Σ est dit *complètement défini* si, pour chaque symbole $s \in \Sigma$ et chaque état e , il y a au moins une transition étiquetée par s qui quitte e . $\square$

2.6 Définition. Un diagramme de transitions sur un alphabet Σ est dit *non ambigu* si, pour chaque symbole $s \in \Sigma$ et chaque état e , il existe au plus une transition quittant e et étiquetée par s . $\square$

2.7 Définition. Un diagramme de transitions *déterministe* sur un alphabet Σ donné est un diagramme complètement défini et non ambigu. $\square$

2.2.2 Table de transitions

Pour une exploitation pratique des diagrammes de transitions, il est plus commun de passer par la forme tabulaire, qui est plus adaptée à être utilisée par un algorithme.

2.8 Définition. Une *table de transitions* associée à un diagramme de transitions non ambigu est une matrice à deux dimensions telle que :

- la matrice a une ligne pour chaque état du diagramme ;
- elle a une colonne pour chaque symbole utilisé comme étiquette d'un arc du diagramme ;
- l'entrée de la n^{ieme} ligne et de la m^{ieme} colonne est l'état atteint dans le diagramme de transitions en quittant l'état n par l'arc dont l'étiquette est celle de la colonne m . Si un tel état n'existe pas, l'entrée contient le mot 'Erreur' ;
- la matrice contient une colonne ayant comme indice le caractère spécial de fin de séquence 'Fin'. Les entrées de cette colonne sont le mot 'Accepte', si l'état sur la ligne correspondante est final, ou le mot 'Erreur' si l'état n'est pas final.

□

Prenons un exemple simple : soit le diagramme de la figure 2.3, défini sur l'alphabet $\{a, b\}$; il représente le langage composé des mots formés uniquement avec la lettre a ; les mots qui contiennent la lettre b ne font pas partie de ce langage.

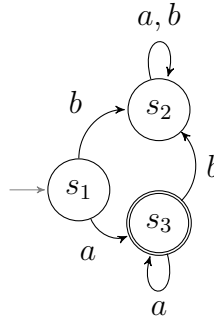


FIGURE 2.3 – Diagramme de transitions sur l'alphabet $\{a, b\}$

La table de transitions correspondante à ce diagramme est donnée ci-dessous.

	a	b	Fin
s_1	s_3	s_2	Erreur
s_2	s_2	s_2	Erreur
s_3	s_3	s_2	Accepte

2.2.3 Automates finis déterministes

2.9 Définition. Un *automate fini déterministe* consiste en un quintuple de la forme $(S, \Sigma, \delta, i, F)$ dont les composantes sont définies comme suit :

- S est un ensemble fini d'état.

- Σ est l'alphabet de l'automate.
- δ est la fonction de transition de $S \times \Sigma$ dans S .
- $i \in S$ est l'état initial.
- $F \subseteq S$ est l'ensemble des états finaux.

□

Il faudrait faire la distinction entre digramme de transitions et automate. En effet, le diagramme de transitions n'est que la représentation de la fonction de transition δ de l'automate. Cependant, il est courant de dire d'un diagramme de transitions que c'est un automate à cause du lien direct qu'il y a entre les deux notions.

Selon cette définition, nous pouvons dégager les différentes caractéristiques suivantes communes à tout automate fini déterministe :

- Pour chaque couple $(p, x) \in S \times \Sigma$, il existe un et un seul $q \in S$ tel que $\delta(p, x) = q$.
- À tout automate fini déterministe correspond un diagramme de transitions déterministe. Ceci découle du fait que la fonction de transitions d'un automate fini déterministe est complètement définie et non ambiguë.
- L'appellation 'fini' est utilisée pour indiquer que l'ensemble d'états de l'automate est fini, que son alphabet est fini et que son ensemble de transitions est aussi fini.

Les automates finis déterministes sont utilisés pour la reconnaissance d'une certaine classe de langage, pour cela il faudrait aussi définir formellement la notion d'acceptation d'une séquence de symboles d'un alphabet Σ par un automate fini déterministe.

2.10 Définition. Un automate fini déterministe $M = (S, \Sigma, \delta, i, F)$ accepte (ou reconnaît) une séquence $x_1x_2x_3 \cdots x_n$ (avec $n \in \mathbb{N}$) si, et seulement si, il y a une séquence d'états $s_0, s_1, s_2, \cdots, s_n$ tels que $s_0 = i, s_n \in F$, et pour $j = 1, \cdots, n, \delta(s_{j-1}, x_j) = s_j$. Dans le cas contraire, on dit que l'automate rejette (ou refuse) la séquence. □

Autrement dit, l'automate reconnaît une séquence de symboles si, et seulement si, il existe une suite d'arcs conduisant de l'état initial à un état final, et qui correspond à la lecture des symboles de la séquence de gauche à droite. L'exemple suivant présente le processus d'acceptation d'une séquence par un automate.

Prenons un exemple pour illustrer l'acceptation d'une séquence par un automate. Soit l'automate A_1 décrit par le diagramme de transition précédent (figure 2.3). Son alphabet est constitué de l'ensemble $\{a, b\}$. Soient les deux séquences $aaaa$ et $abaa$, et examinons le processus d'acceptation par l'automate.

La séquence de reconnaissance de la séquence $aaab$ est donnée dans le tableau suivant : la première colonne indique la position de lecture dans la séquence, et la deuxième colonne indique l'état correspondant dans lequel se trouve l'automate.

Lecture	Etat
<u>aaaa</u>	s_1
<u>a</u> aaa	s_3
aa <u>a</u> a	s_3
aaaa <u>a</u>	s_3
aaaa <u>_</u>	s_3

À la fin de la lecture de la séquence *aaaa*, l'automate se trouve dans l'état s_3 qui est un état final, donc la séquence est acceptée. Par contre pour la séquence *abaa*, on peut voir dans le tableau suivant que lors de la fin de la lecture, l'automate se trouve dans l'état s_2 , qui n'est pas un état final, et donc la séquence est rejetée.

Lecture	Etat
<u>abaa</u>	s_1
<u>a</u> baa	s_3
ab <u>a</u> a	s_2
aba <u>a</u>	s_2
abaa <u>_</u>	s_2

2.11 Définition. Soit M un automate fini déterministe qui a pour alphabet Σ . Nous dénotons par $L(M)$ l'ensemble de toutes les séquences de symboles reconnus par l'automate M . Nous dirons que $L(M)$ est le *langage reconnu* (ou *accepté*) par l'automate fini M . $\square$

Ainsi, en reprenant l'automate A_1 précédant, nous pouvons statuer que l'ensemble des séquences qu'il est capable d'accepter est le langage $L(A_1) = \{a^n : n \in N\}$. Le nombre de langages que l'on peut représenter avec des automates finis déterministes est assez réduit étant donné que leur définition est assez restrictive. De ce fait, les automates déterministes ne sont pas adaptés pour l'analyse de textes informatiques tels que les programmes écrits avec un langage de programmation évolué tel que le langage C.

2.2.4 Automates finis non déterministes

La définition des automates finis non déterministes s'affranchit de la contrainte de déterminisme imposée par un diagramme complètement défini et non ambigu. Cette

simplification amène une réduction du nombre d'états pour la représentation des mêmes langages que les automates déterministes. Un automate fini non déterministe est défini de la sorte :

2.12 Définition. Un *automate fini non déterministe* consiste en un quintuple de la forme (S, Σ, ρ, i, F) . Voici la description des différentes composantes.

- S est un ensemble fini d'états.
- Σ est l'alphabet de la machine.
- ρ est un sous-ensemble de $S \times \Sigma \times S$.
- $i \in S$ est l'état initial.
- $F \subseteq S$ est l'ensemble des états finaux.

□

La classe des langages représentés par les automates non déterministes englobe la classe des langages des automates déterministes. La seule différence entre les définitions des deux automates consiste dans la définition de l'ensemble des transitions. Pour les automates non déterministes, il ne s'agit plus d'une fonction de transitions mais plutôt d'une relation d'un sous-ensemble de $S \times \Sigma \times S$.

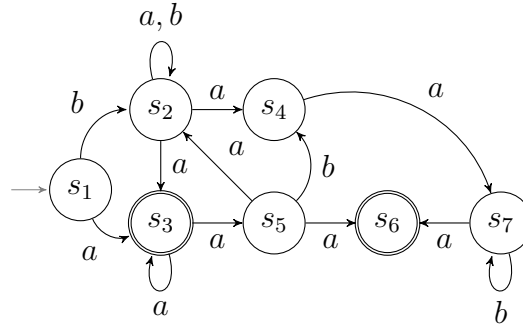
Ainsi, la condition d'acceptation des automates non déterministes est reformulée comme suit :

2.13 Définition. Un automate fini non déterministe $M = (S, \Sigma, \rho, i, F)$ *accepte* (ou *reconnaît*) la séquence $x_1 \dots x_n$ ($n \geq 0$) si, et seulement si, il *existe* une séquence d'états $s_0, \dots, s_n$ tels que $s_0 = i, s_n \in F$, et pour $j = 1, \dots, n$, $(s_{j-1}, x_j, s_j) \in \rho$. Dans le cas contraire, on dit que l'automate refuse la séquence. On dénote par $L(M)$ l'ensemble des séquences acceptées par M . On dit que $L(M)$ est le *langage reconnu* (ou *accepté*) par M . □

Autrement formulé, un automate fini non déterministe M reconnaît une séquence s sur un alphabet Σ s'il existe au moins un chemin à travers les transitions qui mène de l'état initial à un état final, correspondant à la séquence des symboles de la séquence. Par conséquent, une séquence est acceptée s'il est possible d'atteindre un état final en lisant toute la séquence de symboles, même s'il y a d'autres chemins qui mènent à des états non finaux.

Soit l'automate fini non déterministe A_2 ayant pour alphabet $\Sigma = \{a, b\}$ et décrit par le diagramme de transition de la figure 2.4.

Examinons les étapes d'acceptation de la séquence $baaa$ par l'automate A_2 , et l'ensemble des états accessibles à chaque étape.

FIGURE 2.4 – Diagramme de transitions de l'automate A_2

Lecture	Ensemble d'états
$\underline{b}aaa$	$\{s_1\}$
$b\underline{a}aa$	$\{s_2\}$
$ba\underline{a}a$	$\{s_2, s_3, s_4\}$
$baa\underline{a}$	$\{s_2, s_3, s_5, s_7\}$
$baaa\underline{\quad}$	$\{s_2, s_3, s_4, s_5, s_6\}$

La séquence est acceptée puisqu'à la fin de la lecture des symboles, l'ensemble des états contient les états finaux s_3 et s_6 . Par contre, si nous considérons la séquence $baab$, elle n'est pas reconnue par l'automate puisque l'ensemble des états atteints à la fin de la lecture des symboles ne contient aucun état final.

Lecture	Ensemble d'états
$\underline{b}aab$	$\{s_1\}$
$b\underline{a}ab$	$\{s_2\}$
$ba\underline{a}b$	$\{s_2, s_3, s_4\}$
$baa\underline{b}$	$\{s_2, s_3, s_5, s_7\}$
$baab\underline{\quad}$	$\{s_2, s_4, s_7\}$

2.2.5 Limite des automates finis

La classe des langages reconnus par les automates finis se limite aux langages réguliers, il n'est donc pas possible pour un automate fini de reconnaître un langage non régulier. En effet, si nous considérons les expressions parenthésées, un automate fini ne peut pas déterminer si le nombre des parenthèses droite et gauche sont égaux ; cela demanderait un nombre d'états supérieur au nombre de parenthèses, ce qui mènerait à une explosion

d'états si nous voulons reconnaître des expressions de taille considérable. Cette limitation réside dans le fait qu'un automate fini n'est pas en mesure de comptabiliser le nombre de parenthèses droite et gauche en cours de lecture.

Les automates à pile permettent d'augmenter la puissance des automates finis en se dotant d'une mémoire sous forme d'une pile. L'utilisation de la pile permet d'évincer les limites des automates finis, et rend ainsi possible la reconnaissance de langages non réguliers.

2.3 Automates à pile

Un automate à pile est un automate fini auquel on ajoute une mémoire ; mis à part la pile, les deux types d'automates à pile et fini sont identiques. L'alphabet utilisé par la pile lui est propre et peut être différent de l'alphabet d'entrée de l'automate.

L'apport de la pile sur le comportement de l'automate consiste à le doter de deux nouvelles actions : empiler un symbole et dépiler un symbole ; de ce fait, les opérations que peut effectuer un automate à pile sont les suivantes :

- lire un symbole (et avancer la tête de lecture) ;
- dépiler un symbole ;
- empiler un symbole ;
- et changer d'état.

Pour représenter une transition d'un automate à pile, nous utilisons la notation (p, x, y, q, z) où p , x , y , q et z sont respectivement l'état courant, le symbole à l'entrée, le symbole dépilé, le nouvel état et le symbole empilé. Il est à noter que le y est dépilé avant que le z ne soit empilé. L'automate n'est pas obligé de faire à la fois une lecture, un empilement et un dépilement à chaque transition. Si aucun symbole n'est lu, nous mettons un ϵ à la place du x . Si aucun symbole n'est dépilé, un ϵ est utilisé à la place du y . Si aucun symbole n'est empilé, nous mettons un ϵ à la place du z .

2.14 Définition. Un *automate à pile* est un sextuplet $(S, \Sigma, \Gamma, T, \iota, F)$ où

- S est un ensemble fini d'états,
- Σ est l'alphabet d'entrée (alphabet du ruban),
- Γ est l'alphabet de la pile,
- T est un ensemble fini de transitions,
- $\iota \in S$ est l'état initial,
- $F \subseteq S$ est l'ensemble des états finaux.

L'automate à pile démarre dans l'état initial avec une pile vide. Il accepte la séquence d'entrée si et seulement si, il peut atteindre un état final après l'avoir lue en entier. Le

langage accepté par un automate à pile M est noté $L(M)$; c'est l'ensemble des séquences acceptées (c'est-à-dire que $L(M) = \{w : w \text{ est accepté}\}$). $\square$

Remarque : Il n'est pas nécessaire que l'automate à pile atteigne un état final tout de suite après avoir lu le dernier symbole d'entrée. Après avoir terminé la lecture de la séquence d'entrée, l'automate peut effectuer des transitions en manipulant sa pile et atteindre de cette façon un état accepteur.

Soit l'automate à pile $A_3 = (\{1, 2, 3, 4\}, \{a, b\}, \{a, b, \#\}, T, 1, \{1, 4\})$, où T est l'ensemble de transitions suivantes :

$$T = \{(1, \epsilon, \epsilon; 2, \#), (2, a, \epsilon; 2, a), (2, b, a; 3, \epsilon), (3, b, a; 3, \epsilon), (3, \epsilon, \#; 4, \epsilon)\}.$$

La figure 2.5 représente le diagramme de transitions de cet automate.

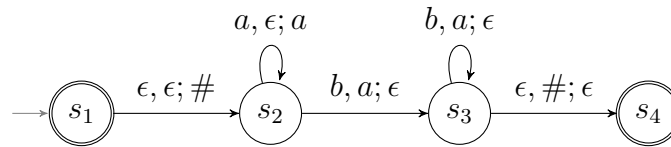


FIGURE 2.5 – Diagramme de transition de l'automate A_3

Le symbole b , bien que faisant partie de l'alphabet de la pile, n'est pas utilisé par celle-ci ; dans ce cas, nous pouvons considérer que l'alphabet de la pile se résume à l'ensemble $\{a, \#\}$. Le fait que l'état initial de l'automate soit aussi un état final, implique que le mot composé de la séquence vide ϵ est accepté par l'automate.

Prenons la séquence $aabb$ et observons les étapes requises pour son acceptation par l'automate A_3 :

État	Lecture	Pile
S_1	<u>a</u> abb	
S_2	a <u>a</u> bb	#
S_2	aa <u>b</u> b	#a
S_2	aab <u>b</u>	#aa
S_3	aabb <u></u>	#a
S_3	baab <u></u>	#
S_4	baab <u></u>	

À présent observons le comportement de l'automate A_3 vis-à-vis de la séquence aab :

État	Lecture	Pile
1	<u>a</u> ab	
2	a <u>a</u> b	#
2	aa <u>b</u>	#a
2	aab <u></u>	#aa
3	aab <u>_</u>	#a

À la fin de la lecture de la séquence, aucune transition n'est possible, car il n'y a plus de symbole b à lire et il n'est pas possible de dépiler le symbole $\#$, puisqu'il y a un a par dessus. La séquence aab est donc rejetée parce que l'état 3 n'est pas un état final.

Le langage accepté par l'automate A_3 est $L(A_3) = \{a^n b^n : n \in \mathbb{N}\}$. Ce langage est non régulier (plus précisément il s'agit un langage hors-contexte) et aucun automate fini ne peut l'accepter. Par contre, il est reconnu par l'automate A_3 puisque ce dernier utilise la pile pour mémoriser le nombre d'occurrences du symbole a en cours de lecture, pour valider ensuite la séquence d'entrée en comptabilisant le nombre d'occurrences du symbole b .

2.4 Grammaires de phrases

Contrairement aux automates finis qui servent pour la reconnaissance des séquences d'un langage, les grammaires permettent de générer un nouveau langage. Les grammaires les plus courantes et les plus familières sont les grammaires BNF.

2.15 Définition. Une *grammaire* (ou encore, *grammaire de phrases* ou *grammaire à structures de phrases*) est un quadruplet (N, T, S, R) où

- N est un ensemble fini de *symboles non terminaux* (alphabet non terminal).
- T est un ensemble fini de *symboles terminaux* (alphabet terminal) noté aussi Σ .
- $S \in N$ est le *symbole de départ* qu'on nomme aussi *axiome* ou *symbole initial*.
- R est un ensemble fini de *règles de réécriture* (ou *productions* ou *règles de substitution*). Ces règles sont formées d'un terme de gauche, d'une flèche ($\rightarrow$) et d'un terme de droite. Les termes gauche et droit peuvent être n'importe quelle combinaison de symboles de N ou T .

Dans certaines grammaires, il est possible que le côté droit soit vide, ce qui est indiqué par le symbole ϵ . Il s'agit dans ce cas de *règles- ϵ* qui s'écrivent sous la forme : $A \rightarrow \epsilon$.

□

Dans la suite, nous appliquerons les conventions suivantes pour noter les éléments de la grammaire :

- les non-terminaux seront notés par des symboles en majuscule ;
- les terminaux par des symboles en minuscules ;
- Le symbole initial sera noté S .

De la sorte, il suffira d'énoncer les règles de production pour avoir une définition complète d'une grammaire.

L'exemple suivant définit une grammaire constituée d'un seul symbole non-terminal S , qui est aussi le symbole de départ, de deux symboles terminaux a, b et de deux règles de production p_1 et p_2

$$\begin{array}{l|l} p_1 : & S \rightarrow aSb \\ p_2 : & S \rightarrow ab \end{array}$$

Cette grammaire permet la génération d'un langage formé des mots du type $a^n b^n$.

La génération de séquences de symboles à partir d'une grammaire se fait par *dérivation*.

La notion de dérivation est la réécriture d'une séquence de symboles par application d'une production.

2.16 Définition. Soit α, β, δ et γ des symboles de $(N \cup T)$. La relation de dérivation, notée $\Rightarrow$, sur l'ensemble $(N \cup T)^* \times (N \cup T)^*$ d'une grammaire G est définie par $\delta\alpha\gamma \Rightarrow \delta\beta\gamma$ si et seulement si $\alpha \rightarrow \beta$ est une production de G . $\square$

Ainsi, en considérant la grammaire définie précédemment, la dérivation par laquelle est généré le symbole $aaabbb$ s'écrit comme suit :

$$S \xRightarrow{p_1} aSb \xRightarrow{p_1} aaSbb \xRightarrow{p_2} aaabbb$$

2.17 Définition. Soit $G = (N, T, S, R)$. Si $T \subseteq \Sigma$ où Σ est un alphabet, on dit que G est une grammaire sur Σ . Soit $L(G) = \{w \in T^* : w \text{ est dérivée de } S\}$. On dit que $L(G)$ est le *langage généré* par G . $\square$

2.4.1 Hiérarchie de Chomsky

La classification des langages décrits par les grammaires formelles se répartit sur cinq familles de grammaires classées selon la complexité de leurs règles de production. Cette

classification forme une hiérarchie imbriquée des grammaires de complexité croissante, allant des grammaires *générales* (grammaire de type 0) aux grammaires à *choix fini* (grammaire de type 4).

Nous présentons brièvement ces types de grammaires avec une attention particulière aux grammaires *hors-contexte*.

2.4.2 Grammaire générale (type 0)

Les grammaires de type 0 sont les grammaires les plus générales, et par conséquent toute grammaire est du type 0. Ce qui signifie que cette classe englobe tous les langages reconnaissables par une machine de Turing. Cette famille de grammaires ont une expressivité maximale. Les règles de production des grammaires du type 0 s'écrivent sous la forme suivante :

$$\alpha \rightarrow \beta$$

avec $\alpha \in V^*TV^*$ et $\beta \in V^*$ où V est le vocabulaire défini comme l'union des symboles terminaux et non terminaux : $V = N \cup T$

2.4.3 Grammaires contextuelles (type 1)

Les grammaires monotones sont définies par des règles de production imposant que la partie droite de chaque règle soit nécessairement plus longue que la partie de gauche. Les grammaires contextuelles ajoutent à cette contrainte le fait que pour toute règle, le non-terminal soit entouré de deux mots à gauche et à droite et qui se retrouvent aussi après la dérivation. D'où l'appellation de grammaire contextuelle, puisque les mots encerclant le non-terminal s'apparentent à un contexte autour de lui. Le fait que la grammaire soit monotone implique que le non-terminal est transformé en un mot non vide.

2.18 Définition. On appelle grammaire contextuelle, ou sensible au contexte, une grammaire G telle que toute production de G est de la forme $\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2$ avec $\alpha_1, \alpha_2, \beta$ dans $(N \cup T)^*$, $\beta \neq \epsilon$ et $A \in N$. $\square$

Chaque production d'une grammaire contextuelle réécrit un seul symbole à la fois, les symboles qui encadrent le non-terminal (et qui représentent le contexte) restent inchangés.

2.19 Définition. On appelle langage contextuel ou langage sensible au contexte, un langage engendré par une grammaire contextuelle ou par une grammaire monotone. $\square$

Le langage $\{a^n b^n c^n, n \geq 1\}$ est un langage contextuel généré par la grammaire suivante :

$$\begin{array}{l|l} p_1 : & S \rightarrow aSQ \\ p_2 : & S \rightarrow abc \\ p_3 : & cQ \rightarrow Qc \\ p_4 : & bQc \rightarrow bbcc \end{array}$$

Examinons les dérivations appliquées pour obtenir les séquences abc , $aabbcc$ et $aaabbbcc$ appartenant à ce langage :

$$\begin{array}{lcl} S & \Rightarrow & abc \quad (p_2) \\ \hline S & \Rightarrow & aSQ \quad (p_1) \\ & \Rightarrow & aabcQ \quad (p_2) \\ & \Rightarrow & aabQc \quad (p_4) \\ & \Rightarrow & aabbcc \quad (p_4) \\ \hline S & \Rightarrow & aSQ \quad (p_1) \\ & \Rightarrow & aaSQQ \quad (p_1) \\ & \Rightarrow & aaabcQQ \quad (p_2) \\ & \Rightarrow & aaabQcQ \quad (p_3) \\ & \Rightarrow & aaabbccQ \quad (p_4) \\ & \Rightarrow & aaabbcQc \quad (p_3) \\ & \Rightarrow & aaabbQcc \quad (p_3) \\ & \Rightarrow & aaabbbccc \quad (p_4) \\ \hline \end{array}$$

Les langages contextuels constituent une classe importante de langages, ils sont essentiellement utilisés dans les domaines de traitement automatique des langages naturels.

2.4.4 Grammaires hors-contexte (type 2)

Les grammaires hors-contexte, appelées aussi les grammaires algébriques, sont les grammaires les plus utilisées en informatique. En effet, la définition de la syntaxe des

langages de programmation se fait généralement à travers une définition BNF¹, qui est une convention de grammaires non contextuelles.

L'introduction d'une nouvelle contrainte sur la partie de gauche des productions, consistant à avoir seulement des symboles non terminaux, permet d'éliminer la notion de contexte. Ainsi, toute grammaire hors-contexte est un cas particulier de grammaire contextuelle.

2.20 Définition. On appelle grammaire hors-contexte (ou grammaire de type 2 ou aussi grammaire algébrique) une grammaire dans laquelle toute production $\alpha \rightarrow \beta$ est de la forme $A \rightarrow \beta$ avec $A \in N$ et β dans $(N \cup T)^+$. $\square$

L'élimination de la notion de contexte dans cette classe de grammaire, apporte la possibilité de remplacer un non-terminal directement en appliquant les règles où il est placé dans la partie droite, contrairement aux grammaires contextuelles, où l'application d'une règle nécessitait à avoir le contexte idéal (deux symboles entourant le non-terminal à remplacer) pour être appliquée.

Cela dit, dans une grammaire hors contexte, il est possible d'avoir plusieurs règles différentes pour un même symbole terminal, l'ordre d'application des règles est arbitraire et aboutit à la génération de la même séquence.

En effet, nous pouvons commencer par appliquer les règles pour la réécriture des éléments non terminaux complètement à gauche. Dans ce cas, nous parlons de *dérivation gauche*. Par contre, nous parlons de *dérivation droite*, si nous commençons par appliquer les règles dans l'objectif de réécrire les éléments non terminaux les plus à droite.

2.21 Définition. On appelle dérivation gauche (respectivement droite) d'une grammaire hors-contexte G , une dérivation dans laquelle chaque étape de dérivation réécrit le non-terminal le plus à gauche (respectivement droite) du mot courant. $\square$

Soit la grammaire non contextuelle ayant pour alphabet $\{a, b, z\}$ et les productions suivantes :

$$\begin{array}{lcl} p_1 : & | & S \rightarrow zMNz \\ p_2 : & | & M \rightarrow aMa \\ p_3 : & | & N \rightarrow bNb \\ p_4 : & | & M \rightarrow z \\ p_5 : & | & N \rightarrow z \end{array}$$

1. Backus-Naur Form ou Backus Normal Form.

Examinons les dérivations appliquées pour obtenir les séquences $zazazz$:

$$\begin{array}{rcl}
 S & \Rightarrow & zMNz \quad (p_1) \\
 & \Rightarrow & zaMaNz \quad (p_2) \\
 & \Rightarrow & zaMazz \quad (p_5) \\
 & \Rightarrow & zazazz \quad (p_4)
 \end{array}$$

Dans cette séquence de dérivation, nous aurions pu appliquer la règle p_4 avant l'application de la règle p_5 , le résultat aura été le même. Même si l'ordre d'application des règles de production n'affecte pas le résultat final, il peut arriver que, pour une séquence terminale donnée, des choix différents de règles résultent en des arbres différents. Dans ce cas nous parlons de grammaire ambiguë.

2.22 Définition. Une grammaire est dite *ambiguë* si elle permet plus d'un arbre de dérivation pour une séquence terminale donnée. $\square$

2.23 Définition. Un langage est dit *non contextuel* s'il est généré par une grammaire non contextuelle. $\square$

À chaque grammaire non contextuelle correspond un automate à pile qui accepte le langage généré par la grammaire. L'inverse est aussi vrai. La démonstration du théorème 2.24 qui énonce ce résultat peut être trouvée dans [14] :

2.24 Théorème. Pour chaque automate à pile M , il existe une grammaire non contextuelle G telle que $L(M) = L(G)$. $\square$

2.4.5 Grammaires régulières (type 3)

Les grammaires régulières simplifient un peu plus la forme des règles de production en introduisant une nouvelle contrainte sur la partie droite des productions.

2.25 Définition. On appelle grammaire régulière (ou grammaire de type 3) une grammaire dans laquelle toute production $\alpha \rightarrow \beta$ est soit de la forme $A \rightarrow aB$, soit de la forme $A \rightarrow a$ avec $A, B \in N$ et a dans T . $\square$

Par définition, toute grammaire régulière est hors-contexte. La spécificité des grammaires régulières est que chaque production introduit au moins un symbole terminal, et au plus un symbole non-terminal à sa droite. La production d'un mot du langage se termine par la réécriture du dernier symbole non-terminal généré par l'application d'une règle de la forme $A \rightarrow a$.

2.26 Définition. Un langage est dit *régulier* s'il est généré par une grammaire régulière. $\square$

Soit le langage $L(G) = \{a^n b^m : m, n \in N\}$ généré par la grammaire régulière suivante :

$$\begin{array}{lcl} p_1 : & | & S \rightarrow ASC \\ p_2 : & | & S \rightarrow B \\ p_3 : & | & B \rightarrow bB \\ p_4 : & | & B \rightarrow \epsilon \\ p_5 : & | & A \rightarrow a \\ p_6 : & | & C \rightarrow c \end{array}$$

Examinons les dérivations appliquées pour obtenir la séquence $aabcc$:

$$\begin{array}{lll} S & \Rightarrow & ASC \quad (p_1) \\ & \Rightarrow & AASCC \quad (p_1) \\ & \Rightarrow & AABCC \quad (p_2) \\ & \Rightarrow & aaBCC \quad (p_5) \\ & \Rightarrow & aaBcc \quad (p_6) \\ & \Rightarrow & aabBcc \quad (p_3) \\ & \Rightarrow & aabcc \quad (p_4) \end{array}$$

Les langages générés par les grammaires régulières, donc les langages réguliers, sont le type de langage qui sont reconnus par les automates finis. Ce résultat est donné par le théorème 2.27

2.27 Théorème. Si L est un langage régulier, il existe un automate fini A qui reconnaît L . Réciproquement, si L est un langage reconnaissable, avec $\epsilon \notin L$, alors il existe une grammaire régulière qui engendre L . $\square$

2.4.6 Grammaire à choix finis (type 4)

Il existe des grammaires encore plus simples que les grammaires régulières. En particulier, les grammaires à choix finis sont telles que tout non-terminal ne réécrit que des terminaux.

2.28 Définition. On appelle grammaire de type 4 (appelé également grammaire à choix finis) une grammaire dans laquelle toute production est de la forme : $A \rightarrow u$, avec $A \in N$ et $u \in \Sigma^+$. $\square$

2.5 Expressions régulières

Les expressions régulières permettent de représenter les langages réguliers en utilisant des opérateurs définis sur l'ensemble des langages. Les opérations introduites par les expressions régulières concernent l'union, la concaténation et la fermeture des langages réguliers. Nous commençons par définir les opérateurs qui permettent de combiner les langages réguliers, nous passons ensuite à la présentation des expressions régulières et de leur utilisation.

2.5.1 Union de deux langages

Soient deux langages réguliers L_1 et L_2 , l'union des deux langages est équivalente à l'union de deux ensembles de séquences. Le résultat est un langage régulier qui englobe les séquences de L_1 et les séquences de L_2 . L'opérateur utilisé pour l'union de deux langages est le même opérateur utilisé pour l'union de deux ensembles : $L_1 \cup L_2$.

2.29 Théorème. *L'union de deux langages réguliers L_1 et L_2 est aussi un langage régulier.* $\square$

2.5.2 Concaténation de deux langages

2.30 Définition. Soient L_1 et L_2 deux langages sur l'alphabet Σ . La *concaténation* de L_1 et L_2 , dénotée par $L_1 \circ L_2$, est définie comme suit :

$$L_1 \circ L_2 = \{w_1w_2 : w_1 \in L_1 \wedge w_2 \in L_2\}.$$

Autrement dit, $L_1 \circ L_2$ contient toutes les séquences formées par la concaténation d'une séquence de L_1 avec une séquence de L_2 . $\square$

À titre d'exemple, soit $L_1 = \{a, b\}$ et $L_2 = \{c, ad\}$ alors

$$L_1 \circ L_2 = \{ac, aad, bc, bad\}$$

2.31 Théorème. La concaténation de deux langages réguliers L_1 et L_2 est aussi un langage régulier. $\square$

2.5.3 Fermeture d'un langage

L'opération de fermeture d'un alphabet Σ , dénotée par Σ^* , est l'ensemble de toutes les séquences finies de symboles de Σ . Lorsqu'elle est étendue aux langages réguliers, elle est définie de la sorte :

2.32 Définition. Soit L un langage. La *fermeture* de L , dénotée L^* , est l'ensemble des séquences formées en concaténant un nombre fini (0 ou plusieurs) de séquences de L . Autrement dit,

$$L^* = \{w_1 \cdots w_n : n \in \mathbb{N} \wedge w_i \in L \text{ pour tout } i = 1 \dots n\}.$$

$\square$

D'après cette définition, on remarque que $\epsilon \in L^*$, pour tout langage L .

Plus clairement, si $L_1 = \{a\}$ alors $L_1^* = \{\epsilon, a, aa, aaa, aaaa, aaaaa, \dots\}$. De même, si $L_2 = \{ab\}$ alors $L_2^* = \{\epsilon, ab, abab, ababab, abababab, \dots\}$. Aussi, si $L_3 = \{a, ab\}$ alors $L_3^* = \{\epsilon, a, ab, aa, aab, aba, abab, aaa, aaba, aabab, abaa, \dots\}$.

2.33 Théorème. La fermeture d'un langage régulier L est un langage régulier. $\square$

2.5.4 Expressions régulières

Les expressions régulières permettent de représenter les langages réguliers. Elles sont essentiellement formées des symboles associés aux opérateurs précédemment définis sur l'ensemble des langages (union, concaténation, fermeture), et des éléments de l'alphabet sur lequel elles sont définies.

2.34 Définition. Une *expression régulière* sur un alphabet Σ se définit par les propositions récursives suivantes :

1. $\emptyset$ est une expression régulière.
2. x est une expression régulière, pour tout $x \in \Sigma$.
3. Si p et q sont des expressions régulières alors $(p \cup q)$ est une expression régulière.
4. Si p et q sont des expressions régulières alors $(p \circ q)$ est une expression régulière.
5. Si p est une expression régulière alors p^* est aussi une expression régulière.
6. Les seules expressions régulières sont celles qu'on peut construire selon 1 à 5.

□

En reprenant ces définitions, nous déduisons les langages réguliers $L(r)$ qui peuvent être représentés par l'expression régulière r .

2.35 Définition. Soit une expression régulière r et un alphabet Σ . Le langage *représenté* par r , noté $L(r)$, est défini par les propositions récursives suivantes :

1. $L(\emptyset) = \emptyset$
2. Pour tout $x \in \Sigma$, $L(x) = \{x\}$
3. Si p et q sont deux expressions régulières alors $L((p \cup q)) = L(p) \cup L(q)$
4. Si p et q sont deux expressions régulières alors $L((p \circ q)) = L(p) \circ L(q)$
5. Si p est une expression régulière alors $L(p^*) = (L(p))^*$

□

Remarque : Il faudrait faire la différence entre expressions régulières et langages. En effet une expression régulière sur un alphabet Σ est une séquence de symboles découlant de l'union de l'alphabet avec un ensemble d'opérateurs : $\Sigma \cup \{\cup, \circ, *, \emptyset\}$. Par contre, un langage est un ensemble de séquences de symboles de l'alphabet Σ . D'ailleurs, il est utile de noter que le langage vide ($\emptyset$) est représenté par l'expression régulière ($\emptyset$) : $L(\emptyset) = \emptyset$

Considérons l'expression régulière $r_1 = ((a \circ a)^* \cup (b \cup b)^*)$ et observons le langage qu'elle représente :

$$\begin{aligned}
 L(r_1) &= L(((a \circ a)^* \cup (b \cup b)^*)) \\
 &= L((a \circ a)^*) \cup L((b \cup b)^*) \\
 &= (L(a \circ a))^* \cup (L(b \cup b))^* \\
 &= (L(a) \circ L(a))^* \cup (L(b) \cup L(b))^* \\
 &= (\{a\} \circ \{a\})^* \cup (\{b\} \cup \{b\})^* \\
 &= \{aa\}^* \cup \{b\}^* \\
 L(r_1) &= \{a^{2n} : n \in N\} \cup \{b^n : n \in N\}.
 \end{aligned}$$

Algèbre des ensembles réguliers

Les langages réguliers sont par définition les langages reconnus par un automate fini (théorème 2.27) ; l'ensemble regroupant tous les langages réguliers est noté REG_Σ . L'approche algébrique concernant les langages réguliers, permet de définir un cadre algébrique, dans lequel l'utilisation des opérateurs sur les ensembles de langage (union, concaténation, fermeture), sert à manipuler des ensembles réguliers (ou langages réguliers, ou événements réguliers) sur un alphabet Σ . Cette approche contribue donc à bâtir un lien entre les expressions régulières et les automates finis.

2.36 Définition. L'algèbre REG_Σ des éléments réguliers sur un alphabet fini Σ est une structure algébrique $\langle \mathcal{R}, \cup, \circ, *, \emptyset, \{\epsilon\} \rangle$, où $\mathcal{R} \subseteq \mathcal{P}(\Sigma^*)$ contient tous les ensembles $\{u\}$, pour tout $u \in \Sigma$. Tout sous-ensemble $L \subseteq \mathcal{R}$ est dit être un *ensemble régulier* sur l'alphabet fini Σ . $\square$

2.6 Algèbre de Kleene

L'algèbre de Kleene a été introduite afin de répondre à une question posée par Stephen Cole Kleene qui se demandait alors s'il existait une axiomatisation intègre et complète de la théorie équationnelle de l'algèbre des ensembles réguliers.

Dans cette section, nous présentons l'axiomatisation de Kozen [28] car elle est considérée la plus générale, dans le sens où toutes les autres axiomatisations intègres et complètes de la théorie équationnelle des ensembles réguliers forment des sous-classes d'algèbres de l'algèbre de Kleene.

2.37 Définition. Soit K un ensemble non vide d'éléments ; soient $+$ et $\cdot$ deux opérations binaires sur K , $*$ une opération unaire sur K , et 0 et 1 deux constantes de K . Une algèbre de Kleene est une structure algébrique $\langle K, +, \cdot, *, 0, 1 \rangle$ telle que :

- $\langle K, +, \cdot, 0, 1 \rangle$ est un demi-anneau idempotent, c'est-à-dire une structure algébrique qui respecte les axiomes suivants :

$$\begin{array}{ll}
 a + (b + c) &= (a + b) + c & a \cdot 0 = 0 \cdot a &= 0 \\
 a + 0 &= a & a \cdot 1 &= a \\
 a + b &= b + a & 1 \cdot a &= a \\
 a + a &= a & a \cdot (b \cdot c) &= (a \cdot b) \cdot c \\
 a \cdot (b + c) &= a \cdot b + a \cdot c & (a + b) \cdot c &= a \cdot c + b \cdot c
 \end{array}$$

pour tout $a, b, c \in K$.

- $\langle K, +, \cdot, *, 0, 1 \rangle$ satisfait les formules suivantes :

$$\begin{array}{ll}
 1 + a \cdot a^* &\leq a^* \\
 1 + a^* \cdot a &\leq a^* \\
 b + a \cdot x \leq x &\rightarrow a^* \cdot b \leq x \\
 b + x \cdot a \leq x &\rightarrow b \cdot a^* \leq x
 \end{array}$$

pour tout $a, b, x \in K$, et où $\leq$ est l'ordre partiel naturel sur le demi-anneau idempotent $\langle K, +, \cdot, 0, 1 \rangle$, c'est-à-dire que $a \leq b \leftrightarrow a + b = b$.

□

En analyse de programmes, l'algèbre de Kleene fournit un cadre théorique intéressant qui a aussi bien été utilisé pour prouver l'équivalence de programmes que pour prouver la correction des optimisations de code réalisées à la compilation. Plus précisément, l'attrait principal de cette algèbre vient du fait que cette algèbre permet d'abstraire facilement les programmes informatiques : ainsi, il est généralement convenu que la constante 0 correspond au programme *halt*, que la constante 1 correspond au programme *skip*, que l'opérateur binaire $+$ correspond à un choix non déterministe entre deux programmes, que l'opérateur binaire $\cdot$ correspond à la séquence de programmes et que l'opérateur unaire $*$ correspond à l'itération sur des programmes. Cela devient d'autant plus intéressant lorsque l'on considère une sous-classe des algèbres de Kleene qui contient des tests atomiques, car il est possible de représenter les programmes *while*. On peut alors définir une simple fonction de traduction permettant de traduire des instructions de programmes en termes algébriques :

$$\begin{array}{ll}
 \lceil p ; q \rceil &= p \cdot q \\
 \lceil \text{if } b \text{ then } p \text{ else } q \rceil &= b \cdot p + \bar{b} \cdot q \\
 \lceil \text{if } b \text{ then } p \rceil &= b \cdot p + \bar{b} \\
 \lceil \text{while } b \text{ do } p \rceil &= (b \cdot p)^* \cdot \bar{b}
 \end{array}$$

où p, q sont des actions atomiques, b est un test atomique et $\bar{b}$ est la négation du test b . Par exemple, avec cette fonction de traduction, le programme

**if b then begin $p; q$ end
else begin $p; r$ end**

se traduit en algèbre de Kleene avec tests par le terme $(b \cdot p \cdot q + \bar{b} \cdot p \cdot r)$.

Une fois des programmes traduits en termes algébriques, il est alors possible d'utiliser les règles de l'algèbre pour prouver que deux termes sont égaux, et donc que les programmes qu'ils abstraient sont équivalents. Considérons l'égalité suivante (dans l'exemple qui suit, pour simplifier la notation, nous notons $a \cdot b$ par ab) :

$$(bc + \bar{b}\bar{c})(bpq + \bar{b}pr) = (bc + \bar{b}\bar{c})p(cq + \bar{c}r)$$

Sous l'hypothèse que " $pc = cp$ ", on peut prouver que cette égalité est valide en réduisant au même terme ses termes de gauche et de droite :

$$\begin{array}{ll} (bc + \bar{b}\bar{c})(bpq + \bar{b}pr) & (bc + \bar{b}\bar{c})p(cq + \bar{c}r) \\ = \langle \text{Distributivité de } \cdot \text{ sur } + \rangle & = \langle \text{Distributivité de } \cdot \text{ sur } + \rangle \\ bcbpq + \bar{b}\bar{c}bpq + bc\bar{b}pr + \bar{b}\bar{c}\bar{b}pr & bcpcq + \bar{b}\bar{c}pcq + bcp\bar{c}q + \bar{b}\bar{c}p\bar{c}r \\ = \langle \text{Commutativité des tests} \rangle & = \langle \text{Hypothèse } pc = cp \rangle \\ bbcpq + \bar{b}\bar{c}bpq + \bar{b}\bar{c}bpr + \bar{b}\bar{c}\bar{b}pr & bccpq + \bar{b}\bar{c}cpq + bc\bar{c}pq + \bar{b}\bar{c}\bar{c}pr \\ = \langle \text{Axiomes } a \cdot a = a \text{ et } a \cdot \bar{a} = 0 \rangle & = \langle \text{Axiomes } a \cdot a = a \text{ et } a \cdot \bar{a} = 0 \rangle \\ bcpq + \bar{b}\bar{c}pr & bcpq + \bar{b}\bar{c}pr \end{array}$$

2.7 Conclusion

La base théorique de tout langage de programmation repose sur la théorie des langages. Nous avons énoncé dans ce chapitre les notions fondamentales de la théorie, sans lesquelles, il serait difficile d'assimiler le processus de compilation des programmes, et par conséquent les méthodes utilisées pour l'atteinte de l'objectif fixé. Dans le chapitre suivant, nous détaillerons encore plus les techniques pratiques couramment utilisées dans la conception des compilateurs.

Chapitre 3

Principes et techniques de compilation

Les langages de programmation sont un pont entre la pensée humaine et l'automatisation des machines. Le rôle premier d'un programme informatique est d'indiquer à la machine la ligne de conduite à suivre pour aboutir aux résultats escomptés. Or le seul langage reconnu par les ordinateurs se compose seulement de 0 et de 1, ce qui n'est pas vraiment facile pour décrire un comportement complexe. Par conséquent, avant qu'un programme informatique ne puisse être exécuté par un ordinateur, une traduction vers le langage machine s'impose. Cette transformation des programmes d'un niveau d'abstraction supérieur vers un langage d'un niveau aussi bas que le langage machine, s'appelle *la compilation*.

Dans ce chapitre nous allons présenter les différentes phases par lesquelles passe un programme écrit dans un langage de haut niveau pour atteindre une allure assimilable par un ordinateur. Cependant, étant donné que la complexité du processus de compilation est considérable, nous aborderons seulement les étapes qui serviront à atteindre l'objectif annoncé, à savoir l'analyse lexicale et l'analyse syntaxique. À la fin de ce chapitre, nous présenterons également les alternatives actuelles pour l'élaboration des “constructeurs d'analyseurs syntaxiques”.

3.1 Compilation de programmes

La compilation est la transformation d'un programme rédigé dans un langage de programmation, le langage *source*, vers un programme équivalent écrit dans un autre langage, le langage *cible*. Les outils responsables de cette traduction sont les *compilateurs*. La multiplicité des langages de programmation a naturellement entraîné l'accroissement,

autant du nombre de compilateurs, que leur diversité. En effet, il existe des compilateurs spécifiques à un langage bien déterminé, des compilateurs optimisés pour certaines machines, d'autres dédiés à des systèmes d'exploitation bien définis ou encore à des domaines particuliers. Néanmoins, ils partagent tous plus ou moins la même structure interne pour le traitement des langages de programmation basés sur les notions théoriques énoncées dans le chapitre précédent.

Nous pouvons distinguer essentiellement deux parties communes à tous les compilateurs : une partie d'analyse, appelée *partie frontale*, et une partie de synthèse, appelée *partie finale*. Généralement, les compilateurs incluent d'autres parties d'optimisation à différents niveaux ; cela dépend encore une fois du langage cible pour lequel ils ont été conçus, ou encore de la plate-forme sur laquelle ils seront utilisés.

La figure 3.1 présente les étapes types que l'on peut trouver dans la plupart des compilateurs.

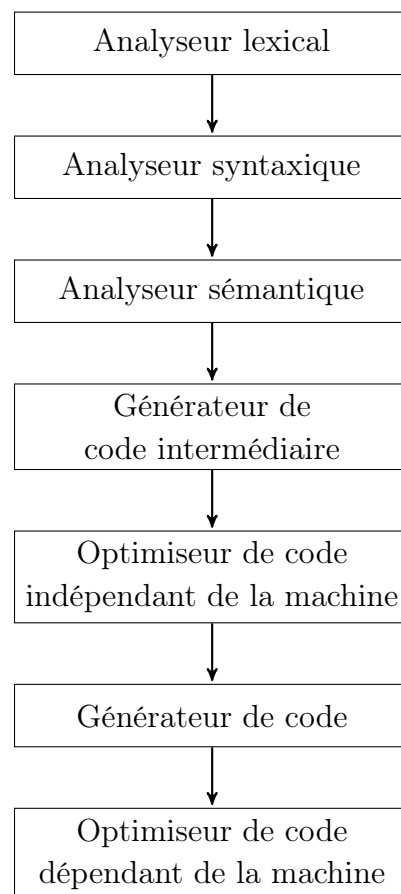


FIGURE 3.1 – Étapes de compilation

La première phase à laquelle est confronté le programme source est l'analyse lexicale. Une des tâches de cette étape est de déceler les éventuelles erreurs lexicales (fautes d'orthographe par exemple) au sein du programme, en vérifiant que chaque mot est

correct vis-à-vis du langage de programmation concerné. En revanche, la vocation première de l'analyse lexicale est de lire le flot de caractères constituant le programme et de les regrouper en séquence appelée *lexèmes*. Ces lexèmes sont ensuite classés sous la forme d'unité lexicale, et répertoriés dans une structure partagée par les différents étages de l'analyseur appelé *Table de symboles*. Cette table sera un élément crucial pour la deuxième phase d'analyse, l'analyse syntaxique.

L'analyseur syntaxique utilise la table de symboles ainsi que la définition des règles de la grammaire régissant le langage de programmation, pour construire une structure dans laquelle est répertorié l'enchaînement et la logique du programme. Généralement, l'issue de cette étape d'analyse débouche sur la construction d'un arbre syntaxique abstrait¹ représentant la structure grammaticale de l'ensemble des unités lexicales. Chaque noeud de l'arbre correspond à une opération, et ses fils, représentent les arguments de cette opération.

L'étape d'analyse sémantique se base sur l'arbre syntaxique et la table de symboles pour vérifier la cohérence des propos du programme source par rapport aux spécifications sémantiques du langage. Autrement formulé, lors de cette étape, le compilateur s'assure que l'enchaînement des instructions du programme est logique et définit un comportement correct à l'exécution. Par exemple, le *contrôle de types* est une technique parmi d'autres, appliquée pendant cette phase d'analyse, pour s'assurer que le programme n'entraîne aucun dysfonctionnement de la machine sur laquelle il sera exécuté. Les trois phases précédemment citées constituent la partie frontale du compilateur.

La partie finale du processus de compilation débute par l'étape de production de code intermédiaire. Généralement, avant de passer à la production du code de bas niveau, les compilateurs construisent une représentation intermédiaire du programme. Cette représentation a l'avantage d'être facile à produire et facile à traduire vers le langage machine, deux critères essentiels pour apporter des optimisations sur le code sans en altérer le comportement.

La phase d'optimisation du code s'occupe d'apporter des améliorations au programme afin d'augmenter les performances lors de l'exécution. C'est dans cette phase du processus que le compilateur vérifie s'il peut optimiser l'allocation de la mémoire, ou encore éliminer les instructions superflues...

1. "AST : Abstract Syntaxe Tree" en anglais.

Finalement, l'étape de production du code s'occupe de réécrire la représentation intermédiaire du programme en un programme écrit dans le langage cible.

Étant donné que l'objectif à atteindre requiert des notions d'analyses lexicale et syntaxique, nous présentons essentiellement ces deux techniques de la partie frontale de la compilation.

3.2 Analyse lexicale

L'analyseur lexical peut inclure une étape préliminaire effectuée par un *scanneur*, qui permet de simplifier le processus d'analyse. Le scanneur effectue des tâches simples telles que la suppression des commentaires, la réduction d'une succession de blancs à un seul espacement...

Par la suite, la phase d'analyse lexicale proprement dite peut commencer ; elle prend le code délivré par le scanneur et produit une suite d'*unités lexicales*. Une unité lexicale est un couple constitué du nom de l'unité lexicale et d'une valeur d'attribut. Le nom de l'unité lexicale représente un type d'unité lexicale (mot clé, suite de caractère d'entrée dénotant un identificateur...). Il faudrait faire la différence entre unité lexicale et *lexème*. En effet, un lexème est une séquence de caractères dans le programme source qui est reconnue par le motif d'une unité lexicale et identifié par l'analyseur lexical comme une instance de cette unité lexicale.

Supposons que le programme source contienne une instruction qui peut s'écrire sous la forme `printf("total=%d\n", score)` ; dans cette séquence les mots `printf` et `score` sont des lexèmes reconnus par l'unité lexicale **id** , par contre le lexème `"total=%d\n"` est reconnu par l'unité lexicale **chaîne**.

Reconnaissance des unités lexicales

La définition des unités lexicales dans une grammaire donnée est fournie en utilisant les expressions régulières. Par exemple, pour définir l'unité lexicale **id** qui représente des identifiants dans un programme source, nous pouvons recourir aux règles suivantes

au sein de la grammaire du langage avec lequel est écrit le programme :

$$\begin{aligned} \text{chiffre} &\rightarrow [\mathbf{0} - \mathbf{9}] \\ \text{chiffres} &\rightarrow \text{chiffre}^+ \\ \text{lettre} &\rightarrow [\mathbf{A} - \mathbf{Za} - \mathbf{z}] \\ \text{id} &\rightarrow \text{lettre}(\text{lettre} \mid \text{chiffres})^* \end{aligned}$$

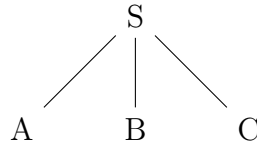
Ces expressions définissent un identifiant *id* comme étant la concaténation d'une lettre, représentée par l'entité *lettre*, avec une ou plusieurs lettres ou chiffres. De même, une entité *lettre* est en fait une combinaison des lettres de *a* à *z* minuscules et majuscules confondus, tout comme une entité *chiffre* est une combinaison des chiffres de 0 à 9. L'utilisation des symboles tels que $+$, $*$, permet de renseigner sur les règles utilisées dans les expressions régulières. Par exemple l'utilisation du symbole $+$ indique que l'unité lexicale à côté de laquelle il est placé peut être répétée une ou plusieurs fois.

L'analyseur lexical utilise la grammaire pour construire un diagramme de transitions pour chaque unité lexicale qui la compose. De cette manière, lors de la lecture d'une séquence d'entrée, tel qu'une phrase ou une ligne de code, les lexèmes de cette séquence sont confrontés au fur et à mesure de la lecture aux états des différents diagrammes de transitions des unités lexicales. Ainsi, lors de la fin de la lecture des symboles qui forment le lexème reçu en entrée, l'unité lexicale qui représente le lexème sera celle dont le diagramme de transitions se trouve dans un état d'acceptation.

3.3 Analyse syntaxique

Tel qu'énoncé dans le chapitre précédent, une grammaire sert à la génération de langages. Pour obtenir un mot spécifique du langage, il suffit de le dériver à partir de l'axiome en remplaçant les non terminaux par la partie droite des productions qui les définissent respectivement, et cela de manière répétée jusqu'à obtenir le mot en question. L'analyse syntaxique utilise le même procédé pour déterminer les dérivations de toutes les constructions grammaticales dans un programme, et de les représenter sous forme d'arbre de dérivation.

Supposons que la grammaire du langage contienne une production de l'axiome définie comme suit : $S \rightarrow A B C$; l'arbre correspondant obtenu par la phase d'analyse syntaxique aura pour sommet le noeud *S* ayant trois fils non terminaux *A*, *B*, *C*. Chaque non-terminal aura ainsi des fils découlant de la production correspondante dans la grammaire, jusqu'à atteindre les feuilles de l'arbre qui sont constitués exclusivement des éléments terminaux de la grammaire.



Dans le cas d'une grammaire non contextuelle, l'arbre produit par la phase d'analyse syntaxique aura les propriétés suivantes :

- la racine de l'arbre est toujours l'axiome, définie par le non terminal S ,
- les feuilles de l'arbre sont les éléments terminaux de la grammaire,
- chaque noeud interne de l'arbre est un non terminal,
- pour tout noeud X de l'arbre, ayant des fils étiquetés de gauche à droite $X_1, X_2 \cdots X_n$, il existe dans la grammaire une production de la forme $X \rightarrow X_1, X_2 \cdots X_n$ où les noeuds fils peuvent être des terminaux ou des non terminaux.

Pour simplifier, on peut dire que l'analyseur syntaxique vérifie que le code source du programme utilise les unités lexicales pour former des séquences correctement formulées selon la grammaire du langage. Afin de valider un programme, l'analyseur entreprend de construire l'arbre de dérivation de tout le programme, et ainsi s'assurer qu'aucune règle grammaticale n'a été violée.

Par analogie au langage naturel, nous pouvons naïvement énoncer que : la phase d'analyse lexicale constitue une "vérification de l'orthographe du programme", et la phase d'analyse syntaxique représente une "vérification grammaticale du programme".

Nous pouvons distinguer principalement deux approches du processus d'analyse syntaxique : l'analyse syntaxique descendante, qui se base sur les dérivations à gauche pour retrouver les productions appliquées à partir de l'axiome jusqu'à parvenir à la reconnaissance de la séquence à vérifier ; et l'analyse syntaxique ascendante, qui part de la séquence des mots à analyser et essaie de retrouver les règles de production qui ont été appliquées pour remonter à l'axiome.

3.3.1 Analyse syntaxique descendante

Les analyseurs syntaxiques par approche descendante se classent majoritairement selon deux sortes : l'analyse syntaxique par descente récursive et l'analyse syntaxique $LL(k)$.

Analyse syntaxique par descente récursive

Cette sorte d'analyseur, comme son nom l'indique, utilise un algorithme récursif pour retrouver les différentes dérivations en se basant sur la chaîne d'entrée. Une procédure est associée à chaque élément non terminal de la grammaire. L'analyse commence par la procédure de l'axiome et se termine avec succès si l'ensemble de la chaîne a été parcouru. Pour chaque production, la procédure qui lui est associée lira un lexème de la chaîne d'entrée ou fera un appel récursif d'une autre procédure. Par exemple, si la grammaire contient une production de la forme

$$\langle \text{if-stm} \rangle :: \text{if } \langle \text{Exp} \rangle \text{ then } \langle \text{Stms} \rangle \text{ end}$$

l'analyseur comportera une procédure pour le non-terminal $\langle \text{if-stm} \rangle$, qui commencera par lire le lexème "**if**", ensuite fera un appel à la procédure qui traite le non-terminal $\langle \text{Exp} \rangle$; après elle procédera à la lecture du lexème "**then**" par la suite il y aura un appel à la procédure qui traite le non-terminal $\langle \text{Stms} \rangle$ et finalement la lecture du lexème "**end**". De toutes les approches d'analyse, celle-ci est la plus évidente, et la plus facile à implémenter manuellement, par conséquent nous ne nous attarderons pas plus sur cette approche.

Analyse syntaxique $LL(k)$

La signification de l'appellation de ce processus d'analyse provient de ses caractéristiques de parcours de la séquence d'entrée, et du type de dérivation qu'il utilise : le parcours de la chaîne d'entrée se fait de gauche à droite², et les dérivations employées sont des dérivation à gauche³. Le chiffre k qui se trouve entre les parenthèses indique le nombre de symboles de pré-vision utilisés à chaque étape de l'analyse syntaxique pour prendre les décisions sur les actions à effectuer.

La classe des grammaires qui peuvent être analysées par ce type de processus est assez riche pour couvrir la plupart des constructions syntaxiques des langages de programmation. Cependant, les grammaires ambiguës ou récursives à gauche ne peuvent être de type $LL(1)$.

Pour qu'une grammaire G puisse être traitée par un analyseur syntaxique $LL(1)$, il faut qu'elle satisfasse les conditions suivantes :

1. pour toutes paires de productions distinctes $A \rightarrow \alpha \mid \beta$ de G , il n'y a aucun terminal a tel que α et β dérivent toutes deux des chaînes commençant par a ;
2. α et β ne peuvent être dérivées toutes deux de la chaîne vide ;

2. "*Left to right*" en anglais, d'où le premier L .

3. "*Leftmost derivation*" en anglais, ce qui explique le deuxième L .

3. si $\beta \Rightarrow^* \epsilon$, alors α ne dérive aucune chaîne commençant par un terminal qui peut apparaître directement à droite de A . De même si $\alpha \Rightarrow^* \epsilon$, alors β ne dérive aucune chaîne commençant par un terminal qui peut apparaître directement à droite de A .

Dès lors, nous pouvons voir que ces conditions sont suffisantes pour annoncer que toute grammaire de type LL est une grammaire hors contexte.

Dans le chapitre précédant, nous avons admis l'hypothèse que le langage généré par une grammaire hors contexte est reconnu par un automate à pile. Nous pouvons prendre avantage de cette faculté pour établir une table d'analyse qui aura pour rôle d'optimiser grandement le processus d'analyse syntaxique en comparaison de l'approche récursive. En effet, cette table sert d'index pour déterminer directement si la séquence d'entrée est valide ou pas en fonction du symbole en lecture, des symboles (de nombre k) en pré-vision et de l'état de la pile de l'automate.

Tables d'analyse LL

La table d'analyse $LL(1)$ est construite de la sorte :

- les rangées de la table sont étiquetées par les symboles non terminaux de la grammaire ;
- les étiquettes des colonnes sont les symboles terminaux de la grammaire ainsi qu'un symbole spécial **Fin**. Ce dernier représente un marqueur de fin de chaîne ;
- l'entrée (X, y) de la table indique l'action à faire lorsque le symbole non terminal X est au sommet de la pile et que le prochain symbole à lire est un y (incluant **Fin**). Si le non terminal X doit être remplacé par le côté droit d'une règle de la grammaire, le côté droit de cette règle constitue l'entrée (X, y) ; autrement, l'entrée est le mot **erreur**.

Prenons l'exemple de la grammaire suivante :

$$\begin{array}{l|l} p_1 : & S \rightarrow zMNz \\ p_2 : & M \rightarrow aMa \\ p_3 : & M \rightarrow z \\ p_4 : & N \rightarrow bNb \\ p_5 : & N \rightarrow z \end{array}$$

La table d'analyse $LL(1)$ correspondante est présentée par le tableau suivant :

	a	b	c	Fin
S	erreur	erreur	$zMNz$	erreur
M	aMa	erreur	z	erreur
N	erreur	bNb	z	erreur

- L'entrée (S, a) est **erreur** car le symbole S peut seulement être remplacé par le côté droit $zMNz$, ce qui amène un z au sommet de la pile. Ce z peut être dépilé seulement si le prochain symbole à lire est z .
- L'entrée (M, a) est aMa , car c'est la seule manière d'amener un a au sommet de la pile.
- Puisque la grammaire n'a pas de règle- ϵ (tel que $S \rightarrow \epsilon$, $M \rightarrow \epsilon$ ou $N \rightarrow \epsilon$), on a toujours **erreur** dans chacune des entrées de la colonne étiquetée **Fin**. En effet, s'il y a un symbole non terminal au sommet de la pile alors que la séquence d'entrée est toute lue, cela constitue une erreur puisque le remplacement du symbole non terminal par le côté droit d'une règle introduit un symbole terminal sur la pile. Ce symbole terminal ne pourra être dépilé, puisque la séquence d'entrée est toute lue ; en conséquence, l'automate ne pourra atteindre l'état final.
- Etc.

Voyons à présent comment la séquence $zazazz$ est reconnue par le processus d'analyse $LL(1)$ en utilisant la table d'analyse :

	Pile	Symbole	À lire
1	S	z	$zazazz$ Fin
2	$zMNz$	z	$zazazz$ Fin
3	MNz	a	$zazazz$ Fin
4	$aMaNz$	a	$zazazz$ Fin
5	$MaNz$	z	azz Fin
6	$zaNz$	z	azz Fin
7	aNz	a	zz Fin
8	Nz	z	z Fin
9	zz	z	z Fin
10	z	z	Fin
11		Fin	

On peut remarquer que l'acceptation de la séquence $zazazz$ correspond à la dérivation à gauche suivante :

$$S \xRightarrow{p_1} zMNz \xRightarrow{p_2} zaMaNz \xRightarrow{p_3} zazaNz \xRightarrow{p_5} zazazz$$

3.3.2 Analyse syntaxique ascendante

Contrairement à l'approche descendante, la construction de l'arbre d'analyse dans l'approche ascendante⁴, commence par les feuilles en remontant jusqu'à la racine de l'arbre pour une chaîne d'entrée donnée. Ce type d'analyse est le plus répandu

4. *Bottom-up Parsing* en anglais.

parmi les analyseurs syntaxiques des langages de programmation notamment parce que, comparativement à l'approche précédente, celle-ci permet d'obtenir des analyseurs plus puissants.

La complexité de ce type d'analyse ne nous permet malheureusement pas de nous approfondir dans les détails de cette approche, c'est pourquoi nous ne présenterons que les notions suffisantes pour assimiler l'idée générale de cette approche d'analyse. Pour les mêmes raisons, nous ne présentons pas le processus de création des tables d'analyse pour cette approche ; par contre ce procédé est très bien décrit dans [14].

Analyse syntaxique $LR(k)$

L'appellation de cette classe d'analyseurs syntaxiques, tout comme l'approche précédente, est motivée par les caractéristiques de lecture de la chaîne d'entrée et du type des dérivations utilisées. En effet, de même que les analyseurs LL , la première lettre indique que le sens de lecture de la séquence d'entrée se fait de la gauche vers la droite⁵. Par contre, l'utilisation des dérivations à droite justifie la lettre R dans cette appellation⁶. Le nombre k indique la quantité de symboles devant être consultés, sans lecture, à partir de la séquence d'entrée pour pouvoir prendre une décision de la production à appliquer.

Nous allons présenter un style général d'analyse ascendante connu sous le nom d'analyse syntaxique par transfert-réduction (ou décalage-réduction). L'idée est de construire un automate à pile non déterministe (voir 2.14) qui représente le processus d'analyse d'une grammaire. Si nous reprenons la grammaire précédente, la construction de l'automate en question se déroule en 5 étapes :

1. l'automate comporte quatre états : i, p, q, f . L'état initial est i et le seul état final est f ;
2. l'ajout des transitions $(i, \epsilon, \epsilon; p, \#)$ et $(q, \epsilon, \#; f, \epsilon)$ est nécessaire pour marquer le fond de la pile lors de l'analyse ;
3. pour chaque terminal x de l'alphabet, la transition $(p, x, \epsilon; p, x)$ est rajoutée. Ces transitions sont appelées des *transferts* puisque chacune d'elles équivaut au transfert sur la pile du symbole à l'entrée ;
4. pour chaque règle de la forme $N \rightarrow w$, nous ajoutons la transition $(p, \epsilon, w; p, N)$. Ces transitions sont appelées des *réductions*, car elles permettent de réduire une séquence de symboles (w) à un seul symbole (N).

5. "Left to right" en anglais.

6. "Rightmost derivation" en anglais.

Si la grammaire compte des productions ϵ , tel que $N \rightarrow \epsilon$, nous avons la transition $(p, \epsilon, \epsilon; p, N)$;

5. nous ajoutons la transition $(p, \epsilon, S; q, \epsilon)$, où S est l'axiome de la grammaire.

Voici à présent la suite de configurations qui valide l'acceptation de la chaîne $zazabzbz$:

	État	Pile	À lire
0	i		$zazabzbz$
1	p	$\#$	$zazabzbz$
2	p	$\#z$	$azabzbz$
3	p	$\#za$	$zabzbz$
4	p	$\#zaz$	$abzbz$
5	p	$\#zaM$	$abzbz$
6	p	$\#zaMa$	$bzbz$
7	p	$\#zM$	$bzbz$
8	p	$\#zMb$	zbz
9	p	$\#zMbz$	bz
10	p	$\#zMbN$	bz
11	p	$\#zMbNb$	z
12	p	$\#zMN$	z
13	p	$\#zMNz$	
14	p	$\#S$	
15	q	$\#$	
16	f		

On constate que l'acceptation de la séquence $zazabzbz$ correspond à la dérivation à droite suivante :

$$S \xRightarrow{p_1} zMNz \xRightarrow{p_4} zMbNbz \xRightarrow{p_5} zMbzbz \xRightarrow{p_2} zaMabzbz \xRightarrow{p_3} zazabzbz$$

La construction de la table d'analyse LR est un processus suffisamment fastidieux pour être réalisé manuellement. Dans la pratique, la construction des tables d'analyse est déléguée à des programmes constructeurs d'analyseurs syntaxiques.

Dans la section suivante, nous présentons quelques constructeurs d'analyseurs syntaxiques, notamment *Lex/Yacc*, le plus populaire et le plus ancien d'entre eux.

Analyse syntaxique SLR et $LALR(k)$

Les analyseurs syntaxiques fondés sur l'approche LR présentent de meilleures performances lors de l'analyse syntaxique d'une grammaire comparativement à l'approche

LL. Cependant, ils sont pratiquement non exploitables à cause de la taille importante des tables d'analyses qu'ils utilisent. Les approches *SLR*⁷ et *LALR(k)*⁸ permettent de réduire la taille de la table utilisée par l'automate pour optimiser l'analyse. *LALR* offre les mêmes performances que *LR* [13] mais en utilisant une table beaucoup plus réduite ; c'est la raison pour laquelle il se trouve à la base de beaucoup de constructeurs d'analyseurs syntaxiques tel que *Yacc* ou *GnuBison*.

Une fois la table d'analyse *LR(0)* construite, l'optimisation se fait différemment selon l'approche utilisée : l'approche *SLR* examine la définition BNF de la grammaire pour minimiser la table d'analyse ; par contre, l'approche *LALR* scrute l'automate construit en fonction du nombre *k* de jetons à lire pour minimiser le nombre d'états, et ainsi obtenir une table d'analyse optimale. Bien qu'elle soit plus compliquée à mettre en oeuvre, la deuxième technique permet d'obtenir de meilleurs résultats [23].

À titre d'exemple, pour la grammaire du langage C, l'automate construit avec l'analyseur syntaxique utilisant l'approche *LR* compte plus de 10,000 états ; en revanche, en utilisant la technique d'analyse *LALR* ce nombre est réduit à seulement 350 états.

3.4 Constructeurs d'analyseurs syntaxiques

La construction d'un analyseur syntaxique n'est pas une tâche aisée, surtout si le langage à analyser est généré par une grammaire non contextuelle déterministe (i.e grammaire BNF), ce qui est généralement le cas. Habituellement, la phase de conception d'un langage de programmation débouche sur une définition de la grammaire sous la forme BNF, qui est une forme de grammaire non contextuelle. Ensuite, il faut bâtir un analyseur syntaxique capable de traiter les programmes (script ou autre forme de commandes) exprimés dans ce langage. Il faudrait noter que l'analyse syntaxique n'est qu'une phase préliminaire à l'interprétation des programmes écrits dans le nouveau langage, et ce serait une perte de temps de consacrer plus d'efforts sur l'analyse que sur la sémantique et l'exécution des programmes.

Pour remédier à cette situation, des constructeurs d'analyseurs syntaxiques ont été conçus pour apporter une économie d'effort et un gain de temps considérable. La démarche à suivre est semblable pour tous les constructeurs d'analyseurs syntaxiques : le constructeur part d'une définition formelle de la grammaire du langage à analyser, et génère un nouveau programme (écrit dans un langage de programmation tel que C ou Java ou autre) qui s'occupera de l'analyse syntaxique et permettra au programmeur de se concentrer sur les actions à donner aux instructions déchiffrées dans le programme à

7. Simple LR Parsing.

8. Look Ahead LR Parsing.

analyser [8].

Nous présentons trois des constructeurs d’analyseurs syntaxiques les plus utilisés : Lex/Yacc, ANTLR et GoldParser.

3.4.1 Lex/Yacc

Le constructeur d’analyseurs syntaxiques Yacc⁹ est le plus ancien dans son genre. Il a été développé dans les laboratoires d’AT&T en 1979 par Stephen C. Johnson. Inclus par défaut dans les systèmes UNIX, il est utilisé pour la création d’analyseurs syntaxiques en utilisant le langage de programmation C et C++. En fait, le terme Lex/Yacc fait référence à deux programmes distincts, chacun s’occupe du développement d’une partie du processus d’analyse. Le premier programme est l’analyseur lexical “Lex”. Il génère un nouveau programme “lex.yy.c” et utilise un automate fini déterministe pour l’analyse lexicale. Le deuxième composant, “Yacc”, génère un autre programme nommé “y.tab.c” qui implémente un analyseur syntaxique basé sur l’approche LALR. La combinaison des deux programmes .c obtenus constitue le système d’analyse syntaxique entier. La figure 3.2 illustre le fonctionnement des deux programmes pour la génération de l’analyseur syntaxique.

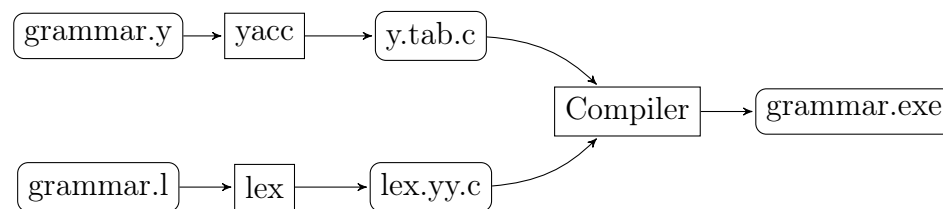


FIGURE 3.2 – Diagramme de flux de Lex/Yacc.

Les fichiers de définition des grammaires dans la syntaxe de Yacc/Lex contiennent des définitions, des règles ainsi que des instructions directement en langage C. Le fichier `grammaire.l`, destiné à l’analyseur lexical Lex, contient la déclaration des ensembles de caractères qui servent à définir chaque élément terminal de la grammaire. Par exemple, le terminal `digit`, qui décrit les entiers est déclaré par l’expression régulière suivante : `digit [0-9]` ; de même, le terminal représentant une lettre de l’alphabet est déclaré sous la forme `letter [A-Za-z]`.

La deuxième partie du fichier destiné à l’analyseur syntaxique contient les définitions de chaque terminal sous forme d’expressions régulières, suivi éventuellement d’instructions en langage C destinées à l’usage de l’analyseur syntaxique. Par exemple,

9. Yet Another Compiler-Compiler.

lors de la lecture d'une séquence de caractères alphabétiques, le lexème ID est renvoyé à l'analyseur. Cette définition est représentée dans le fichier `grammar.1` par : `{letter}{id_tail}*{return(ID) ;}`.

Les fichiers destinés à l'analyseur syntaxique ont une extension `.y`. Le fichier `grammaire.y` débute par la déclaration des types utilisateurs, s'il y a lieu, ensuite la déclaration de l'axiome (le symbole non terminal initial) de la grammaire et les identifiants qui représentent les lexèmes. La seconde partie de ce fichier liste les règles de production de la grammaire, avec les instructions en langage C qui seront exécutées lors de la réduction de la règle. Par exemple, la production suivante, décrivant une instruction `while`, invoque la fonction `whileStmt` lorsque la règle est réduite :

```
WHILE exp DO stats END { $$=whileStmt($3,$5) ;}
```

3.4.2 ANTLR

Le constructeur d'analyseurs syntaxiques ANTLR¹⁰ utilise le paradigme objet pour la génération d'analyseurs syntaxiques. Contrairement à Yacc, ANTLR est capable de générer des analyseurs syntaxiques pour plusieurs langages de programmation dont la syntaxe est relativement semblable à celle du C++, tel que Java, C# et évidemment C++.

L'approche de cet outil est différente de celle de Yacc dans la mesure où ANTLR engendre des classes `Java` au lieu d'utiliser des fichiers `.c`. Les classes `parser`, `lexer` et `treeParser` sont créées en se basant sur le pseudolangage de définition de grammaire, et sont ensuite héritées par les classes écrites par les développeurs pour représenter les actions de la grammaire.

La classe `lexer` fait office d'analyseur lexical ; elle s'occupe de la lecture du code source et de la production des lexèmes. Les classes `parser` et `treeParser` utilisent toutes les deux l'approche *LL* pour l'analyse syntaxique proprement dite. Elles offrent les mêmes fonctionnalités d'analyse, la différence réside dans les structures de données qu'elles admettent en entrée : la classe `parser` s'utilise pour analyser une suite de lexèmes, telle qu'un programme classique, tandis que la classe `parserTree` permet d'analyser une structure abstraite sous forme d'un arbre. ANTLR est parmi peu d'analyseurs qui permettent l'analyse de structures de données sous forme d'arbre.

En revanche, l'utilisation de l'approche d'analyse syntaxique *LL(k)* implique que les productions ne peuvent contenir des dérivations à gauche.

La syntaxe de la grammaire utilisée par ANTLR se base sur l'utilisation des majuscules pour faire la différence entre éléments terminaux et non-terminaux. Les éléments

10. Another Tool for Language Recognition.

terminaux commencent par une lettre majuscule, alors que les éléments non-terminaux de la grammaire commencent par une lettre minuscule.

Par exemple, la syntaxe `: Identifier : ('a'..'z')+` ; déclare un élément terminal noté *Identifier*, tandis que la syntaxe `identifier : (Identifier)+` ; indique que l'élément non terminal *identifier* est composé d'une série d'éléments terminaux *Identifier*.

3.4.3 GoldParser

L'objectif derrière la conception de GoldParser [9] est de créer un système d'analyse syntaxique qui peut être écrit en plusieurs langages de programmation, indépendamment de la grammaire à analyser. Dans cette optique il n'est pas concevable d'introduire des instructions dédiées à l'analyseur dans la définition de la grammaire, tel qu'il est d'usage dans les constructeurs d'analyseurs syntaxiques présentés précédemment (Lex/Yacc et ANTLR). Ainsi, le système consiste à utiliser deux composants différents : le constructeur de la table d'analyse et le moteur de génération de l'analyseur syntaxique.

Le constructeur de la table d'analyse¹¹ est la pierre angulaire du système. Il s'occupe d'analyser la description de la grammaire et de la génération de la table d'analyse LALR ainsi que des différents états de l'automate. Les données calculées lors de cette phase sont stockées dans un fichier `grammaire.cgt`, représentant le résultat de la "compilation" de la grammaire¹².

Le moteur de génération de l'analyseur syntaxique¹³ utilise le fichier produit par la phase précédente pour engendrer l'analyseur syntaxique correspondant à la grammaire décrite. Le programme du moteur peut être écrit dans n'importe quel langage de programmation (C, C++, Java, C#...), d'où la libération de la contrainte de dépendance entre grammaire et langage de programmation.



FIGURE 3.3 – Les deux composants et le fichier partagé.

Par exemple, il est possible d'avoir un moteur de constructeurs d'analyseurs syntaxiques développé en Python, qui engendre un analyseur syntaxique pour analyser un code source écrit en langage C.

La figure 3.3 présente la chaîne d'interactions entre les deux composants du système. La

11. Composant portant le nom de Gold Builder.

12. L'extension `.cgt` fait référence à *Compiled Grammar Tables*.

13. Composant portant le nom de Gold Engine.

syntaxe de la grammaire de description (où le GOLD meta-langage) qui est servie en entrée au constructeur de la table d'analyse se doit de respecter les critères suivants :

- la grammaire de description décrit seulement la syntaxe à analyser et ne doit pas contenir d'instructions dépendantes d'un langage de programmation spécifique ;
- la notation doit être claire et doit respecter les normes de la théorie des langages. Par conséquent, la définition des éléments terminaux est assurée par l'utilisation des expressions régulières et la définition des règles de production utilise la notation Backus-Naur Form ;
- la syntaxe contient des méta-données, destinées au constructeur, qui permettent de déterminer certaines propriétés du langage décrit telles que la sensibilité à la casse, le nom de l'auteur ;

Voici à présent les étapes suivies pour l'analyse d'un code source écrit dans un langage donné :

1. la première étape pour la construction d'un compilateur est d'écrire la grammaire du langage qui sera implanté. Du moment que la syntaxe de la grammaire ne demande aucun encodage spécifique, il est possible d'utiliser n'importe quel éditeur de texte ou encore l'éditeur inclus dans le constructeur ;
2. une fois la grammaire terminée, elle est analysée par le constructeur pour vérifier s'il n'y a pas d'ambiguïtés ou autre problème de conception. Durant cette phase, les tables d'analyse LALR et l'automate fini sont construits ;
3. lors de la fin de la phase d'analyse, les tables sont enregistrées dans un fichier qui sera utilisé ultérieurement par l'analyseur syntaxique. À ce stade, le travail du constructeur est achevé ;
4. les tables d'analyses sont acquises par l'analyseur syntaxique ; ce dernier peut être sous forme d'une DLL, d'un module “.Net” ou tout autre implémentation dans un autre langage de programmation ;
5. dans l'étape finale, le moteur procède à l'analyse du code source du texte et la génération de l'arbre syntaxique correspondant.

L'enchaînement de ce processus est résumé dans la figure 3.4.

3.4.4 Comparaison des pseudo-langages de description de grammaire

La syntaxe des pseudo-langages utilisée par les constructeurs d'analyseurs syntaxiques est un critère important à prendre en considération. Nous considérons une grammaire simple qui décrit les expressions arithmétiques et voyons sa définition avec les différents

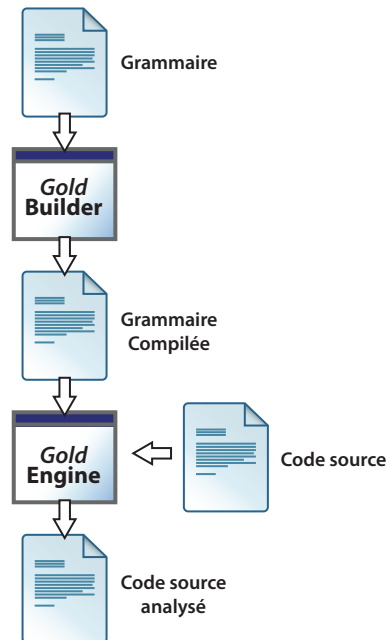


FIGURE 3.4 – Étapes d’analyse d’un code source.

constructeurs d’analyseurs syntaxiques. La grammaire en question permet la définition d’une expression arithmétique comme la somme, la multiplication ou la division de deux autres expressions arithmétiques. De plus, toujours selon la grammaire, une expression arithmétique peut prendre la forme d’un nombre, d’une variable ou d’une expression.

Pseudo-langage de GoldParser

Comme mentionné précédemment, la syntaxe utilisée par GoldParser pour décrire la grammaire n’est rien d’autre que sa définition BNF. La table 3.1 présente la syntaxe de la grammaire telle qu’utilisée par GoldParser. Nous pouvons remarquer que cette syntaxe comporte les méta-données qui renseignent sur le nom de la grammaire, le nom de l’auteur ainsi que des attributs, destinés au constructeur, concernant le symbole de départ et la sensibilité à la casse.

Pseudo-langage de Lex/Yacc

La syntaxe utilisée par Yacc ne diffère pas beaucoup de celle utilisée par GoldParser, du moment que les deux analyseurs utilisent l’algorithme LALR et la notation BNF pour

```

"Name" = 'Simple Grammar Example'
"Author" = 'Devin Cook'
"About" = 'This grammar defines simple mathematical expressions'
"Case Sensitive" = False "Start Symbol" = <Expression>
! ----- Terminals -----
{ID Tail} = {AlphaNumeric} + [_]
ID        = {Letter}{ID Tail}*
Number    = {Digit}+ ( '.' {Digit}+ )?
! ----- Rules -----
<Expression> ::= <Expression> '+' <Mult Exp>
               | <Expression> '-' <Mult Exp>
               | <Mult Exp>
<Mult Exp> ::= <Mult Exp> '*' <Negate Exp>
               | <Mult Exp> '/' <Negate Exp>
               | <Negate Exp>
<Negate Exp> ::= '-' <Value> | <Value>
<Value> ::= ID | Number | '(' <Expression> ')'

```

TABLE 3.1 – Grammaire des expressions arithmétiques en GoldParser.

la description des règles. Cependant, il est possible d'utiliser des instructions spécifiques dans la syntaxe de la grammaire de Yacc (telles que les mots-clefs `%left`, `%right`) destinées à l'analyseur, alors que la syntaxe de GoldParser s'en tient à la notation BNF. La syntaxe de Yacc peut contenir des règles explicites ou des règles implicites. La table 3.2 présente la version avec les règles implicites.

```

%start expression
%token ID NUMBER LPARAN RPARAN
%left PLUS MINUS
%left TIMES DIVIDE
%nonassoc UNARY_MINUS
expression : expression PLUS expression
           | expression MINUS expression
           | expression TIMES expression
           | expression DIVIDE expression
           | MINUS expression %prec UNARY_MINUS
           | ID
           | NUMBER
           | LPARAN expression RPARAN;

```

TABLE 3.2 – Syntaxe de la grammaire utilisant Yacc.

Pseudo-langage de ANTLR

L'approche de ANTLR pour décrire la grammaire des expressions arithmétiques est différente des deux pseudo-langages précédents. La définition de la grammaire passe tout d'abord par la création de deux nouvelles classes héritières des classes `Lexer` et `Parser`. Les éléments terminaux de la grammaire sont désignés en majuscule tandis que les expressions non terminales sont déclarées en minuscule, en utilisant les autres expressions et les éléments terminaux de la grammaire. La table 3.3 présente la grammaire des expressions arithmétiques selon la syntaxe de ANTLR. Nous pouvons y voir un exemple du mélange entre les instructions destinées à être utilisées par l'analyseur syntaxique final avec la définition de la grammaire. À titre d'exemple, dans la règle définissant les espacements (*Whitespace*), nous pouvons clairement voir que la procédure *newline()* sera utilisée lorsque l'analyseur rencontre un blanc (' '), une tabulation ('\t') ou encore un retour à la ligne ('\r \n').

```
// ——— Terminals ———
class TestLexer extends Lexer ;
tokens { PLUS = "+" ;
        MINUS = "-" ;
        TIMES = "*" ;
        DIVIDE = "/" ;
        LPARAN = "(" ;
        RPARAN = ")" ;
        }

// Whitespace
WS      : ( ' ' | '\t' | '\f'
          | ( "\r\n" | '\r' | '\n' )
          {newline(); }
          )
        { _ttype = Token.SKIP; };
ID      : ( 'a'..'z' | 'A'..'Z' )
        ( 'a'..'z' | 'A'..'Z' | '_' | '0'..'9' )* ;
NUMBER  : ( '0'..'9' )+ ( '.' ( '0'..'9' )+ )? ;

// ——— Rules ———
class TestParser extends Parser ;
expression : mult_exp ( ( PLUS | MINUS ) expression )* ;
mult_exp  : negate_exp ( ( TIMES | DIVIDE ) mult_exp )* ;
negate_exp : ( MINUS )? value
value      : ID | NUMBER | LPARAN expression RPARAN
```

TABLE 3.3 – Syntaxe de la grammaire utilisant ANTLR.

3.5 Conclusion

La compilation est le moyen de décrire, dans un langage assimilable par une machine, le comportement d'un programme écrit dans un langage de haut niveau. La complexité croissante des langages de programmation requiert la mise en oeuvre de techniques de compilation de plus en plus élaborées. C'est précisément dans ce contexte que l'utilisation des constructeurs d'analyseurs syntaxiques permet de faciliter grandement la tâche de conception des compilateurs.

Chapitre 4

Vérification de programmes

Les logiciels sont des programmes informatiques conçus pour répondre à un certain besoin, que ce soit dans un milieu professionnel ou domestique. Hormis les logiciels libres, dont le code source est disponible pour l'utilisateur final, la majorité des programmes informatiques commercialisés de nos jours le sont sous forme binaire que seules les machines peuvent interpréter. Cette forme de programmes “fermés” pose un problème de confiance pour l'utilisateur, qui se trouve dans l'obligation d'accepter les risques d'anomalies que peuvent comporter ces programmes. Les risques sont encore plus importants si du *code malicieux* est inclus dans les programmes [4, 12] ; des cas réels tels que des failles de sécurité ou encore des portes dérobées dans des logiciels, ont déjà causés beaucoup de tort par le passé [33, 42].

Dans le domaine de la recherche informatique, la vérification de programme propose des solutions pour parer à ce genre de situations, et pour établir un cadre de sécurité minimisant les préjudices introduits par des logiciels potentiellement dangereux.

Dans ce chapitre, nous allons survoler deux approches de vérification des programmes : les techniques d'analyse dynamique dont la particularité est de surveiller le comportement des programmes lors de l'exécution, et les techniques d'analyse statique qui eux, vérifient si un programme est conforme à une politique de sécurité avant de passer à son exécution. Le travail présenté dans ce document peut être classé dans la seconde approche.

4.1 Analyse dynamique de programmes

L'analyse dynamique consiste à surveiller le comportement d'un programme au moment même de son exécution. Ceci signifie que l'analyseur responsable de cette tâche

est à l'affût de chaque opération effectuée par le programme cible pour vérifier tout comportement suspicieux de la part de ce dernier. La surveillance est généralement accomplie par un autre programme informatique : l'analyseur dynamique, qui est désigné la plupart du temps par le terme "moniteur". La surveillance se résume à l'analyse de l'interaction du programme avec le noyau du système d'exploitation. C'est pourquoi nous pouvons classer les approches utilisées pour implémenter un analyseur dynamique selon le degré d'implication du moniteur. Nous distinguons trois approches pour la surveillance dynamique des programmes : la figure 4.1 résume ces trois approches, et représente les différents emplacements où le moniteur peut intervenir entre le noyau et le programme à surveiller.

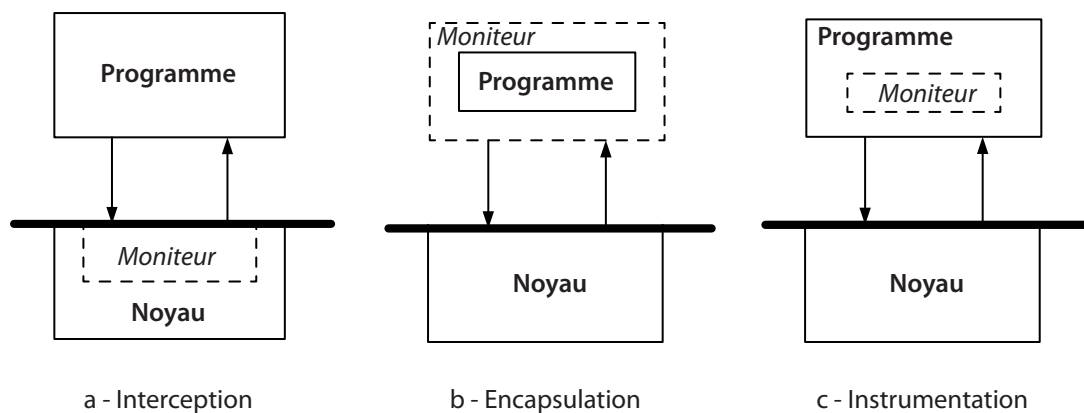


FIGURE 4.1 – Techniques de surveillance dynamique du code.

Dans la première approche, l'analyseur est à l'écoute des appels des fonctions du noyau invoqués par le programme. Une politique de sécurité, préalablement établie, permet à l'analyseur d'identifier si l'action demandée par le programme est légitime ou suspecte. Si l'action est prohibée par la politique de sécurité, le moniteur peut l'intercepter et lever une alerte de sécurité ou, dans les cas les plus critiques, arrêter l'exécution du programme. De cette manière, tout risque de préjudice au noyau est écarté. Par contre, l'inconvénient de cette technique est la lenteur qu'elle impose au programme cible [21].

L'encapsulation¹ est une approche qui consiste à remplacer les différentes fonctions système "API" par de nouvelles. Les fonctions système critiques qui sont utilisées par le programme sont encapsulées dans de nouvelles fonctions qui intègrent des tests spécifiques définis par la politique de sécurité. Ainsi, lors de l'exécution du programme, l'invocation des fonctions systèmes originales n'est autorisée que si les tests réussissent. Dans le cas contraire, cela veut dire que le programme ne respecte pas la politique de

1. *Wrapping* en anglais.

sécurité et l'exécution est arrêtée. Une utilisation de cette technique est décrite dans [43].

La troisième approche est l'instrumentation du programme. Cette technique consiste à enrichir le programme cible par des tests, ou toute autre action spécifique, de sorte que lors de l'exécution, la politique de sécurité est respectée. À la différence des deux autres techniques, cette approche présente la particularité d'inclure le moniteur au sein même du programme.

4.2 Analyse statique de programmes

L'approche statique se distingue par le fait que l'analyse porte sur le code source du programme plutôt que sur son exécution. Plusieurs techniques d'analyse peuvent être classifiées sous l'analyse statique de programmes, chacune se différencie par l'analyse d'un aspect particulier du code du programme. Par exemple, l'analyse par typage, consiste à représenter la sémantique d'un programme par un *système de types* ; l'analyse des flots s'intéresse au flot de données et au flot de contrôle du programme, ou encore l'analyse par évaluation de modèle qui analyse une abstraction du programme original. Par souci de concision, nous nous contentons de présenter les techniques d'analyse TAL, PCC et nous nous attardons un peu plus sur la technique d'analyse par évaluation de modèle.

4.2.1 TAL

La citation de Roert Milner “*well-typed programs cannot go wrong*” [34] résume bien l'idée derrière l'analyse par typage : il s'agit de s'assurer statiquement que le type des données reçues, manipulées et retournées par le programme reste valide pour toutes les instructions de ce dernier, et ainsi garantir l'absence de faille lors de l'exécution. Ce procédé d'analyse par typage permet d'assurer un nombre de propriétés [44], telles que :

- la sécurité de mémoire, qui est violée si le programme tente d'accéder à des zones mémoire qui ne lui sont pas allouées ;
- la sécurité de flot de contrôle, ce qui résulte par la certitude que le programme n'exécute que du code valide ;
- la préservation des abstractions, ce qui se traduit par le fait qu'un programme n'utilise un type de données que par son interface.

L'approche du *langage assembleur typé*² a été mise au point au sein de l'Université Cornell dans le cadre des recherches sur la conservation des informations de typage lors de la compilation [36]. Le langage assembleur typé offre la possibilité d'ajouter des assertions de vérification des types de données au sein de chaque mot machine du programme assembleur correspondant au programme source. Cette technique est composée de trois éléments qui régissent l'annotation du code assembleur original pour obtenir un programme assembleur typé : la sémantique statique qui spécifie la syntaxe des règles de typage, la sémantique dynamique qui énonce les règles d'évaluation et l'algorithme d'inférence qui permet de typer un programme en utilisant les règles de typage.

Syntaxe de TAL

Dans la syntaxe du langage assembleur typé, un programme est considéré comme une structure qui contient un tas (H), un fichier de registres (R) et une séquence d'instructions (S). Le détail de cette syntaxe est présenté dans la figure 4.2, on y retrouve la définition d'un programme comme étant un triplet composé du tas, du fichier de registre et de la séquence d'instructions. Le tas contient des correspondances entre les étiquettes (l) et les valeurs du tas (h), le fichier de registres énumère les associations entre les noms des registres (r) et les valeurs des mots (w), et la séquence d'instructions se détaille en une instructions (ι) suivie d'une nouvelle séquence d'instruction ou en un saut (jmp) ou finalement un arrêt du programme ($halt$). Les types du tas et du fichier de registres, permettent d'attribuer les types, respectivement, aux étiquettes (des blocs d'instructions) et aux registres.

Sémantique opérationnelle de TAL

Les règles de la sémantique opérationnelle de TAL définissent comment un programme s'exécute. Considérons un programme TAL défini par $P = (H, R, i_1; S)$, où i_1 est l'instruction de départ du programme et S une séquence d'instructions, la sémantique opérationnelle permet de réécrire le programme P en un programme $P' = (H', R', S)$. La sémantique opérationnelle permet ainsi de définir H' et R' qui sont les nouvelles versions du tas et du fichier de registres après l'exécution de l'instruction i_1 . La sémantique opérationnelle présentée par la figure 4.3 contient le détail des différentes instructions et les actions à entreprendre pour annoter un programme. À titre d'exemple,

2. Typed Assembly Language en anglais.

types	$\tau, \sigma ::= \alpha \mid \text{int} \mid \forall[\vec{\alpha}].\Gamma \mid < \tau_1^{\varphi_1}, \dots, \tau_n^{\varphi_n} > \mid \exists \alpha. \tau$
drapeaux d'initialisation	$\varphi ::= 0 \mid 1$
types de tas	$\Psi ::= \{l_1 : \tau_1, \dots, l_n : \tau_n\}$
types de fichiers des registres	$\Gamma ::= \{r_1 : \tau_1, \dots, r_n : \tau_n\}$
contexte de type	$\Delta ::= \alpha_1, \dots, \alpha_n$
registres	$r ::= r_1 \mid r_2 \mid \dots$
valeurs des mots	$w ::= l \mid i \mid ?\tau \mid w[\tau] \mid \text{pack } [\tau, w] \text{ as } \tau'$
petites valeurs	$v ::= r \mid w \mid v[\tau] \mid \text{pack } [\tau, v] \text{ as } \tau'$
valeurs de tas	$h ::= < w_1, \dots, w_n > \mid \text{code } [\vec{\alpha}]\Gamma.I$
tas	$H ::= \{l_1 \mapsto h_1, \dots, l_n \mapsto h_n\}$
fichiers de registres	$R ::= \{r_1 \mapsto w_1, \dots, r_n \mapsto w_n\}$
instructions	$\iota ::= \text{add } r_d, r_s, v \mid \text{bnz } r, v \mid \text{ld } r_d, r_s[i] \mid$ $\text{malloc } r_d[\vec{\tau}] \mid \text{mov } r_d, v \mid \text{mul } r_d, r_s, v \mid$ $\text{st } r_d[i], r_s \mid \text{sub } r_d, r_s, v \mid \text{unpack } [\alpha, r_d], v$
séquence d'instructions	$S ::= \iota; S \mid \text{jmp } v \mid \text{halt } [\tau]$
programmes	$P ::= (H, R, S)$

FIGURE 4.2 – Syntaxe de TAL.

l'instruction **ld** $r_d, r_s[i]$ charge le contenu de la $i^{\text{ème}}$ composante du tuple, dont l'adresse est stockée dans le registre r_s , dans le registre r_d . L'instruction **st** $r_d[i], r_s$ sauvegarde la valeur du registre r_s à la $i^{\text{ème}}$ position du tuple dont l'adresse est contenue dans le registre r_d . L'instruction **jump** v permet de faire un saut vers le bloc étiqueté v . L'instruction de branchement **bnz** r, v indique qu'il faut exécuter l'instruction suivante si la valeur du registre r est nulle, sinon il faudra faire un branchement vers le bloc dont l'étiquette est contenu dans la la valeur v . Finalement, l'instruction **unpack** $[\alpha, r_d], v$ où v est de la forme **pack** $[\tau', v']$, permet de substituer τ' par α dans la séquence d'instructions S' et de lier le registre r_d à la valeur v' .

$(H, R, S) \mapsto P$ où	
Si $S =$	alors $P =$
add $r_d, r_s, v; S'$ bnz $r, v; S'$ lorsque $R(r) = 0$	$[(H, R\{r_d \mapsto \hat{R}(r_s) + R(v)\}, S')]$ (H, R, S')
bnz $r, v; S'$ lorsque $R(r) = i$ et $i \neq 0$	$(H, R, S'[\tau/\alpha])$ où $\hat{R}(v) = l[\vec{\tau}]$ et $H(l) = \text{code}[\vec{\alpha}]\Gamma.S''$
jump v	$(H, R, S'[\vec{\tau}/\vec{\alpha}])$ où $\hat{R}(v) = l[\vec{\tau}]$ et $H(l) = \text{code}[\vec{\alpha}]\Gamma.S'$
ld $r_d, r_s[i]; S$	$(H, R\{r_d \mapsto w_i\}, S')$ où $R(r_s) = l$ et $H(l) = \langle w_0, \dots, w_{n-1} \rangle$ avec $0 \leq i < n$
malloc $r_d[\tau_1, \dots, \tau_n]; S'$	$(H\{l \mapsto \langle ?\tau_1, \dots, ?\tau_n \rangle\}, R\{r_d \mapsto l\}, S')$ où $l \notin H$
mov $r_d, v; S'$	$(H, R\{r_d \mapsto \hat{R}(v)\}, S')$
st $r_d[i], r_s; S'$	$(H\{I \mapsto \langle w_0, \dots, w_{i-1}, R(r_s), w_{i+1}, \dots, w_{n-1} \rangle\}, R, S')$ où $R(r_d) = l$ et $H(l) = \langle w_0, \dots, w_{n-1} \rangle$ avec $0 \leq i < n$
unpack $[\alpha, r_d], v; S'$	$(H, R\{r_d \mapsto w\}, S'[\tau/\alpha])$ où $\hat{R}(v) = \text{pack}[\tau, w]$ as τ'
où $\hat{R}(v) = \begin{cases} R(r) & \text{quand } v = r \\ w & \text{quand } v = w \\ \hat{R}(v')[\tau] & \text{quand } v = v'[\tau] \\ \text{pack}[\tau, \hat{R}(v')] \text{ as } \tau' & \text{quand } v = \text{pack}[\tau, v'] \text{ as } \tau' \end{cases}$	

FIGURE 4.3 – Sémantique opérationnelle de TAL.

4.2.2 PCC

Le code incorporant une preuve³ est une technique de certification de code développée à l'université Carnegie Mellon sous la direction du Professeur Peter Lee en collaboration avec George Necula de l'Université de Barkley [37]. Le principe de cette approche repose sur l'interaction entre le producteur, l'entité responsable de la production du code, et le consommateur, l'utilisateur final du code. L'idée est de permettre au consommateur de définir la politique de sécurité qu'il juge nécessaire à ses besoins et de la fournir au producteur. Ce dernier est capable alors de présenter au consommateur une preuve assurant que la politique de sécurité est respectée par le code fourni. Après vérification, le consommateur peut exécuter le code en toute sécurité. La particularité de cette technique, est que la preuve de fiabilité est intégrée dans le code lui-même, d'où le nom de *Proof Carrying Code* ce qui signifie code incorporant une preuve.

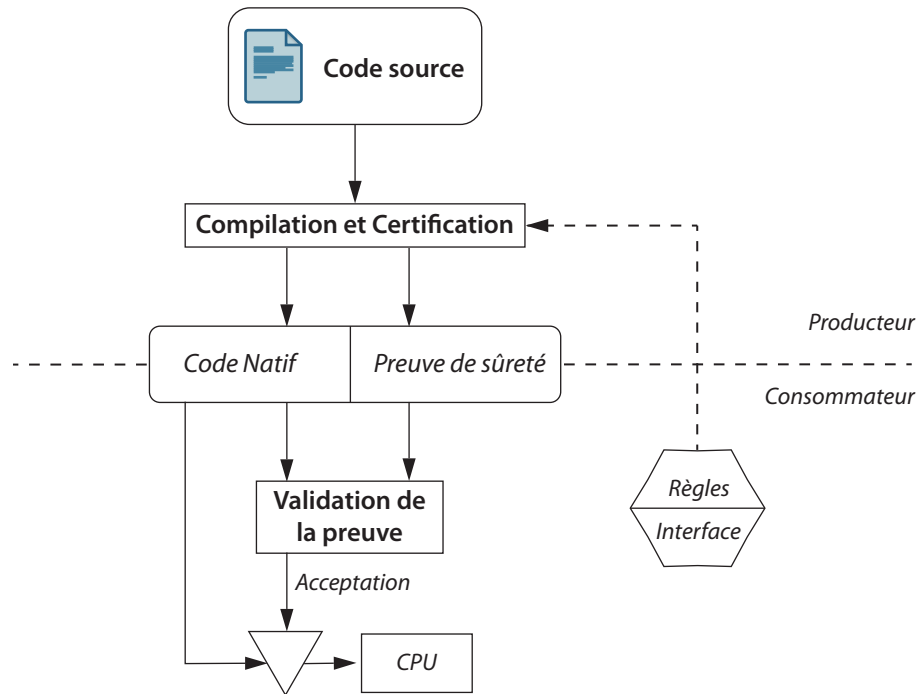


FIGURE 4.4 – Architecture de PCC.

3. Proof Carrying Code en anglais.

Politique de sécurité

La figure 4.4 présente l'architecture de PCC. Elle s'articule autour de la politique de sécurité dans la mesure où c'est une ressource utilisée à la fois par le consommateur et le producteur. Elle se compose de deux éléments : des règles de sûretés qui énoncent les pré-conditions associées à chaque opération permise, et une interface qui définit les invariants à respecter pour chaque appel de fonction. Entre autres, la politique de sécurité définit les conditions à respecter avant de terminer l'exécution sous forme de post-conditions. Concrètement, les conditions de sûreté consistent en un prédicat de logique de premier ordre selon la logique émise par Floyd [20]. Cette étape de négociation de la politique de sécurité entre le consommateur et le producteur est essentielle pour la réussite du processus PCC au complet.

Le cycle de compilation et de certification se déroule en trois étapes : *la certification*, *la validation* et finalement *l'exécution*.

La certification Lors de cette étape, le producteur compile et génère une preuve que le programme est bien conforme à la politique de sécurité fournie au préalable par le consommateur. La preuve est ensuite encodée et incorporée au code binaire PCC qui sera livré au consommateur.

La validation Le code PCC reçu par le consommateur contient le code natif et la preuve. Il suffit de confronter la preuve au code natif pour s'assurer de la validité de ce dernier vis-a-vis de la politique de sécurité définie au début du processus. Il est à noter que si la preuve a été modifiée, la validation ne sera pas effective. Par contre si des modifications sont apportées à la fois sur le code et la preuve de sorte qu'il satisfont toujours la politique de sécurité, dans ce cas la validation se déroulera normalement, même si le code final ne correspond plus aux exigences initiales du consommateur. Ceci dépeint une faiblesse de PCC dans la mesure où la validation ne vérifie que la conformité de la sécurité et non pas la correction du code.

L'exécution Une fois l'étape de validation dépassée avec succès, le consommateur peut alors exécuter le code en toute sécurité autant de fois qu'il le souhaite, sans avoir à le valider encore une fois.

Une implémentation de PCC basée sur la logique LF^4 est décrite dans [40]. Cette

4. Logical Frameworks en anglais.

plate-forme permet de décrire des règles lexicales ainsi que les règles d'inférence des langages de programmation sous forme de prédicats de premier ordre.

4.2.3 Analyse par évaluation de modèle

En reprenant la définition donnée dans [24] “Le model checking est une technique automatique, qui partant d'un automate fini modélisant un système et d'une propriété formelle, vérifie systématiquement si la propriété est respectée par (un état donné) le modèle du système”.

L'analyse par évaluation de modèle⁵ est une technique de vérification qui explore tous les états possibles d'un système en utilisant le procédé de “force brute”. De la même manière qu'un programme de jeu d'échec examine tous les coups possibles à jouer dans une partie, un vérificateur par évaluation de modèle analyse tous les scénarios possibles du système de manière automatique. Ainsi, il est possible de démontrer qu'un modèle du système donné vérifie une propriété particulière. Il est ainsi possible d'examiner tous les états d'un système aussi complexe qu'un processeur ou une mémoire. Les vérificateurs classiques peuvent examiner un nombre allant jusqu'à 10^9 états d'un système ; par contre en faisant appel à des algorithmes intelligents et des structures de données succinctes, un nombre beaucoup plus élevé d'états peuvent être sondés (de 10^{20} jusqu'à 10^{476}) et de cette manière, même les plus infimes erreurs qui n'ont pas été détectées par simulation ou avec les tests, peuvent être mises à nu avec la technique du *model checking*.

Les propriétés qui peuvent être vérifiées en utilisant l'analyse par évaluation de modèle sont typiquement qualitatives, qui peuvent être exprimées par des interrogations telles que “Est-ce que le résultat est correct ?”, “Est-ce que le système peut atteindre un état de blocage ?”. Il est aussi possible d'utiliser le *model checking* pour vérifier des propriétés à caractère temporel, formulées par exemple par les questions suivantes : “Est ce que le système peut tomber dans un état de blocage 1 heure après son déclenchement ?”, “Est-ce que la réponse est toujours fournie avant 8 minutes”.

La figure 4.5 présente l'architecture de vérification par évaluation de modèle. La modélisation du système à vérifier est déduite de sa spécification formelle de manière automatique, généralement rédigée dans une version simplifiée d'un langage de programmation comme le langage C ou Java. Il est important de mentionner que la spécification formelle décrit ce que doit et ne doit pas faire le système ; par contre le modèle décrit comment le système réagit. Le vérificateur examine tous les états possibles du système et peut déterminer s'ils valident la propriété en question et dans le cas contraire, le

5. Model Checking en anglais.

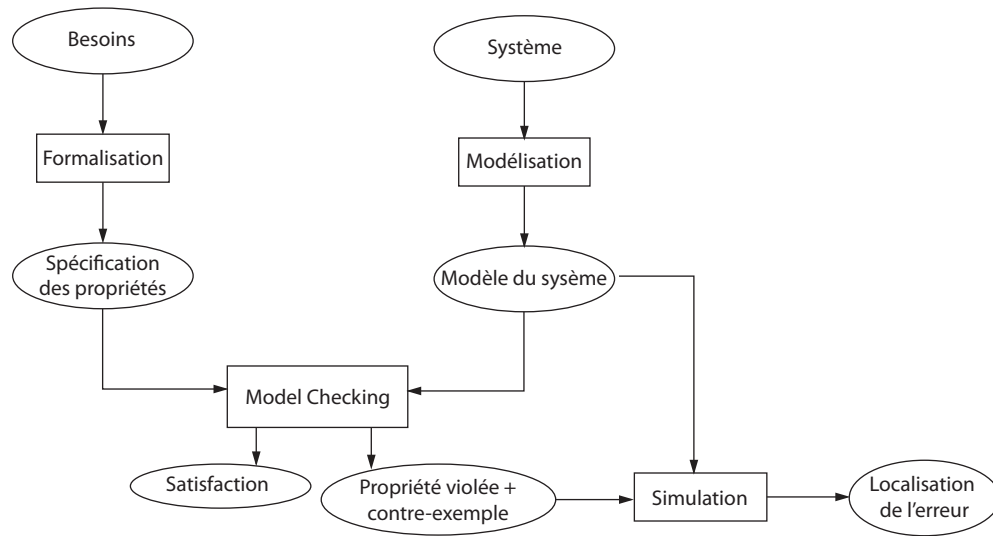


FIGURE 4.5 – Schématisation de l'approche du Model Checking.

vérificateur génère un contre-exemple qui montre l'état qui ne valide pas la propriété. Le contre-exemple fournit le chemin d'exécution qui part de l'état initial jusqu'à l'état qui ne respecte pas la propriété. De cette manière, l'utilisateur peut avoir des informations utiles pour rejouer le scénario non conforme, et d'adapter le modèle, ou la propriété.

Le model checking est assez répandu dans la vérification des systèmes d'informations. À titre d'exemple, cette technique a été utilisée pour découvrir qu'il y avait des boucles infinies dans des systèmes de réservation de billets d'avion. Les protocoles de commerce électroniques, et les protocoles de communication sont généralement vérifiés par évaluation de modèle, avant d'être utilisés dans le domaine public. Un autre exemple d'utilisation de cette méthode d'analyse est son application sur le module d'exécution du contrôleur de la sonde spatiale *Deep space 1*. Il en a résulté la découverte de cinq erreurs inédites, dont une critique [24].

Processus du model checking

La mise en oeuvre de l'analyse par évaluation de modèle passe par les trois phases suivantes :

- Phase de modélisation : il faut modéliser le système en question en utilisant le langage de modélisation reconnu par le vérificateur. Il est d'usage d'effectuer quelques tests préliminaires pour valider la fidélité du modèle vis-à-vis du système ;
- Phase d'exécution : exécution de l'analyseur pour vérifier la validité de la propriété

sur le modèle du système ;

- Phase d’analyse : si la propriété est validée, passer à la propriété suivante s’il y en a, sinon, s’il y a violation de la propriété, la démarche à suivre consiste à :
 1. utiliser la simulation pour analyser le contre-exemple généré par l’analyseur,
 2. raffiner le modèle ou la propriété,
 3. recommencer le processus en entier ;

S’il y a un dépassement de la capacité de mémoire de l’analyseur, il faudrait réduire la taille du modèle et recommencer le processus depuis le début.

Modélisation Le système est généralement décrit par un automate fini, permettant d’exprimer son comportement de façon précise et non ambiguë tout en restant le plus fidèle possible au comportement original du système. Ainsi chaque état de l’automate représente une étape de l’exécution du système, caractérisée par différents types d’informations telles que les valeurs des variables du système, l’instruction qui vient d’être exécutée... Par contre, les transitions de l’automate décrivent comment le système évolue d’un état vers l’état suivant. La description du modèle peut être écrite dans une grammaire dérivée des langages de programmation tels que C/C++, Java, VHDL ou autres.

Un premier essai du modèle permet d’éliminer les erreurs de conception simples, ainsi un gain de temps et de calcul peut être épargné. Pour une analyse rigoureuse, les propriétés doivent être rédigées de façon précise et non ambiguë, pour cela les logiques temporelles (LTL, CTL...) sont habituellement adoptées en raison de leur expressivité qui est la plus adaptée au domaine des systèmes d’informations. Les logiques temporelles sont une extension de la logique propositionnelle traditionnelle en ajoutant des opérateurs permettant de formuler des propriétés qui évoluent à travers le temps. Ceci permet la spécification d’un large éventail de propriétés tel que des propriétés d’exactitude (“Est-ce que le système fait ce qu’il est supposé faire?”), des propriétés d’accessibilité (“Est-ce qu’un état de blocage peut être atteint?”), des propriétés de sûreté (“Quelque chose de mal n’arrivera jamais”), des propriétés de vivacité (“Quelque chose de bien arrivera inévitablement”), des propriétés d’équité (“Est-ce que sous certaines conditions, un événement peut être observé indéfiniment”) et des propriétés de temps-réel (“Est-ce que le système répond à temps?”).

Exécution du vérificateur de modèle Le vérificateur de modèle doit d’abord être correctement initialisé par le réglage des options diverses et des directives qui peuvent être utilisées pour effectuer une vérification exhaustive. Par la suite, l’analyse par évaluation de modèle prend effectivement place. Il s’agit essentiellement d’une

approche purement algorithmique dans laquelle la validité de la propriété en question est vérifiée dans tous les états du modèle du système.

Analyse des résultats Il y a principalement trois résultats possibles : La propriété est validée par le vérificateur, la propriété n'est pas validée, le modèle est trop grand et dépasse la capacité physique de la mémoire de l'ordinateur.

Si la propriété a été validée, l'analyseur passe à la propriété suivante, et si toutes les propriétés ont été traitées avec succès, cela veut dire que le modèle est correct et qu'il respecte les propriétés nécessaires.

Dans la mesure où la propriété n'a pas été validée, les résultats négatifs peuvent avoir plusieurs origines. Il est possible qu'il y ait une erreur de modélisation, qui fait que le modèle n'est pas fidèle au système qu'il est sensé représenter. Dans ce cas, il faudrait corriger le modèle et recommencer le processus avec la nouvelle version du modèle corrigé. Il est évident qu'il faudrait reprendre la validation de la propriété qui a révélé l'invalidité. Si l'analyse montre qu'il n'y a pas d'écart entre le modèle et le système en question, et que malgré la nouvelle version du modèle, la propriété n'est toujours pas validée, il faudrait revoir la conception du système, ou revoir la définition de la propriété. Dans le cas où le modèle n'est pas en cause, et que la propriété n'est pas validée, cela signifie que cette dernière ne reflète pas les caractéristiques adéquates du système, et devra être reformulée.

Si le modèle est trop volumineux pour être stocké en mémoire, il faudrait appliquer des techniques qui permettent d'optimiser le modèle, tel que la modélisation par des diagrammes symboliques, ou encore la réduction basée sur l'ordre partiel. Idéalement une abstraction rigoureuse du système complet peut être appliquée, néanmoins il faut veiller à ce que cette abstraction n'altère pas les propriétés qui doivent faire l'objet de validation.

4.3 Conclusion

Cette brève description des techniques de validation des programmes permet de préciser le cadre dans lequel se situe le travail présenté par ce document. Dans le chapitre suivant, nous introduisons une approche algébrique pour l'analyse de programmes, qui sera utilisée pour la confrontation d'un programme vis-à-vis d'une politique de sécurité afin de garantir sa validité.

Chapitre 5

Technique de traduction

Cette section présente les techniques mises en oeuvre pour la traduction du code source d'un programme écrit en langage C vers une expression régulière. La transformation est réalisée suivant deux étapes : la traduction des expressions, qui permet de dénouer les expressions imbriquées ; et la traduction des instructions, qui a pour objectif de réécrire les instructions dans une syntaxe plus légère que la syntaxe originale. Nous commencerons par présenter le cadre algébrique dans lequel prendra place cette transformation, ensuite nous détaillerons les phases de la traduction proprement dite.

5.1 Approche algébrique

L'approche algébrique pour l'analyse de programmes découle de l'analyse statique par évaluation de modèles à la différence près de ramener l'étape de vérification à une simple résolution d'équations dans une algèbre donnée.

En effet pour déterminer si un modèle P satisfait une politique de sécurité représentée par la formule ϕ , il suffit de vérifier que l'inéquation

$$|P| \leq \|\phi\|$$

est valide, où $|P|$ représente la traduction du modèle et $\|\phi\|$ représente la traduction de la politique de sécurité.

En se référant à la méthode de résolution énoncée dans [31], l'algèbre $AL_{\mathcal{G}}$ qui y est définie est utilisée dans l'étape de vérification du modèle.

Cette méthode permet de partir de deux expressions de l'algèbre de Kleene représentant

respectivement le programme et la formule, et de retourner une trace de la résolution de l'inéquation, qui est la preuve formelle que le programme respecte, ou non, la politique de sécurité. La figure 5.1 résume globalement l'utilisation de cette approche algébrique.

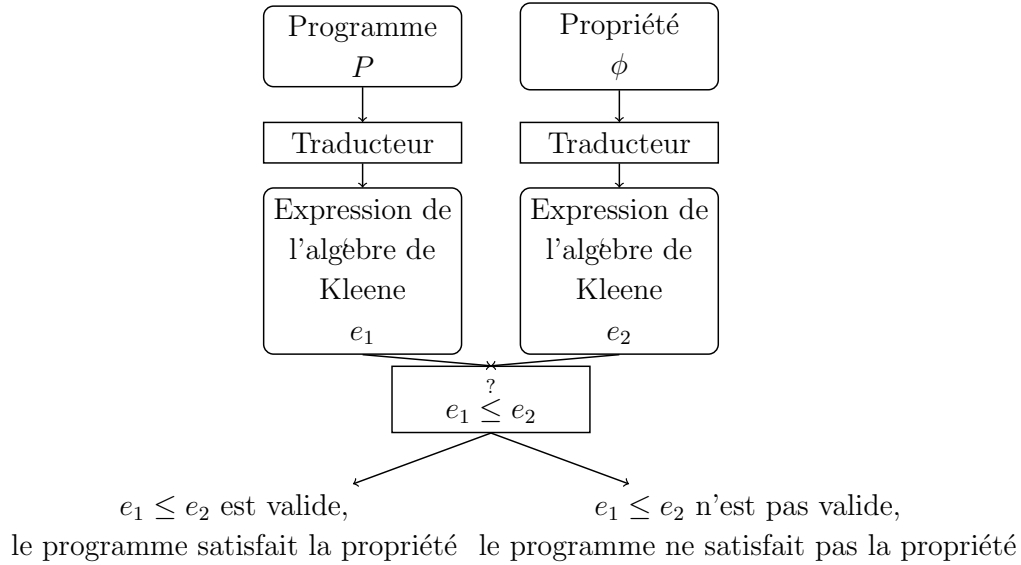


FIGURE 5.1 – Approche algébrique à la vérification de modèle.

Néanmoins, il reste à noter que cette méthode ne peut pas être utilisée directement pour la vérification d'un programme réel écrit dans un langage de programmation de haut niveau (tel que le langage C). Pour cela il faut passer par une phase de traduction du code source du programme pour aboutir au modèle équivalent sous forme d'une expression de l'algèbre de Kleene. Or il existe une méthode présentée par Kozen en [28] qui permet d'obtenir à partir d'un automate fini déterministe l'expression équivalente en algèbre de Kleene. Il ne reste donc qu'à décortiquer le code source du programme à vérifier et d'en dégager l'automate équivalent au comportement que l'on souhaite vérifier.

Nous proposons de réaliser un traducteur qui permettrait de partir d'un code source en langage C d'un programme réel, et d'aboutir à une expression régulière en Algèbre de Kleene qui sera utilisable par cette approche algébrique de vérification par évaluation de modèles. De ce fait, on peut situer notre travail dans un élan de concrétisation de cette méthode de résolution, en mettant en place une couche supplémentaire de traduction de programmes.

La traduction à réaliser se divise en deux parties plus ou moins complémentaires : premièrement, nous allons extraire une version bien particulière du flot de contrôle sous forme d'un automate fini déterministe, ensuite nous pourrions utiliser les résultats de Kozen [28] pour passer de la représentation de l'automate à une expression de l'algèbre

de Kleene.

Ainsi en ajoutant cette couche de traduction à la méthode de vérification, il est possible d'avoir une implantation complète d'un analyseur statique de programme en utilisant l'approche algébrique.

5.2 Transcription de programmes

Nous allons nous intéresser à des programmes écrit en langage C pour arriver à une modélisation sous forme d'un automate fini déterministe. Par contre, il suffit de considérer la BNF de ce langage (annexe B) pour se rendre compte que la tâche n'est pas si évidente. En effet, le langage C a la particularité d'être extrêmement expressif, ce qui fait sa popularité, et il n'est pas trivial d'extraire le comportement d'un programme directement à partir du code source sans de préalables modifications.

La syntaxe du langage C peut se diviser en trois parties : les instructions, les expressions et les déclarations. Le modèle que l'on cherche à bâtir repose principalement sur les fonctions utilisées par le programme ainsi que leur enchaînement, c'est pour cela que nous allons nous intéresser aux instructions et aux expressions qui représentent le mieux, dans notre cas, le comportement que l'on souhaite analyser.

Pour atteindre notre but, notre idée est de procéder en deux phases successives :

1. une réécriture du code source : ce qui va nous permettre d'éliminer plusieurs éléments de la syntaxe originale du langage pour avoir un programme plus linéaire et moins complexe ; ainsi, nous pouvons extraire facilement le comportement qu'on souhaite vérifier ;
2. une modélisation sous forme d'automate fini déterministe, facilement déductible d'un programme linéaire qui ne contient que des appels de fonctions et des instructions de saut.

5.2.1 Réécriture de programmes

Cette première phase sert à "linéariser" le comportement du programme en simplifiant les instructions et les expressions qui engendrent un comportement complexe lors de l'exécution. Autrement dit, cette phase permet d'avoir un programme sémantiquement équivalent au programme initial en utilisant une syntaxe beaucoup moins compliquée. Cette simplification consiste à remplacer les différentes instructions itératives (`while`, `do`, `for`) ainsi que les instructions de sélection (`switch`, `break`, `continue`) par une combinaison

de branchement (`if`, `else`) et de saut (`goto`). Bien qu'il ait été énoncé dans [15] que l'emploi de branchement est fortement déconseillé, l'utilisation que nous comptons en faire ne lève pas de contraintes puisque la sémantique des instructions à remplacer est préservée. La réécriture englobe aussi la simplification des expressions utilisées dans le programme pour éliminer les imbrications et le sucre syntaxique qui peuvent exister. Malgré la simplicité apparente de cette transformation, elle doit être bien pensée pour ne pas modifier la sémantique des expressions originales, d'autant plus que les expressions peuvent être imbriquées dans les instructions du programme. Cela signifie que si des erreurs se glissent lors de la transformation des expressions, cela affecterait la sémantique des instructions du programme réécrit.

Le processus de réécriture est effectué à travers un parcours en profondeur de l'arbre syntaxique du programme de manière récursive. Pour cela, nous utilisons un parseur (*GoldParser* [8] dans le cas présent) pour extraire l'arbre syntaxique initial du programme. La grammaire du langage **C** [25] stipule qu'un programme est constitué de plusieurs fonctions, et qu'une fonction est un enchaînement de déclarations, d'expressions et d'instructions. De ce fait, nous introduisons les fonctions $\mathcal{P}, \mathcal{F}, \mathcal{I}, \mathcal{E}$ qui produisent la réécriture respective d'un programme, d'une fonction, d'une instruction et d'une expression.

La réécriture se fait en un seul parcours de l'arbre et en appliquant au fur et à mesure les règles de transformations. Un environnement dynamique Γ est utilisé par ces fonctions lors de l'application des règles.

Nous définissons la syntaxe des fonctions de transformation comme suit :

$$| o |_{\mathcal{H}}^{\Gamma} = S$$

Avec :

- $\mathcal{H}$, la fonction représentant la transformation en cours, dans ce cas $\mathcal{P}, \mathcal{F}, \mathcal{I}$ ou $\mathcal{E}$ respectivement pour la transformation d'un programme, d'une fonction, d'une instruction ou d'une expression ;
- o , l'objet à réécrire (instruction, expression, fonction ou programme) ;
- Γ , l'environnement dynamique représentant le contexte de chaque règle ;
- S , une suite d'instructions/expressions résultantes de la transformation.

La hiérarchie des éléments du langage **C** est assez cohérente : un programme se compose de plusieurs fonctions ; une fonction est une succession d'instructions ; une instruction est une construction d'une ou plusieurs expressions ; et finalement, une expression est un assemblage des éléments terminaux de la syntaxe (lettres, chiffre, symboles...). Ainsi pour que le programme réécrit ait du sens, la transformation devra

tenir compte de cette logique. De ce fait, il faudrait commencer par transformer les expressions, ensuite transformer les instructions en employant la transformation des expressions qu'elles contiennent, pour finalement réécrire des fonctions en se basant sur la transformation des instructions qui les composent.

Contexte de transformation

Les règles de transformation permettront d'éliminer différentes instructions de la syntaxe originale du langage **C** tout en respectant la sémantique de ces dernières. Ainsi nous définissons une règle de transformation pour chaque instruction de la grammaire originale. Il est impératif de tenir compte du contexte lors de la transformation de certaines instructions pour aboutir à une sémantique équivalente. Pour cela, la fonction de transformation des instructions $\mathcal{I}$ fait intervenir un environnement Γ qui permet de stocker différentes informations en cours de transformation.

Dépendamment de l'élément à transformer (instruction, expression ou fonction) l'environnement sera utilisé pour stocker :

- les différentes étiquettes générées lors de la transformation des instructions **break**, **continue** et **return** (représentées respectivement par les étiquettes l_{break} , $l_{continue}$ et l_{return}) ;
- les expressions et les étiquettes résultantes lors de la transformation de l'instruction d'aiguillage **switch-case**.

L'environnement Γ permet d'associer des valeurs à des identificateurs. Par exemple, la syntaxe $\Gamma \uparrow [l_b \mapsto l_1]$ est utilisée pour associer la valeur l_1 à la variable l_b et l'ajouter à l'environnement Γ ; de même, la syntaxe $\Gamma[v_1 \mapsto 5, l_{break} \mapsto l_1]$ signifie que dans Γ , la variable v_1 est associée à la valeur 5 et la variable l_{break} à l'étiquette l_1 .

5.2.2 Transformation de programmes

Un programme **C** est une suite de fonctions utilisateurs avec une fonction d'amorçage **main()**. D'autres instructions peuvent exister dans un programme, comme les directives du préprocesseur, mais dans le cadre de la transformation souhaitée, on se limite à considérer un programme comme une suite de fonctions (on suppose donc que le programme source a déjà été traité par le préprocesseur). Ainsi la réécriture d'un programme se résume à la réécriture des différentes fonctions qui le composent. Si $\mathcal{P}$ est

la fonction de réécriture des programmes et $\mathcal{F}$ la fonction de réécriture des fonctions, alors la transformation d'un programme p composé de n fonctions serait la somme de la transformation des n fonctions qui le composent :

$$Si\ p ::= f_1(), \dots, f_n() \text{ Alors } |p|_{\mathcal{P}} = |f_1|_{\mathcal{F}}, \dots, |f_n|_{\mathcal{F}}$$

5.2.3 Transformation des fonctions

On prend pour référence la grammaire originale du langage **C** [25] pour tirer les définitions exactes des éléments à transformer. Ainsi, dans la BNF du langage **C**, une fonction est définie comme suit :

$$\begin{array}{l} f ::= \textit{SpecificateurDeclaration Declaration Instructions} \\ \quad | \quad \textit{Declareur Instructions} \end{array}$$

TABLE 5.1 – Définition d'une fonction en langage **C**.

Une fonction est principalement composée :

- d'une déclaration qui représente le nom de la fonction et ses paramètres ;
- d'un bloc d'instructions qui représente le corps de la fonction.

L'autre élément de la définition qui sera utile lors de la réécriture est le *spécificateur de déclaration* qui représente le type de la fonction. Cet élément est facultatif et il peut être omis dans le cas des fonctions ne retournant aucune valeur.

En effet, il est commun de rencontrer des fonctions sans type ou encore du type vide (**void**), ce qui implique qu'elles ne renvoient aucune valeur. Dans le cadre de ce travail nous ne considérons que deux sortes de fonctions : les fonctions ayant explicitement un type, donc celles qui retournent une valeur, et les fonctions sans type ou du type **void**, donc celles qui ne retournent aucune valeur.

La réécriture des fonctions se fait grâce à la fonction de réécriture $\mathcal{F}$. Le résultat est similaire à la fonction initiale à quelques instructions près.

Règles de transformation

Pour simplifier la définition d'une déclaration de fonction, nous adopterons la syntaxe définie dans le tableau 5.2.

$$\begin{array}{l}
 f ::= t \ f(p_1, p_2, \dots, p_n) \ s \\
 \quad | \quad f(p_1, p_2, \dots, p_n) \ s
 \end{array}$$

TABLE 5.2 – Syntaxe simplifiée de définition d’une fonction.

Dans cette syntaxe, t représente le type de la fonction (*spécificateur de déclaration*), f le nom de la fonction, $p_1, p_2, \dots, p_n$ les paramètres passés à la fonction et s est le corps de la fonction. Il est à remarquer que dans la syntaxe BNF du langage, le nom et les paramètres ainsi que leur type sont regroupés sous le non-terminal *declarateur*.

La réécriture d’une fonction ne prend pas de contexte de traduction ; par contre elle en crée un qui servira à la transformation du corps de la fonction.

Comme mentionné précédemment, on ne traite pas les cas particuliers des fonctions déclarées sans type, ou du type `void`, et qui retournent tout de même une valeur. Ainsi nous allons considérer comme équivalentes les fonctions déclarées sans type et les fonctions déclarées avec un type `void`.

Fonction sans type : La transformation de ces fonctions consiste tout simplement à créer un nouveau contexte dans lequel il faut réécrire le corps de la fonction.

$$| f(p_1, \dots, p_n) \ s |_{\mathcal{F}} = f(p_1, \dots, p_n) \mid s \mid_{\mathcal{I}}^{[]}$$

où $[]$ est un nouvel environnement dynamique vide.

Fonction avec un type : Dans le cas où la fonction retourne une valeur, la transformation consiste à ajouter la déclaration d’une variable `__xs` du même type que celui du résultat de la fonction et une instruction étiquetée “`L_Return : return __xs ;`” à la fin de la transformation du corps de la fonction. L’étiquette (fraîche) ajoutée à la fin de la fonction permettra d’avoir un seul point de sortie de la fonction réécrite. Ainsi, si le corps de la fonction contient une instruction `return` ou `exit`, la transformation consistera à rediriger l’exécution vers la fin de la fonction.

$$\begin{array}{l}
 | t \ f(p_1, \dots, p_n) \ s |_{\mathcal{F}} = t \ f(p_1, \dots, p_n) \\
 \quad \{ t \ __xs \ ; \\
 \quad | \ s \mid_{\mathcal{I}}^{[]} \\
 \quad \text{L_return : return } __xs \ ; \}
 \end{array}$$

Exemple de réécriture de fonction

Afin de donner un exemple concret de la réécriture des fonctions, nous allons réécrire le programme qui calcule la factorielle d'un entier en utilisant une fonction de calcul récursive. Le programme est composé de deux fonctions : une fonction **main** qui n'admet pas de type, et une fonction récursive **fact** qui prend un paramètre de type entier et retourne un entier. Cet exemple illustre à lui seul les deux sortes de transformation précédemment présentées.

```
1 main ()
2 { int i=10 ;
3   printf("fact de %d= %d", i, fact(i));
4 }
5
6 int fact(int i)
7 { if(i>0) return i*fact(i-1);
8   else return 1;
9 }
```

TABLE 5.3 – Programme de calcul du factoriel.

Nous pouvons observer que la fonction de réécriture des fonctions $\mathcal{F}$ n'a pas d'effet sur la fonction **main** puisque celle-ci ne retourne pas de valeur (table 5.4) . Par contre la transformation de la fonction **fact** consiste à déclarer la variable `__xs` et à ajouter l'instruction étiquetée "`l_Return : return __xs`".

5.2.4 Transformation des expressions

En étudiant la syntaxe du langage C, on peut clairement remarquer que les expressions ne peuvent être utilisées autrement que dans une instruction. Ceci se traduit par le fait que dans l'arbre syntaxique d'un programme, les expressions seront toujours les noeuds fils ou les feuilles des noeuds représentant les instructions du programme. En partant de cette constatation, il devient évident qu'il faudrait commencer par transformer les expressions pour utiliser ensuite le résultat de cette transformation lors de la réécriture des instructions. Ainsi la conséquence de la transformation d'une expression aura pour résultat une nouvelle expression et un bloc d'instructions qui sera utilisé lors de la transformation de l'instruction parente.

La signature de la fonction de transformation des expressions, notée $\mathcal{E}$, se présente

```

1  main ()
2  { int i=10;
3    printf ("fact de %d= %d",i,fact(i));
4  }
5
6  int fact (int i)
7  { int __xs;
8    { if (i >0)
9      { __xs=i*fact(i-1);
10       goto l_Return;
11     }
12     else
13     { __xs=1;
14       goto l_Return;
15     }
16   }
17   l_Return : return __xs;
18 }

```

TABLE 5.4 – Réécriture du programme de calcul du factoriel.

comme ceci :

$$|_{\varepsilon} : exp \rightarrow (exp \times B)$$

avec exp une expression et B un bloc d'instructions.

Il est à noter que lors de la transformation de plusieurs expressions imbriquées, plusieurs blocs d'instructions seront générés, et que le résultat final sera toujours une seule expression et un seul bloc résultant de la combinaison de tous les blocs obtenus lors des transformations intermédiaires. Nous détaillerons le processus de combinaison de ces blocs lors de la présentation des règles de transformations des expressions.

Syntaxe des expressions

Les règles décrivant les expressions dans la grammaire du langage **C** sont assez nombreuses ; par conséquent, par souci de clarté nous ne présentons qu'une abstraction de cette partie de la grammaire. Néanmoins, nous veillerons à ne pas altérer la hiérarchie établie des différents types d'expressions lors de cette abstraction. En effet, les différentes expressions binaires mettant en relation deux expressions et un opérateur binaire sont regroupées, dans cette abstraction, sous la règle *binary* qui n'existe pas

dans la grammaire originale. La priorité des opérateurs binaires n'interférant pas dans les transformations à effectuer, cette abstraction permettra ainsi d'obtenir une meilleure clarté et une concision des règles de transformations qui seront présentées.

La syntaxe abstraite des expressions est décrite dans le tableau 5.5. On peut facilement constater qu'une expression, simple ou compliquée, est le résultat d'une combinaison récursive des différents types de sous-expressions s'achevant par les ensembles de terminaux, dénotés en gras dans la syntaxe. Ces derniers sont des ensembles de règles élémentaires permettant de reconnaître les jetons terminaux de la grammaire. À titre d'exemple, l'ensemble **Id** définit la syntaxe d'un identifiant tel qu'il est reconnu par le parseur, qui se compose d'un entête composé obligatoirement d'une lettre ou du caractère `_`, suivi d'une ou plusieurs lettres ou chiffre.

$$\begin{aligned} \{Id\ Head\} &= \{Letter\} + [_] \\ \{Id\ Tail\} &= \{Id\ Head\} + \{Digit\} \\ Id &= \{Id\ Head\}\{Id\ Tail\}^* \end{aligned}$$

Il en est de même pour l'autre élément terminal, **Literal**, qui dans le cadre de cette syntaxe abstraite, représente les éléments terminaux du langage **C** comme les chaînes de caractères (**StringLiteral**) ou les entiers (**DecLiteral**) ou encore les nombres hexadécimaux (**HexLiteral**). Cette abstraction est utile dans la mesure où le traitement des différents littéraux par la fonction de traduction des expressions est exactement le même.

Une expression e en langage **C** se présente, soit sous la forme d'une expression d'affectation (*assignment*) soit une expression suivie d'une virgule et d'une affectation. Cette dernière se décline aussi soit en une expression conditionnelle (*conditional*), soit en une expression unaire (*unary*) suivie d'un opérateur et d'une autre affectation.

Une expression conditionnelle peut être soit une expression binaire (*binary*), soit une expression binaire suivie du symbole `" ? "` suivie d'une expression e suivie du symbole `" : "` et une expression conditionnelle pour finir.

Une expression binaire, quant à elle, est soit une expression de forçage de type (*cast*) soit une opération binaire suivie d'un opérateur et d'une expression de forçage de type.

Une expression de forçage de type est soit une expression unaire soit le nom d'un type (*typeName* liste des type définit dans la BNF [25]) compris entre deux parenthèses.

De même, une expression unaire est soit une expression post-fixée (*postfix*), soit une expression unaire précédée d'un opérateur unaire, de l'un des opérateur `++`, `--`, `sizeof`, ou encore d'une expression de forçage de type précédée d'un opérateur unaire.

Une expression post-fixée peut prendre plusieurs formes :

```

primary ::= Literal | Id( e ) | Id( ) | Id | ( e )
postfix ::= primary | postfix[ e ] | postfix( ) | postfix(argList)
          | postfix . Id | postfix -> Id | postfix ++ | postfix --
argList  ::= assignment | argList , assignment
unary    ::= postfix | ++ unary | -- unary | unaryOp cast
          | sizeof unary | sizeof(typeName)
unaryOp  ::= & | * | + | - | ~ | !
cast     ::= unary | (typeName)
binary   ::= cast | binary binaryOp cast
binaryOp ::= * | / | % | + | - | << | >> | < | > | >= | <= | == | !=
          | & | ^ | | | && | ||
conditional ::= binary | binary ? e : conditional
assignment ::= conditional | unary assignmentOp assignment
AssignmentOp ::= = | *= | /= | %= | += | -= | >>= | <<= | &= | |= | ^=
e ::= assignment | e , assignment

```

TABLE 5.5 – Syntaxe abstraite des expressions du langage C.

- une expression primaire ;
- une expression post-fixée suivie de deux crochets et une expression (tel est le cas des expressions désignant un tableau) ;
- une expression post-fixée suivie de deux parenthèses, représentant généralement une fonction ;
- une expression post-fixée suivie de deux parenthèses et une liste d'arguments ;
- une expression post-fixée suivie d'un point et d'un identifiant, ou une flèche et un identifiant ; ces deux cas sont rencontrés en général avec l'utilisation des structures ;
- une expression post-fixée suivie de l'un des deux opérateurs ++ ou --.

Finalement, une expression primaire (*primary*), qui est le type de sous expressions qui se trouvent en bas de la hiérarchie des expressions, éventuellement sous la forme

- d'un littéral ;
- d'un identifiant ;
- d'un identifiant suivi d'une expression comprise entre parenthèses ;
- d'un identifiant suivi d'une paire de parenthèses ;
- ou d'une expression comprise entre parenthèses.

Imbrication des blocs d'instructions

Comme nous l'avons mentionné précédemment, la transformation d'une expression conduit à la production de deux éléments : une nouvelle expression et un bloc d'instructions. La technique utilisée pour la simplification des expressions est semblable à celle utilisée dans le code à trois adresses [2]. Dans ce dernier, une instruction complexe telle que $x + y * z$ est transformée en une suite d'instructions à trois adresses comme la suivante :

$$\begin{aligned} t_1 &= y * z \\ t_2 &= x + t_1 \end{aligned}$$

où t_1 et t_2 sont des noms temporaires inventés par le compilateur. Pour la simplification des expressions, une expression composée de plusieurs sous-expressions est transformée en plusieurs instructions chacune ne contenant qu'une sous-expression de la syntaxe originale. Ces instructions générées lors de la transformation représentent le bloc d'instructions produit par la transformation d'une expression.

Dans le cas de la transformation d'une expression complexe, il faut gérer les blocs d'instructions générés par la traduction des expressions imbriquées pour aboutir au résultat final d'une expression et d'un seul bloc d'instructions.

Pour cela nous introduisons la notion de concaténation des blocs d'instructions, représentée par l'opérateur $\bullet$, qui consiste à créer un nouveau bloc d'instructions à partir des instructions de deux blocs distincts. La concaténation de deux blocs B_1 et B_2 résulte en un nouveau bloc constitué des instructions du premier bloc suivi des instructions du deuxième bloc.

$$B_1 = \begin{array}{l} \text{“ } instr_1; \\ \quad \quad \quad instr_2; \end{array} \quad \text{”} \quad B_2 = \begin{array}{l} \text{“ } instr_3; \\ \quad \quad \quad instr_4; \end{array} \quad \text{”}$$

$$B_1 \bullet B_2 = \begin{array}{l} \text{“ } instr_1; \\ \quad \quad \quad instr_2; \\ \quad \quad \quad instr_3; \\ \quad \quad \quad instr_4; \end{array} \quad \text{”}$$

Les instructions obtenues lors du développement des expressions seront entremêlées avec celles résultantes de la transformation des instructions parentes afin de conserver le comportement original du programme.

Règles de traduction

Les expressions à transformer sont celles qui sont susceptibles de contenir des appels de fonctions. Parmi celles-ci on peut clairement dégager les expressions de la forme $Id()$ ou encore les expressions se présentant sous la forme $Id(e)$ dans lesquelles l'expression e peut contenir un autre appel de fonction.

Il est utile de mentionner que la transformation à apporter consiste en une réécriture de certaines expressions énoncées dans le tableau 5.5 de la syntaxe des expressions du langage C. Ainsi les règles présentées ci-dessous permettent d'obtenir une décomposition élémentaires des expressions les plus pertinentes de la syntaxe abstraite.

Identifiant : Étant un ensemble d'éléments terminaux de la syntaxe, la transformation d'une expression de la forme Id reste inchangée et ne génère pas d'instructions intermédiaires. Cette règle sera utilisée lors de la transformation d'une variable, ou le nom d'une fonction dans une expression.

$$| \text{Id} |_{\mathcal{E}} = \langle \text{Id}, \emptyset \rangle$$

Littéral : De la même manière que pour les identifiants, les littéraux sont un ensemble d'éléments terminaux et leur traduction n'engendre pas d'instructions intermédiaires. Les littéraux comprennent les chaînes de caractères, les nombres (décimaux ou hexadécimaux)...

$$| \text{Literal} |_{\mathcal{E}} = \langle \text{Literal}, \emptyset \rangle$$

Appel de fonction : La transformation de cette expression consiste à stocker le résultat de l'appel de fonction dans une variable intermédiaire v_1 et de remplacer l'expression originale par la variable v_1 . La seule instruction qui résulte de cette transformation est l'instruction d'affectation du résultat de la fonction à la variable temporaire v_1 .

$$| \text{Id}() |_{\mathcal{E}} = \langle v_1, \text{“ } v_1 = \text{Id}() \text{ ;”} \rangle$$

Appel de fonction avec arguments : Dans le cas d'un appel d'une fonction avec des arguments, il est indispensable de transformer en premier lieu les arguments de la fonction. En effet, s'il s'agit d'une composition de fonctions, les arguments peuvent contenir éventuellement d'autres appels de fonctions. Les instructions intermédiaires

résultantes de la transformation de ce type d'expressions sont constituées de la concaténation des blocs d'instructions obtenus lors de la transformation des arguments. Le résultat de la fonction appliquée aux arguments simplifiés est ensuite sauvegardé dans une nouvelle variable de service v_1 qui est la transformation de l'expression initiale.

$$\begin{aligned} | \text{ld}(e) |_{\mathcal{E}} &= \underline{\text{let}} \langle v_1, B_1 \rangle = | e |_{\mathcal{E}} \\ &\quad \underline{\text{in}} \langle v_2, B_1 \bullet \text{“} v_2 = \text{Id}(v_1); \text{”} \rangle \\ &\quad \underline{\text{end}} \end{aligned}$$

Prenons un exemple pour expliquer cette transformation ; considérons l'expression constituée de l'imbrication de deux appels de fonction $f(g())$. Il est évident que la transformation se déroule sur deux étapes :

- l'application de la règle $\text{ld}()$ pour la transformation de l'expression $g()$;
- l'application de la règle $(\text{Id}(e))$ pour la simplification de l'imbrication de l'appel de fonction dans $f(g())$.

Par un souci de clarté, nous présentons les étapes d'application des règles de transformation sous forme d'un arbre de preuve en prenant soin de mentionner à chaque étape la règle utilisée.

$$\frac{\frac{\square}{|g()|_{\mathcal{E}} \rightarrow \langle v_1, \text{“} v_1 = g(); \text{”} \rangle} (\text{ld}()) \quad B_1 = \text{“} v_1 = g(); \text{”} \bullet \text{“} v_2 = f(v_1); \text{”}}}{|f(g())|_{\mathcal{E}} \rightarrow \langle v_2, B_1 \rangle} (\text{Id}(e))$$

Remarque : La transformation des appels de fonctions énoncées par les deux règles précédentes sont données dans le cas le plus général. Cependant, il existe un cas particulier qui mérite d'être mentionné : dans le cas où la fonction à appeler ne retourne pas de valeur, il n'est nullement nécessaire de stocker le résultat de son exécution.

Ainsi la transformation de la première forme d'appel de fonction est semblable à la transformation de la règle ld , et a pour résultat la syntaxe de l'appel original et un bloc d'instructions vide.

$$| \text{ld}() |_{\mathcal{E}} = \langle \text{ld}(), \emptyset \rangle$$

La transformation de la deuxième forme d'appel de fonction résulte en une expression équivalente à l'originale à la différence près que les arguments ont été transformés.

$$\begin{aligned} | \text{ld}(e) |_{\mathcal{E}} &= \underline{\text{let}} \langle v_1, B_1 \rangle = | e |_{\mathcal{E}} \\ &\quad \underline{\text{in}} \langle \text{id}(v_1), B_1 \rangle \\ &\quad \underline{\text{end}} \end{aligned}$$

Expression (e) : Cette forme d'expression se retrouve entres autres dans l'application de fonction du type $(e_1)(e_2)$ où la première expression e_1 peut être un pointeur

de fonction et la deuxième expression les arguments à passer.

La transformation de cette forme d'expression engendre une nouvelle variable intermédiaire v_1 qui contiendra la transformation de e , et un nouveau bloc d'instructions contenant v_1 .

$$\begin{aligned} | (e) |_{\mathcal{E}} = & \underline{let} \ \langle v_1, B_1 \rangle = | e |_{\mathcal{E}} \\ & \underline{in} \ \langle (v_1), B_1 \rangle \\ & \underline{end} \end{aligned}$$

Expression binaire : Les expressions binaires mettent en oeuvre deux expressions et un opérateur binaire. Ces expressions étant “calculables”, leur transformation consiste à simplifier les deux expressions et stocker le résultat de l’opération en question dans une variable intermédiaire. Le bloc d'instructions à générer consiste en la combinaison des blocs obtenus lors de la transformation des deux expressions avec un troisième bloc contenant la variable finale contenant le résultat.

$$\begin{aligned} | e_1 \ \underline{bop} \ e_2 |_{\mathcal{E}} = & \underline{let} \ \langle v_1, B_1 \rangle = | e_1 |_{\mathcal{E}} \\ & \langle v_2, B_2 \rangle = | e_2 |_{\mathcal{E}} \\ & \underline{in} \ \langle v_3, B_1 \bullet B_2 \bullet “ v_3 = v_1 \ \underline{bop} \ v_2; ” \rangle \\ & \underline{end} \end{aligned}$$

Affectation : Bien qu’à première vue l’affectation ressemble à une opération binaire, la transformation de cette expression est différente dans la mesure où elle n’est pas calculable. En effet cette expression modifie l’état des variables et il serait inutile de stocker le résultat dans une variable intermédiaire. La simplification d’une affectation est aussi une expression d’affectation, sauf qu’il faut au préalable transformer l’expression se trouvant à gauche du symbole d’affectation.

$$\begin{aligned} | e_1 \ \underline{assop} \ e_2 |_{\mathcal{E}} = & \underline{let} \ \langle v_1, B_1 \rangle = | e_1 |_{\mathcal{E}} \\ & \langle v_2, B_2 \rangle = | e_2 |_{\mathcal{E}} \\ & \underline{in} \ \langle v_1 \ \underline{assop} \ v_2, B_1 \bullet B_2 \rangle \\ & \underline{end} \end{aligned}$$

Expression conditionnelle : Ce type d’expressions permet de créer un branchement en fonction de l’évaluation d’une expression binaire. La transformation de cette expression reprend la logique du branchement en le stockant dans le bloc d'instructions généré.

Remarque : Pour des raisons de clarté, nous généralisons les expressions binaires (*binary*)

et conditionnelles (*conditional*) dans cette formulation à des expressions (e).

$$\begin{aligned}
 | e_1 ? e_2 : e_3 |_{\mathcal{E}} = & \underline{let} \quad \langle v_1, B_1 \rangle = | e_1 |_{\mathcal{E}} \\
 & \langle v_2, B_2 \rangle = | e_2 |_{\mathcal{E}} \\
 & \langle v_3, B_3 \rangle = | e_3 |_{\mathcal{E}} \\
 & B_4 = \text{“ } if (v_1) v_4 = v_2; \\
 & \quad \quad \quad else v_4 = v_3; \text{”} \\
 & \underline{in} \quad \langle v_4, B_1 \bullet B_2 \bullet B_3 \bullet B_4 \rangle \\
 & \underline{end}
 \end{aligned}$$

Liste d’arguments : Ce type d’expressions est utilisée dans le cas de l’application d’une expression e_1 à un ensemble d’arguments séparés par des virgules ($e_2, e_3, \dots$). Pour la transformation de la liste des arguments, il faut simplifier toutes les expressions constituant les arguments de la liste. Selon la grammaire du langage $\mathcal{C}$, une liste d’arguments est un élément récursif, composé d’une liste d’argument suivie d’une virgule et d’une expression d’affectation.

Pour simplifier la règle de transformation, nous généralisons les deux types d’expressions à des expressions e .

$$\begin{aligned}
 | e_1, e_2 |_{\mathcal{E}} = & \underline{let} \quad \langle v_1, B_1 \rangle = | e_1 |_{\mathcal{E}} \\
 & \langle v_2, B_2 \rangle = | e_2 |_{\mathcal{E}} \\
 & \underline{in} \quad \langle v_1, v_2, B_1 \bullet B_2 \rangle \\
 & \underline{end}
 \end{aligned}$$

Application : La simplification de l’application consiste à transformer la liste des arguments en premier lieu, et ensuite à transformer l’expression de la fonction à appliquer sur les arguments. Le résultat final sera une expression du type $e_1(e_2, e_3, \dots)$ et un bloc d’instructions qui est la combinaison des différents blocs d’instructions générés lors des transformations préliminaires.

$$\begin{aligned}
 | e_1 (e_2) |_{\mathcal{E}} = & \underline{let} \quad \langle v_1, B_1 \rangle = | e_1 |_{\mathcal{E}} \\
 & \langle v_2, B_2 \rangle = | e_2 |_{\mathcal{E}} \\
 & B_3 = B_1 \bullet B_2 \bullet \text{“ } v_3 = v_1(v_2); \text{”} \\
 & \underline{in} \quad \langle v_3, B_3 \rangle \\
 & \underline{end}
 \end{aligned}$$

Exemple de transformation d'expressions

Pour mettre en pratique ces règles de transformation, nous allons procéder à la simplification de l'expression suivante : “ $x = (i == a) ? y : z$ ”. Cette expression est composée d'une expression d'affectation dont le premier terme est un identifiant, et le deuxième terme est une expression conditionnelle. À la fin de la transformation, nous obtiendrons une nouvelle expression d'affectation et une suite d'instructions intermédiaires.

$$\begin{array}{c}
 \text{(Id)} \frac{\square}{|i|_{\mathcal{E}} \rightarrow \langle i, \emptyset \rangle} \quad \text{(Id)} \frac{\square}{|a|_{\mathcal{E}} \rightarrow \langle a, \emptyset \rangle} \quad B_1 \\
 \text{(binary)} \frac{}{|i == a|_{\mathcal{E}} \rightarrow \langle i == a, B_1 \rangle} \\
 \text{((e))} \frac{}{|(i == a)|_{\mathcal{E}} \rightarrow \langle (v_1), B_1 \rangle} \\
 \text{(cond)} \frac{}{|(i == a)?y : z|_{\mathcal{E}} \rightarrow \langle v_2, B_2 \rangle} \quad \mathcal{A}_1 \quad \mathcal{A}_2 \quad B_1 \\
 \text{(aff)} \frac{}{|x = (i == a)?y : z|_{\mathcal{E}} \rightarrow \langle x = v_2, B_2 \rangle}
 \end{array}$$

Les sous arbres $\mathcal{A}_1$ et $\mathcal{A}_2$ sont définis comme suit :

$$\mathcal{A}_1 = \frac{\square}{|y|_{\mathcal{E}} \rightarrow \langle y, \emptyset \rangle} \text{(Id)} \quad \mathcal{A}_2 = \frac{\square}{|z|_{\mathcal{E}} \rightarrow \langle z, \emptyset \rangle} \text{(Id)}$$

Les blocs d'instruction intermédiaires s'écrivent sous la forme suivante :

$$B_1 = \text{“ } v_1 = i == a; \text{ ”} \quad B_2 = B_1 \bullet \begin{array}{l} \text{“ if } (v_1) \ v_2 = y; \\ \text{else } v_2 = z; \text{ ”} \end{array}$$

Le résultat final de la transformation de l'expression “ $x = (i == a) ? y : z$ ” est la nouvelle expression “ $x = v_2$ ” avec les instructions intermédiaires du bloc B_2 :

$$\begin{array}{l}
 B_2 = \text{“ } v_1 = i == a; \\
 \text{if } (v_1) \ v_2 = y; \\
 \text{else } v_2 = z; \text{ ”}
 \end{array}$$

5.2.5 Transformation des instructions

Nous allons maintenant nous intéresser à la transformation des instructions d'un programme. La fonction de traduction des instructions, notée $\mathcal{I}$, permettra de réécrire

les instructions au fur et à mesure du parcours de l'arbre syntaxique du programme source.

Il serait utile de noter que même si le parcours de l'arbre syntaxique se fait dans un sens descendant (partant des noeuds pour arriver aux feuilles), la réécriture du programme résultant ne se contraint pas à cette règle. En effet comme nous allons le voir avec la transformation de certains éléments de la syntaxe, leur transformation exige d'insérer des instructions à un endroit bien précis dans le nouveau programme linéarisé.

Domaine de définition

Afin de restreindre notre attention uniquement sur les instructions de la syntaxe du langage $\mathbb{C}$ [25], nous définissons le langage $\mathcal{L}_I$ comme étant une réduction de la syntaxe limitée aux définitions des instructions.

Dans cette nouvelle syntaxe, le symbole non terminale e représente une expression telle qu'elle a été définie précédemment (table 5.5) et les termes en caractère gras représentent des symboles terminaux.

$ \begin{aligned} s &::= \textit{compound} \mid \textit{label} \mid \textit{expression} \mid \textit{selection} \\ &\quad \mid \textit{iteration} \mid \textit{jump} \\ \textit{compound} &::= \{ \} \mid \{ sList \} \mid \{ d \ sList \} \\ sList &::= s \mid sList \ s \\ \textit{expression} &::= e \ ; \ ; \\ \textit{label} &::= id \ : \ s \mid \textbf{case } e \ : \ s \mid \textbf{default} \ : \ s \\ \textit{selection} &::= \textbf{if}(e) \ s \mid \textbf{if}(e) \ s_1 \ \textbf{else} \ s_2 \mid \textbf{switch}(e) \ s \\ \textit{iteration} &::= \textbf{while}(e) \ s \mid \textbf{do } s \ \textbf{while}(e) \mid \textbf{for}(e_1; e_2; e_3) \ s \\ \textit{jump} &::= \textbf{goto } id \mid \textbf{continue} \mid \textbf{break} \mid \textbf{return} \mid \textbf{return } e \end{aligned} $

TABLE 5.6 – Syntaxe du langage $\mathcal{L}_I$.

Une instruction peut alors être : une instruction composée (*compound*), une instruction étiquetée (*label*), une expression (*expression*), une instruction de sélection (*selection*), une structure itérative (*iteration*) ou une instruction de saut (*jump*).

La transformation à effectuer portera sur les instructions d'itération, l'instruction de sélection **switch** et les instructions de saut **continue**, **break**, **return**, les autres éléments de la syntaxe ne seront pas transformés. De ce fait nous pouvons définir le langage cible $\mathcal{L}'_I$

dans lequel seront exprimées les instructions après avoir été transformées. La syntaxe de ce dernier langage est décrite dans le tableau 5.7.

s	$::=$	$compound \mid label \mid expression \mid selection \mid jump$
$compound$	$::=$	$\{ \} \mid \{ sList \} \mid \{ d sList \}$
$sList$	$::=$	$s \mid sList s$
$expression$	$::=$	$e \mid ;$
$label$	$::=$	$id : s$
$selection$	$::=$	$\mathbf{if}(e) s \mid \mathbf{if}(e) s_1 \mathbf{else} s_2$
$jump$	$::=$	$\mathbf{goto} id \mid \mathbf{return} \mid \mathbf{return} e$

TABLE 5.7 – Syntaxe du langage $\mathcal{L}'_I$.

Nous pouvons constater que la syntaxe résultante est toujours celle du langage C mais elle est plus concise.

La fonction de transformation des instructions permet d'éliminer les structures complexes en les remplaçant par des instructions de branchement et des instructions de saut tout en préservant la fonctionnalité du programme. Cette fonction prend une instruction du langage $\mathcal{L}_I$ et un environnement et retourne une suite d'instructions appartenant au langage $\mathcal{L}'_I$:

$$| _ |_I : \mathcal{L}_I \rightarrow \Gamma \rightarrow \mathcal{L}'_I$$

Le parcours de l'arbre syntaxique se fait de manière récursive ; ainsi l'environnement dynamique Γ est mis à jour avant l'appel récursif de la fonction et l'application des règles pour que la transformation se fasse dans le contexte adéquat. Autrement dit, la transformation d'un noeud établit un contexte particulier qui sera pris en considération lors de la transformation des feuilles.

Contexte de traduction

On définit le contexte de traduction comme étant un état des variables contenues dans l'environnement dynamique Γ à un instant donné. Ainsi on parle de changement de contexte lorsqu'on met à jour les variables de l'environnement.

Les deux variables l_{break} et $l_{continue}$ font partie de l'environnement dynamique et conservent les étiquettes vers lesquelles brancher lors de la traduction des instructions **break** et **continue**. Ces variables sont mises à jour lors d'un changement de contexte.

À titre d'exemple, nous pouvons citer le cas de l'instruction **while** : elle est constituée d'une expression e et d'une instruction s . Lors de la transformation de s , il faut instaurer un contexte Γ_1 , ce qui revient à mettre à jour les valeurs des variables l_{break} et $l_{continue}$. Cette mise à jour sert à la transformation des instructions **break** ou **continue** éventuellement comprises dans s . Une fois la transformation achevée, on revient au contexte précédent en remplaçant les anciennes valeurs des variables l_{break} et $l_{continue}$.

Règles de transformation

L'objectif à atteindre est d'avoir un programme linéarisé, dans le sens où il ne contient que des appels de fonctions et des instructions de branchement et de saut, une structure facilement assimilable à un automate d'états finis. Pour ce faire, la transformation à appliquer sur les instructions cible celles qui introduisent implicitement un comportement complexe. Ceci dit, nous allons quand même énumérer les différentes instructions de la syntaxe et présenter leur version réécrite.

Instruction composée : Par définition elle est composée d'une liste d'instructions, éventuellement précédée par un ensemble de déclarations, et entourée d'accolades. Comme les déclarations (représentées par le symbole d dans cette règle) ne sont pas affectées par la fonction $\mathcal{I}$, la transformation d'une instruction composée revient à la transformation de la liste d'instructions qui la composent.

$$\begin{aligned} | \{ \} |_{\mathcal{I}}^{\Gamma} &= \{ \} \\ | \{ sList \} |_{\mathcal{I}}^{\Gamma} &= \{ | sList |_{\mathcal{I}}^{\Gamma} \} \\ | \{ d sList \} |_{\mathcal{I}}^{\Gamma} &= \{ d | sList |_{\mathcal{I}}^{\Gamma} \} \end{aligned}$$

Liste d'instructions : C'est un élément non-terminal récursif de la grammaire qui permet de construire une liste de plusieurs instructions successives. La liste est définie comme une liste suivie d'une instruction, ou d'une seule instruction finale qui termine la liste. La transformation d'une liste d'instructions consiste à transformer la première instruction, puis la suivante jusqu'à la dernière instruction de la liste.

$$| sList s |_{\mathcal{I}}^{\Gamma} = | sList |_{\mathcal{I}}^{\Gamma} \mid s |_{\mathcal{I}}^{\Gamma}$$

Expression : Cette construction n'est rien d'autre qu'une expression suivie d'un point virgule, ce qui en fait une instruction à part entière. Désignée par "*expression* –

statement" dans la syntaxe originale du langage $\mathbb{C}$, elle sert à considérer les expressions comme des instructions. À titre d'exemple, l'expression $\text{ld}()$ représente un appel de fonction dans un programme ; par contre la formulation $\text{ld}()$; est considérée par le parseur comme une instruction composée d'une seule expression. La transformation de cette instruction se résume à placer le résultat de la transformation de e suivi d'un point virgule.

$$\begin{aligned} | e ; |_{\mathcal{I}}^{\Gamma} &= \underline{\text{let}} \langle v_1, B_1 \rangle = | e |_{\mathcal{E}} \\ &\quad \underline{\text{in}} \quad B_1 \\ &\quad v_1 ; \\ &\quad \underline{\text{end}} \end{aligned}$$

Instruction étiquetée : Elle est composée d'une étiquette et d'une instruction. La transformation à appliquer consiste à garder l'étiquette et transformer l'instruction :

$$| \text{id} : s |_{\mathcal{I}}^{\Gamma} = \text{id} : | s |_{\mathcal{I}}^{\Gamma}$$

Instruction case : La structure d'une instruction **case** se compose d'une expression e et d'une instruction s . La transformation d'une instruction **case** résulte en une instruction étiquetée composée d'une étiquette fraîche l_{case} et de la transformation de l'instruction s . Une mise à jour du contexte actuel est aussi effectuée en ajoutant la paire (e, L_{case}) dans le tableau A qui se trouve dans l'environnement. Ce tableau a été ajouté au contexte de transformation actuel par l'instruction **switch** parente de l'instruction **case** :

$$| \text{case } e : s |_{\mathcal{I}}^{\Gamma[A]} = l_{\text{case}} : | s |_{\mathcal{I}}^{\Gamma \dagger[A::(e, l_{\text{case}})]}$$

Remarque : Du moment que l'expression e n'apparaît pas dans le résultat de la transformation de l'instruction **case**, elle sera placée dans l'environnement dynamique sans être transformée. Nous procéderons à la transformation de l'expression e lors du traitement des éléments du tableau A par la fonction *BigIfThenElse*.

Instruction default : Équivalente à l'instruction **case**, sa transformation aboutit à une instruction étiquetée et une mise à jour du tableau A se trouvant dans l'environnement de traduction. À la différence du **case**, l'instruction **default** ne contient pas d'expression, c'est pour cela que la paire placée dans le tableau A contient le texte "*default*" à la place d'une expression.

$$| \text{default} : s |_{\mathcal{I}}^{\Gamma[A]} = L_{\text{default}} : | s |_{\mathcal{I}}^{\Gamma \dagger[A::("default", L_{\text{case}})]}$$

Instruction *if then else* : La simplification de cette structure passe en premier lieu par la transformation de l'expression e contenue dans la clause *if*, puis par la transformation des instructions contenues dans les clauses *then* et *else*. Les instructions intermédiaires obtenues de la transformation de l'expression sont placées avant le résultat de la transformation de l'instruction *if-then-else*. De ce fait, le résultat de la transformation d'une instruction *if-then-else* est une liste d'instructions.

$$\begin{aligned}
 | \text{if}(e) \ s \ |_{\mathcal{I}}^{\Gamma} &= \underline{\text{let}} \ \langle v_1, B_1 \rangle = | e \ |_{\mathcal{E}} \\
 &\quad \underline{\text{in}} \ B_1 \\
 &\quad \text{if}(v_1) \ \ | \ s \ |_{\mathcal{I}}^{\Gamma} \\
 &\quad \underline{\text{end}} \\
 | \text{if}(e) \ s_1 \ \text{else} \ s_2 \ |_{\mathcal{I}}^{\Gamma} &= \underline{\text{let}} \ \langle v_1, B_1 \rangle = | e \ |_{\mathcal{E}} \\
 &\quad \underline{\text{in}} \ B_1 \\
 &\quad \text{if}(v_1) \ \ | \ s_1 \ |_{\mathcal{I}}^{\Gamma} \ \ \text{else} \ \ | \ s_2 \ |_{\mathcal{I}}^{\Gamma} \\
 &\quad \underline{\text{end}}
 \end{aligned}$$

Instruction interrupteur (*switch*) : L'instruction *switch* est composée d'une expression e et d'un bloc d'instructions s . Concrètement, la transformation de la construction *switch* consiste à créer un nouveau tableau A et une nouvelle étiquette pour l_{break} ; ces deux nouveaux éléments constituent le nouvel environnement dynamique Γ_1 utilisé comme contexte de transformation du bloc d'instructions s (qui contient les différentes instructions *case*). À l'achèvement de l'étape de transformation des instructions contenues dans s , le tableau A renfermera des paires (*expression*, *étiquette*). Ce dernier, ainsi que l'évaluation de l'expression e , sont passés à la fonction **BigIfThenElse** qui permet de créer un bloc de plusieurs instructions *if-then-else* imbriquées, dont le rôle est d'aiguiller vers l'instruction adéquate lors de l'exécution. Les instructions du bloc B_1 résultant de la transformation de l'expression e sont placées en amont des instructions générées par la fonction **BigIfThenElse**, suivies de la transformation des instructions englobées par s et finalement une étiquette l_1 servant à passer l'exécution à la fin du traitement du *switch* (comportement engendré par la transformation de l'instruction *break*.) Cette technique de transformation de l'instruction *switch* est plus complète et plus

générale que celle décrite dans [2].

$$\begin{aligned}
 | \text{switch}(e) \ s \ |_{\mathcal{I}}^{\Gamma} = & \ \underline{\text{let}} \ \langle v_1, B_1 \rangle = | e \ |_{\mathcal{E}} \\
 & \ C = | s \ |_{\mathcal{I}}^{\Gamma_1} \\
 & \ B = \text{BigIfThenElse}(v_1, \Gamma_1[A]) \\
 & \ \underline{\text{in}} \ \{ \ B_1 \\
 & \ \quad B \\
 & \ \quad C \\
 & \ \quad l_1 : ; \ \} \\
 & \ \underline{\text{end}}
 \end{aligned}$$

avec $\Gamma_1 = \Gamma \uparrow [A \mapsto \text{New Array}, l_{\text{break}} \mapsto l_1]$

Une fois la transformation du bloc d'instructions s terminée, le contexte original est rétabli. Par exemple une variable l_{break} associée à une étiquette l_i dans Γ avant la transformation de l'instruction **switch**, sera attribuée à l'étiquette l_j pendant la traduction du bloc d'instructions dans Γ_1 , et réassociée à l_i à la fin de la transformation du **switch**.

Fonction *BigIfThenElse* : Cette fonction prend comme paramètre une expression v et un tableau (ou une liste) A de paires (*expression*, *étiquette*), et construit un bloc d'instructions **if-then-else** imbriquées. L'expression v , ainsi que les paires du tableau passé en paramètre sont du type chaîne de caractères. Il en est de même pour le bloc construit et retourné par la fonction. Parmi les paires du tableau passé en paramètre, il en existe éventuellement une qui correspond à l'instruction **default** ; dans ce cas, la fonction gardera son traitement pour le dernier **else** de la structure à construire. Ainsi, si on considère que le type d'un tableau est une liste, la signature de la fonction est

$$String \rightarrow (String \times String) \text{ list} \rightarrow String$$

Le pseudo-code de cette fonction est énoncé dans le tableau 5.8.

Instructions d'itération : Le principe de transformation des instructions d'itération est pratiquement le même pour les trois types d'instructions (**do**, **while** et **for**). La différence réside dans l'emplacement des étiquettes l_{break} et l_{continue} pour conserver le bon

```

BigIfThenElse(e,A)
{
  strReturn="";
  if(!empty(A))
  {elem= hd A;
   tmp=BigIfThenElse(e,tl A);
   if (#1 elem ="default")
   strReturn= tmp ^ "goto" ^ #2 elem;
   else
   strReturn="if ("^e^"=="^#1 elem^) goto "^#2 elem^" else";
  }
  return strReturn;
}

```

TABLE 5.8 – Fonction *BigIfThenElse*.

comportement. La transformation de l’instruction **while** se présente comme ci-dessous :

$$\begin{array}{l}
 | \text{ while } (e) \text{ } s \mid_{\mathcal{I}}^{\Gamma} = \underline{\text{let}} \quad \langle v_1, B_1 \rangle = | e |_{\mathcal{E}} \\
 \quad \underline{\text{in}} \quad l_1 : B_1 \\
 \quad \text{if}(! v_1) \text{ goto } l_2; \\
 \quad | s \mid_{\mathcal{I}}^{\Gamma_1} \\
 \quad \text{goto } l_1; \\
 \quad l_2 : ; \\
 \underline{\text{end}}
 \end{array}$$

où $\Gamma_1 = \Gamma \uparrow [l_{\text{continue}} \mapsto l_1; l_{\text{break}} \mapsto l_2]$

La transformation de l’instruction **do** diffère par l’utilisation d’une troisième étiquette nécessaire pour les **break** éventuellement présents dans le bloc d’instruction s :

$$\begin{array}{l}
 | \text{ do}(s) \text{ while}(e) \mid_{\mathcal{I}}^{\Gamma} = \underline{\text{let}} \quad \langle v_1, B_1 \rangle = | e |_{\mathcal{E}} \\
 \quad \underline{\text{in}} \quad l_1 : | s \mid_{\mathcal{I}}^{\Gamma_1} \\
 \quad \quad l_2 : B_1 \\
 \quad \text{if}(v_1) \text{ goto } l_1; \\
 \quad \quad l_3 : ; \\
 \underline{\text{end}}
 \end{array}$$

où $\Gamma_1 = \Gamma \uparrow [l_{\text{continue}} \mapsto l_2; l_{\text{break}} \mapsto l_3]$

L’instruction **for** utilise trois expressions et une instruction. Lors de la transformation il faut placer judicieusement les instructions qui découlent de la traduction des

expressions.

$$\begin{aligned}
 | \text{for}(e_1; e_2; e_3) s |_{\mathcal{I}}^{\Gamma} = & \underline{\text{let}} \quad \langle v_1, B_1 \rangle = | e_1 |_{\mathcal{E}} \\
 & \langle v_2, B_2 \rangle = | e_2 |_{\mathcal{E}} \\
 & \langle v_3, B_3 \rangle = | e_3 |_{\mathcal{E}} \\
 & \underline{\text{in}} \quad B_1 \\
 & v_1 \\
 & l_1 : B_2 \\
 & \text{if} (! v_2) \text{ goto } l_2; \\
 & | s |_{\mathcal{I}}^{\Gamma_1} \\
 & B_3 \\
 & v_3 \\
 & \text{goto } l_1; \\
 & l_2 : ; \\
 & \underline{\text{end}}
 \end{aligned}$$

où $\Gamma_1 = \Gamma \uparrow [l_{\text{continue}} \mapsto l_1; l_{\text{break}} \mapsto l_2]$

Les instructions de saut : La transformation d'un saut vers une étiquette ne change pas après la simplification :

$$| \text{goto } L |_{\mathcal{I}}^{\Gamma} = \text{goto } L$$

La transformation des deux instructions **break** et **continue** repose surtout sur la bonne manipulation des étiquettes l_{break} et l_{continue} au cours de la transformation des instructions précédentes. En effet, elle se résume à générer un saut vers les étiquettes sauvegardées.

$$| \text{continue} |_{\mathcal{I}}^{\Gamma} = \text{goto } \Gamma[l_{\text{continue}}]$$

$$| \text{break} |_{\mathcal{I}}^{\Gamma} = \text{goto } \Gamma[l_{\text{break}}]$$

Concernant le **return**, nous le transformons par un saut vers une étiquette unique l_{return} qui se trouve toujours à la fin de la fonction en cours de réécriture.

$$| \text{return} |_{\mathcal{I}}^{\Gamma} = \text{goto } l_{\text{return}}$$

Dans le cas où il faut retourner une valeur, la variable `--xs` est mise à jour avant de rajouter l'instruction de saut vers l'étiquette l_{return} .

$$\begin{aligned}
 | \text{return } e |_{\mathcal{I}}^{\Gamma} = & \underline{\text{let}} \quad \langle v_1, B_1 \rangle = | e |_{\mathcal{E}} \\
 & \underline{\text{in}} \quad B_1 ; \\
 & \text{--xs} = v_1 ; \\
 & \text{goto } l_{\text{return}}; \\
 & \underline{\text{end}}
 \end{aligned}$$

5.2.6 Exemple de traduction

Nous allons prendre un exemple simple pour mettre en évidence quelques-unes des règles que nous venons d'énoncer.

L'exemple choisi met en évidence la transformation de l'instruction **switch** et l'utilisation de l'environnement dynamique lors de la transcription du programme. La simplicité de cet exemple est justifiée par le nombre important d'étapes à parcourir et de règles à appliquer pour atteindre la transformation finale du programme. Nous allons lister toutes les étapes de la transcription du programme en prenant soin de commenter et de mettre l'accent sur les transformations apportées à chaque étape.

```

1 int main()
2 {int i=1;
3   switch(i)
4   {case 1 : printf("un");break;
5     default : printf("default");
6   }
7 }
```

TABLE 5.9 – Programme p à transformer.

Comme nous l'avons mentionné précédemment, la transformation d'un programme repose sur la transformation des fonctions qu'il contient :

$$| p |_{\mathcal{P}} = \left| \begin{array}{l} \mathbf{int} \text{ main}() \\ \{\mathbf{int} \text{ i}=1; \\ \mathbf{switch}(i) \\ \quad \{\mathbf{case} \text{ 1 : printf("un"); break;} \\ \quad \mathbf{default} : \text{printf("default");} \\ \quad \} \\ \} \end{array} \right|_{\mathcal{F}}$$

La traduction de la fonction crée un environnement dynamique Γ utilisé lors de la traduction des instructions. L'environnement Γ initialement vide est noté $[]$; il sera peuplé au fur et à mesure de la transformation des instructions dépendamment des règles à appliquer.

La fonction **main** admettant un type **int**, l'application de la règle de transformation se résume par le rajout la déclaration de la variable **__xs** ainsi que l'instruction étiquetée "**L_Return : return __xs ;**". Il faudrait noter aussi les accolades ouvrantes et fermantes ajoutées par la fonction $\mathcal{F}$ qui encercle la transformation du corps de la

fonction.

$$\begin{array}{c}
 \text{int main()} \\
 \{ \text{int } _xs; \\
 \quad \{ \text{int } i=1; \\
 \quad \quad \text{switch}(i) \\
 \quad \quad \quad \{ \text{case } 1 : \text{printf("un"); break;} \\
 \quad \quad \quad \text{default : printf("default");} \\
 \quad \quad \quad \} \\
 \quad \} \\
 \text{Lreturn : return } _xs; \\
 \}
 \end{array}
 \begin{array}{c}
 \left[\begin{array}{c} \\ \\ \\ \\ \\ \end{array} \right] \\
 \mathcal{I}
 \end{array}
 \begin{array}{c}
 \\ \\ \\ \\ \\ (\{s\})
 \end{array}$$

Le corps de la fonction est une suite d'instructions entourée d'accolades ; sa transcription passe par l'application de la règle de transformation d'une instruction composée ($\{sList\}$). L'application de cette transformation résulte par la transformation des instructions comprises à l'intérieur des accolades.

$$\begin{array}{c}
 \text{int main()} \\
 \{ \text{int } _xs; \\
 \quad \{ \\
 \quad \quad \text{int } i=1; \\
 \quad \quad \text{switch}(i) \\
 \quad \quad \quad \{ \text{case } 1 : \text{printf("un"); break;} \\
 \quad \quad \quad \text{default : printf("default");} \\
 \quad \quad \quad \} \\
 \quad \} \\
 \text{Lreturn : return } _xs; \\
 \}
 \end{array}
 \begin{array}{c}
 \left[\begin{array}{c} \\ \\ \\ \\ \\ \end{array} \right] \\
 \mathcal{I}
 \end{array}
 \begin{array}{c}
 \\ \\ \\ \\ \\ (sList)
 \end{array}$$

Les deux instructions qui constituent le corps de la fonction sont :

- une déclaration d'initialisation de la variable `i` ;
- une instruction interrupteur (*switch*).

La transformation d'une liste d'instructions consiste à transformer chaque instruction

qui la compose.

$$\begin{array}{c}
 \text{int main()} \\
 \{ \text{int } __xs; \\
 \{ \\
 \quad \text{int } i=1; \quad \left[\begin{array}{c} [] \\ \mathcal{I} \end{array} \right] \quad (\text{init declaration}) \\
 \quad \text{switch}(i) \quad \left[\begin{array}{c} [] \\ \mathcal{I} \end{array} \right] \quad (\text{switch}) \\
 \quad \quad \{ \text{case } 1 : \text{printf("un"); break;} \\
 \quad \quad \quad \text{default : printf("default");} \\
 \quad \quad \} \\
 \} \\
 \text{Lreturn : return } __xs; \\
 \}
 \end{array}
 =$$

La fonction de transformation des instructions n'a aucun effet sur les déclarations de variable, c'est pour cela que la première instruction est transcrite telle qu'elle sans aucune modification. Par contre la deuxième instruction réclame l'application de la règle de transformation de l'instruction **switch**.

$$\begin{array}{c}
 \text{int main()} \\
 \{ \text{int } __xs; \\
 \{ \\
 \quad \text{int } i=1; \\
 \quad \text{switch}(i) \quad \left[\begin{array}{c} [] \\ \mathcal{I} \end{array} \right] \quad (\text{switch}) \\
 \quad \quad \{ \text{case } 1 : \text{printf("un"); break;} \\
 \quad \quad \quad \text{default : printf("default");} \\
 \quad \quad \} \\
 \} \\
 \text{Lreturn : return } __xs; \\
 \}
 \end{array}
 =$$

Nous allons nous concentrer à présent sur la transformation de l'instruction **switch** seulement. D'après la règle, la réécriture d'une telle instruction requiert l'accomplissement des étapes suivantes :

1. la transformation de l'expression i : $\langle v_1, B_1 \rangle = | i |_{\mathcal{E}}$;
2. la création d'un nouveau tableau A et d'une nouvelle étiquette l_{break} qui représentent le nouveau contexte de transformation Γ_1 ;

3. la transformation des instructions dans le nouveau contexte Γ_1 :

$$\left| \begin{array}{l} \{\mathbf{case} \ 1 : \text{printf}(\text{"un"}); \mathbf{break}; \\ \quad \mathbf{default} : \text{printf}(\text{"default"}); \\ \} \end{array} \right|_{\mathcal{I}}^{\Gamma_1} \quad (\{s\})$$

4. et finalement l'appel de la fonction *BigIfThenElse* avec en paramètre le tableau A et l'expression v_1 .

La transformation de l'expression i se résume par l'application de la règle $|\text{ld}|_{\mathcal{E}}$ de la fonction de transformation des expressions, dont le résultat n'est autre que l'expression elle-même et aucune instruction intermédiaire : $\langle v_1, B_1 \rangle = \langle i, \emptyset \rangle$.

Le nouvel environnement Γ_1 , constitué d'un tableau vide et de l'association de la variable l_{break} à l'étiquette l_1 , s'écrit sous la forme : $\Gamma_1 = \Gamma \uparrow [A \mapsto [], l_{break} \mapsto l_1]$.

L'application de la règle de composition $(\{s\})$ revient à transformer les instructions se trouvant à l'intérieur des accolades :

$$\begin{array}{l} | \mathbf{case} \ 1 : \text{printf}(\text{"un"}); \mathbf{break}; |_{\mathcal{I}}^{\Gamma_1} \quad (\{\text{case}\}) \\ | \mathbf{default} : \text{printf}(\text{"default"}); |_{\mathcal{I}}^{\Gamma_1} \quad (\{\text{default}\}) \end{array}$$

La règle de transformation de l'instruction **case** stipule que son application résulte en une instruction étiquetée, suivie de la transformation des instructions du **case**, et la mise à jour du tableau A dans l'environnement. Cette mise à jour consiste à ajouter la paire (*expression*, *etiquette*) et à associer l'étiquette l_1 à la variable l_{break} :

$$l_case_1 : | \text{printf}(\text{"un"}); \mathbf{break}; |_{\mathcal{I}}^{[A \mapsto [(1, l_case_1)], l_{break} \mapsto l_1]} \quad (\text{sList})$$

À présent, il suffit de simplifier les deux instructions “`printf()` ;” et “`break` ;”. La transformation de la première fait intervenir la règle $(\mathbf{e};)$ suivi de l'invocation de la fonction de transformation des expressions, et l'application de la règle $|\text{ld}(\mathbf{e})|_{\mathcal{E}}$:

$$\begin{aligned} | \text{printf}(\text{"un"}); |_{\mathcal{I}}^{[A \mapsto [(1, l_case_1)], l_{break} \mapsto l_1]} &= | \text{printf}(\text{"un"}) |_{\mathcal{E}} ; \quad (\text{ld}(\mathbf{e})) \\ &= \text{printf}(\text{"un"}) ; \end{aligned}$$

La transformation du “`break` ;” consiste en une nouvelle instruction “**goto** l_{break} ” en remplaçant la valeur de l_{break} par l'étiquette présente dans l'environnement dynamique :

$$| \mathbf{break}; |_{\mathcal{I}}^{[A \mapsto [(1, l_case_1)], l_{break} \mapsto l_1]} = \mathbf{goto} \ l_1; \quad (\mathbf{goto})$$

La transformation de l'instruction **default** est semblable à celle de l'instruction **case** : le résultat est une instruction étiquetée, et il y a une mise à jour du tableau A avec une nouvelle paire (*expression*, *etiquette*). Cependant, il est noté que l'expression de cette

paire se résume à la chaîne de caractère "default", vu que cette instruction ne contient pas d'expression.

| **default** : printf("un"); $l_{\mathcal{I}}^2 = l_default$: printf("un"); (default)

avec $\Gamma_2 = [A \mapsto [("1", "l_case_1"), ("default", "l_default")], l_{break} \mapsto l_1]$

La dernière étape de la transformation de l'instruction **switch** est l'invocation de la fonction *BigIfThenElse* avec le tableau A et l'expression i . Cette fonction permettra de construire le bloc d'instructions if-then-else qui permettra l'aiguillage de l'exécution en fonction de la valeur de i .

Il est utile d'indiquer que les expressions comprises dans les paires du tableau A seront transformées avant l'appel de cette fonction et que les instructions intermédiaires qui s'ensuivent seront placées avant les imbrications if-then-else.

$BigIfThenElse(i, \Gamma_2[A]) = \text{" if}(i==1) \text{ goto } l_case_1 ; \text{ else goto } l_default ; \text{"}$

Finalement la transformation complète du programme p (page 85) est le programme décrit dans la table 5.10.

```

1  int main ()
2  {int __xs;
3    {
4      int i=1;
5      if(i==1) goto l_case_1;
6      else goto l_default;
7      {l_case_1 : printf ("un");
8        goto l_1;
9        l_default : printf ("default");
10     }
11     l_1 : ;
12   }
13 }
14 l_return : return __xs;
15 }
```

TABLE 5.10 – Programme p après transformation.

5.3 Modélisation en automate fini déterministe

En considérant les caractéristiques des automates finis déterministes, nous pouvons constater que le programme résultant de la transformation précédente peut être facilement

modélisé par un tel automate.

L'objectif d'avoir une modélisation sous forme d'automate du programme est de pouvoir effectuer la traduction directe du comportement du programme vers une expression de l'algèbre de Kleene. Or le comportement qui nous intéresse dans cette démarche réside dans l'enchaînement des fonctions, API certes, mais surtout les fonctions utilisateur invoquées au sein du programme.

Pour cela, le modèle à construire consiste en un automate dont les transitions représentent les appels de fonctions de toutes sortes, et dont les états constituent des étapes d'exécution du programme. Ainsi, le principe de modélisation de ce programme sous forme d'automate fini est simple : il suffit de parcourir l'arbre syntaxique du programme et de construire au fur et à mesure du parcours l'automate équivalent. Nous présentons dans ce qui suit les préceptes que nous avons adoptés pour la construction de l'automate.

5.3.1 Principes de modélisation

La première étape de transformation du code source permet d'obtenir un programme qui utilise seulement les instructions de saut (*jump*), les branchements (*selection*) et les expressions.

Par conséquent le point de départ de la modélisation prend appui sur le programme transformé selon la syntaxe du langage $\mathcal{L}'_T$ (5.7), ce qui signifie que ce dernier ne peut être composé que des éléments suivants :

- instructions de sélection : `if` et `if else` ;
- instructions de saut : `goto` ;
- instructions étiquetées : `id` ;
- expressions : `expression` ; .

La modélisation en automate du programme sera composée de la représentation de chacune de ses instructions en un ensemble d'états et de transitions. Comme le parcours de l'arbre syntaxique du programme se fait de manière récursive, la construction de l'automate final devra se faire de manière progressive. En effet lors de la modélisation de chaque instruction du programme, nous sommes contraints d'utiliser des transitions de service étiquetées ϵ permettant de relier ensemble les différents états.

Une fois le comportement complet du programme représenté par un automate, nous pouvons éliminer les transitions inutiles et ne garder que les transitions qui nous intéressent, à savoir, les appels de fonction.

Lors du parcours de l'arbre syntaxique du programme, nous nous consacrons à ne modéliser que les expressions qui font intervenir les appels de fonction. De cette façon nous ne gardons que les transitions qui sont susceptibles de reproduire le flot de contrôle du programme.

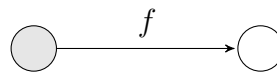
Pour que l'assemblage des différents états se réalise convenablement, nous choisissons d'uniformiser la modélisation de toutes les instructions de façon à ce qu'elles commencent par un seul état, et se terminent aussi par un seul état, quitte à utiliser des transitions de services pour les relier.

Nous décrivons dans ce qui suit la modélisation des différentes instructions de la syntaxe du programme transformé.

Modélisation des expressions

Les instructions du programme à modéliser ne contiennent désormais qu'au plus une seule expression. Vu que seulement les appels de fonction nous intéressent, nous ne gardons que les expressions du type $f()$ ou $f(e)$.

Ainsi une telle expression est représentée par une transition portant une étiquette du titre de la fonction appelée. Ces deux éléments viennent se greffer sur le dernier noeud obtenu des représentations précédentes.



Modélisation des instructions de type : $f()$ et $f(e)$.

Modélisation des branchements

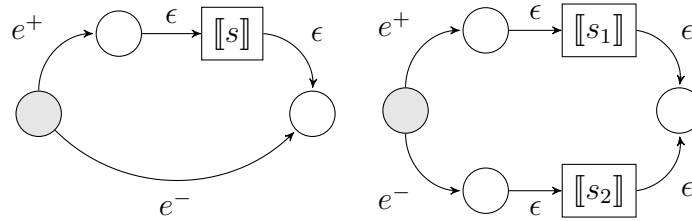
Les instructions `if` et `if else` reflètent un embranchement puisqu'elles permettent d'orienter l'exécution vers un traitement particulier ou vers un autre. Cela se traduit par la création d'un état avec deux transitions : l'une mène à l'exécution des instructions de la clause du `if` et l'autre à l'exécution des instructions de la clause `else`.

La modélisation adéquate de cette instruction consiste à créer :

- un état final vers lequel se rendra l'exécution une fois l'instruction `if` consommée ;
- deux transitions, partant de l'état initial, qui reflètent chacune la validité ou non de la condition.

Selon que nous traitons l'instruction `if` ou l'instruction `if else`, les éléments à créer diffèrent. Dans le cas d'une instruction de branchement sans l'alternative `else`, nous procédons à la création de deux nouveaux états qui se placeront en amont et en aval de la modélisation des instructions du bloc `if`. Par contre lors de la modélisation d'une instruction `if else`, nous créons quatre nouveaux états en tout, deux états qui encerclent la modélisation des instructions du bloc `if`, et deux états qui entourent la modélisation

des instructions du bloc **else**. Les figures suivantes résument bien la modélisation à réaliser (le terme $\llbracket s \rrbracket$ représente la modélisation de l'instruction s).



Modélisation des instructions de branchement : **if**(e) s ; et **if**(e) s_1 ; **else** s_2 ;

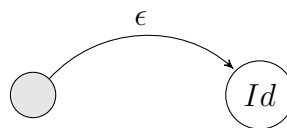
Modélisation des instructions de saut (**goto**)

Une instruction de saut permet de rediriger l'exécution vers un nouvel état identifié par une étiquette. Intuitivement, nous pouvons statuer que la représentation de cette instruction n'est rien d'autre qu'une transition partant de l'état actuel vers l'état portant le label correspondant.

Toutefois, le langage **C** permet de faire un saut vers une instruction étiquetée dont l'emplacement au sein du programme est postérieur à celui de l'instruction de saut. Pour cela nous utilisons deux structures de données différentes :

- un tableau (*labelTable*) pour sauvegarder toutes les correspondances entre les étiquettes et les états de l'automate qui représentent une instruction étiquetée ;
- un deuxième tableau (*gotoTable*) qui permet de conserver tous les états qui représentent un saut vers une étiquette particulière.

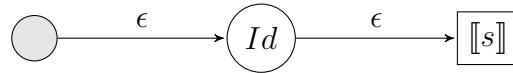
Ainsi lors de la modélisation de l'instruction de saut, il faudrait vérifier si l'étiquette en question a déjà été modélisée, et dans ce cas il suffit de tracer une transition ϵ de l'état actuel vers cet état. Sinon, il faudrait sauvegarder (dans *gotoTable*) l'état actuel comme un état qui génère un saut vers l'étiquette en question.



Modélisation de l'instruction de saut : **goto** *id*.

Modélisation des instructions étiquetées

La modélisation des instructions étiquetées consiste à créer une transition portant le label ϵ vers un nouvel état. Cependant lors de la modélisation de cette instruction, il faut vérifier (dans *gotoTable*) s'il existe des instructions de saut déjà modélisées en état et qui réorientent l'exécution vers l'état actuel. Dans ce cas, il faut tracer les transitions nécessaires partant de ces états vers l'état actuel. De plus, lors de la modélisation de cette instruction il faut mettre à jour la structure *labelTable* en y ajoutant la correspondance entre l'étiquette en question et l'état en cours.



Modélisation des instructions étiquetées : `id` : `s`.

5.3.2 Élimination des ϵ -transitions

Une fois toutes les instructions du programme modélisées, nous avons une structure complète qui représente le flot de contrôle du programme. Nous pouvons alors passer à l'étape de suppression des transitions que nous jugeons inutiles pour la modélisation en expressions régulières. En effet, plus le modèle de l'automate est concis, plus facile sera la traduction. Toutefois, il faudra supprimer les éléments inutiles en veillant à ne pas altérer le comportement original du programme, et garder intact l'enchaînement des appels des fonctions. D'où l'utilité d'avoir pris le temps d'étiqueter les transitions superflues avec le label ϵ .

Afin de venir à bout des éléments inutiles de l'automate, nous procédons par une approche qui permet de garder que les états indispensables et leurs transitions. Nous procédons à la simplification de l'automate en construisant un nouvel automate final qui ne contient que les états et les transitions indispensables pour la représentation du flot de contrôle. Cette approche se décline en quatre étapes :

1. éliminer les états superflus : les états qui ne sont pas atteignables par aucune transition, et qui ont uniquement des transitions du type ϵ vers les autres états de l'automate, ne sont pas conservés dans l'ensemble des états de l'automate final ;
2. pour chaque état, calculer l'ensemble des états atteignables par transition ϵ : ainsi pour chaque état, nous pouvons savoir quels sont les chemins possibles pour atteindre un état inclus dans l'ensemble des états de l'automate final ;
3. éliminer les transitions ϵ entre deux états : pour chaque état de l'ensemble des états de l'automate final, éliminer les transitions ϵ qui se trouvent dans les chemins vers les autres états du même ensemble d'états de l'automate final.

4. calcul des états finaux de l'automate : pour tous les états, s'il existe un chemin à travers des ϵ transitions vers un état final, il faut ajouter cet état dans l'ensemble des états finaux de l'automate final.

L'application de ces étapes de simplification permet de ne garder que les transitions étiquetées avec des appels de fonctions. Ainsi, l'automate final obtenu représente le flot de contrôle du programme original.

5.3.3 Table des transitions

La table des transitions est une matrice de dimension $n \times n$ où n représente le nombre d'états de l'automate. Elle permet de clairement identifier les transitions qui existent entre deux états différents de l'automate. De plus cette table servira comme point de départ pour la transformation de l'automate en expressions régulières. Le processus d'élaboration de cette table est relativement simple : il suffit de parcourir tous les états de l'automate, et mettre à jour la matrice avec les étiquettes des transitions sortantes de chaque état.

Plus précisément, un élément de la matrice de coordonnées (i, j) représente la somme de toutes les transitions qui partent de l'état i vers l'état j : si l'automate n'admet pas de transitions entre deux états i et j , le chiffre 0 sera placé dans la case de coordonnées (i, j) de la table de transition ; par contre, si plus d'une transition relie deux états i et j , la somme des étiquettes de ces transitions (les étiquettes de toutes les transitions de i vers j séparées avec l'opérateur d'addition $+$) sera placée dans la case de coordonnées (i, j) .

5.3.4 Exemple de modélisation

Nous présentons à présent un exemple concret afin d'illustrer la démarche présentée pour la modélisation d'un programme transformé.

Le programme suivant (table 5.11), commence par lire le nombre de lignes dans un fichier, puis, il passe à la lecture des données dans une boucle `while`. Par la suite, selon la valeur d'une variable a , il y a invocation de la fonction *encrypt* ou de la fonction *decrypt*. Finalement, suivant le résultat du test sur une variable b , il y a un appel de la fonction *print* ou la fin de l'exécution à travers la directive *exit*.

```

void f ( )
{
    flag1 : read (nbLine );
    l_begin_while : if(eof) goto l_end_while ;
    read (c);
    goto l_begin_while;
    l_end_while : ;
    if(a) decrypt ( );
    else encrypt ( );
    if(b)
    {
        send ( );
        goto flag2 ;
    }
    else
    {
        print ( );
        goto flag1 ;
    }
    flag2 : exit ;
}

```

TABLE 5.11 – Programme p après transformation.

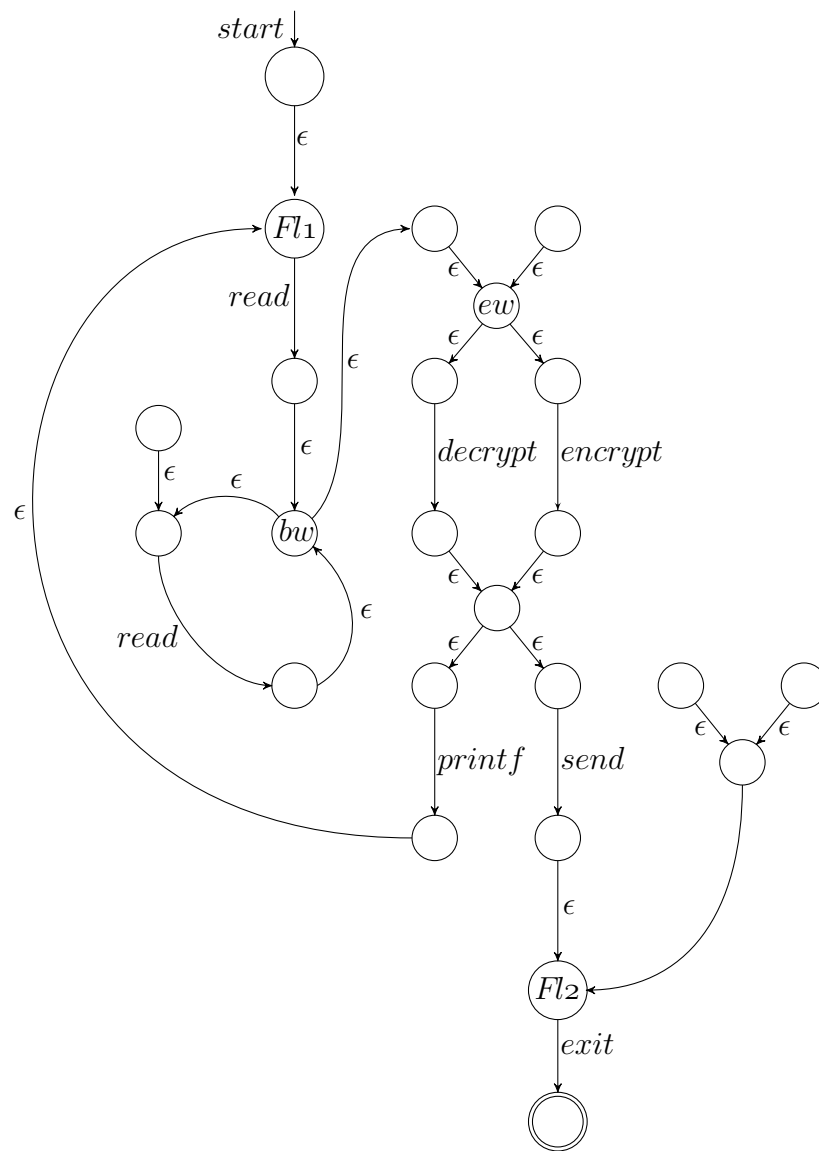
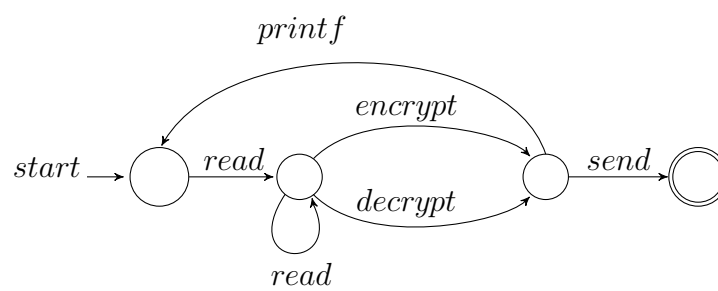
En suivant les principes de modélisation énoncés précédemment, nous aboutissons à l'automate préliminaire de la figure 5.2. En auscultant cet automate, nous pouvons clairement distinguer la structure des différentes instructions qui composent le programme. En effet, la structure de la boucle **while** est facilement décelable à travers les états notés bw (**begin_while**) et ew (**end_while**). Il en est de même pour la modélisation des instructions étiquetées **flag1** et **flag2** qui sont représentées par les états $Fl1$ et $Fl2$. Il serait utile de mentionner que, lors de cette modélisation, nous considérons que l'appel de la directive **exit** représente l'état final du programme

La simplification de cet automate se fait en éliminant les transitions ϵ ainsi que les états sans valeur ajoutée pour la représentation du flot de contrôle du programme.

Le résultat de la simplification de l'automate est donné par la figure 5.3; on y retrouve le flot de contrôle qui représente l'agencement des appels de fonction du programme.

La table de transitions des cet automate représente les fonctions qui permettent de passer d'un état d'exécution à un autre. Elle se présente sous la forme de la matrice suivante :

$$\begin{bmatrix} 0 & r & 0 & 0 \\ 0 & r & d+e & 0 \\ p & 0 & 0 & s \\ 0 & 0 & 0 & 0 \end{bmatrix}$$


 FIGURE 5.2 – Modèle du programme p

 FIGURE 5.3 – Flot de contrôle du programme p

où $r \stackrel{\text{déf}}{=} read$, $p \stackrel{\text{déf}}{=} print$, $s \stackrel{\text{déf}}{=} send$, $e \stackrel{\text{déf}}{=} encrypt$ et $d \stackrel{\text{déf}}{=} decrypt$

5.4 Modélisation en expressions régulières

Selon Kozen [28], en algèbre de Kleene, un automate fini s'écrit sous la forme d'un triplet

$$\mathcal{A} = (u, A, v)$$

avec u, v deux vecteurs de l'ensemble $\{0, 1\}^n$ et A une matrice carrée de l'ensemble $\mathcal{M}(n, \mathcal{K})$, l'ensemble des matrices carrées d'ordre n de l'algèbre de Kleene. En prenant les états de l'automate pour indices des lignes et des colonnes, le vecteur u représente le vecteur des états initiaux et le vecteur v représente le vecteur des états finaux de l'automate. La matrice A est une matrice carrée de $n \times n$ qui représente la table de transitions de l'automate.

Ainsi, en algèbre de Kleene, le langage accepté par un automate fini est définie par la relation :

$$u^T A^* v \in \mathcal{K}$$

De ce fait, la traduction du programme 5.11 en algèbre de Kleene est donnée par

$$\begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & r & 0 & 0 \\ 0 & r & d+e & 0 \\ p & 0 & 0 & s \\ 0 & 0 & 0 & 0 \end{bmatrix}^* \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = (rr^*(d+e)p)^* rr^*(d+e)s$$

L'algorithme de calcul de l'étoile d'une matrice carrée est fourni dans l'annexe A.

Finalement, pour procéder à la vérification par évaluation de modèle, il suffit de confronter le programme exprimé en algèbre de Kleene à une propriété de sécurité. Soit la propriété suivante : "Aucune information ne peut être envoyée sur le réseau après la lecture d'un fichier". En vue de vérifier si le programme respecte cette propriété ou pas, il serait plus évident de vérifier si le flot de contrôle du programme respecte la forme positive de la propriété, à savoir "le programme lit une information, ensuite il envoie des données sur le réseau". Ainsi, si le programme respecte cette propriété, on en déduit qu'il ne vérifie pas la propriété originale.

La traduction de cette nouvelle propriété sous forme d'une expression de la logique L tel qu'énoncée dans [31], s'écrit :

$$\|\phi\| = \infty r \infty s \infty$$

avec r représente l'action *read* ($r \stackrel{\text{déf}}{=} \text{read}$), s représente l'action *send* ($s \stackrel{\text{déf}}{=} \text{send}$) tandis que ∞ désigne n'importe quel séquence d'actions possibles.

Ainsi, la vérification du programme par l'approche algébrique se résume à la vérification de l'inéquation suivante :

$$(rr^*(d + e)p)^*rr^*(d + e)s \leq \infty r \infty s \infty$$

La résolution de l'inéquation tel que détaillée dans [31] s'ensuit par le postulat que le programme permet l'invocation de la fonction **read** suivie de l'appel de la fonction **send**. Ce qui en découle que le programme ne respecte pas la propriété de sécurité originale. En effet, il suffit d'observer l'automate de la figure 5.3 pour s'apercevoir qu'il est possible que le programme envoie des informations sur le réseau après la lecture des données du fichier.

5.5 Conclusion

La démarche présentée dans ce chapitre permet d'obtenir une modélisation d'un programme écrit en langage C sous forme d'une expression régulière de l'algèbre de Kleene. Cette traduction est une phase primordiale pour pouvoir mettre en application la vérification de programmes par un procédé exclusivement algébrique. Dans le chapitre suivant, nous présentons un outil permettant de mettre en pratique la méthode de traduction de même qu'une application directe de la résolution d'équations algébriques, sous forme d'une application Web, en vue de la vérification de programmes.

Chapitre 6

Implantation

Le travail de recherche réalisé dans le cadre de cette maîtrise s'inscrit dans un travail de plus grande envergure qui consiste à analyser des programmes informatiques en utilisant une approche purement algébrique [29, 30, 32]. Plus précisément, dans cette approche, on considère l'analyse de programmes comme une résolution d'équations en une sous-algèbre de l'algèbre de Kleene admettant des inconnues.

En outre, en considérant une version restreinte du μ -calcul linéaire, une nouvelle approche algébrique pour la vérification de modèles a été développée. Dans cette approche, le programme à analyser ainsi que la propriété que l'on veut que ce dernier respecte sont traduits en termes algébriques. Le but de la résolution est de démontrer que le terme qui représente le programme est *plus petit* que celui qui représente la propriété. Ensuite, une série de lois et de théorèmes sont appliqués afin de déterminer si le programme satisfait ou non la propriété. La vérification de programmes est ramenée ainsi à une simple résolution d'équations. L'utilisation de variables dans les équations offre la possibilité de vérifier des programmes admettant des parties non encore implantées ; dans un contexte d'une équipe développant, de manière décentralisée, une même application, ceci permet de vérifier des versions partielles du code développé.

Les contributions apportées à ce vaste projet consistent, en partie, dans la conception et l'implantation d'une application web permettant d'automatiser la vérification de programme par la résolution d'une équation. Plus précisément, cette application permet de vérifier qu'une abstraction d'un programme écrit en langage $\mathbf{C}$, qui se résume à son flot de contrôle, respecte une propriété définie à partir d'une version restreinte du μ -calcul linéaire.

D'autre part, au niveau de l'abstraction de code $\mathbf{C}$, une concrétisation des contributions réside dans la conception et l'implantation d'un outil de traduction de programmes

écrit en langage C en termes de l'algèbre de Kleene. Cet outil permet d'obtenir une abstraction d'un programme écrit en langage C en termes algébriques, et par conséquent apte à être analysé via l'approche algébrique d'analyse de programmes.

Dans ce qui suit, nous présentons un aperçu de l'application web permettant la confrontation du modèle d'un programme à une propriété, ensuite nous nous intéressons à l'outil de traduction qui génère un modèle algébrique d'un programme écrit en langage C selon l'approche présentée dans le chapitre précédent.

6.1 Application Web pour l'analyse de programmes

L'interface de l'application présentée par la figure 6.1 est divisée en trois sections :

- section “Program Verification”
- section “Solving Equations”
- section “Solving Steps”

La première section de l'outil, intitulée “Program verification”, permet d'utiliser l'outil pour la vérification de programmes C relativement à des propriétés logiques proches du μ -calcul linéaire. Cette section est directement liée à la deuxième section, “Solving Equations”, par deux boutons (“Translate”) qui permettent respectivement de traduire un programme C en un terme algébrique représentant une abstraction des appels d'APIs du programme, et une propriété en un terme algébrique correspondant. Au niveau du code C , le module qui réalise la traduction correspond à l'implantation de la technique proposée au chapitre 5. Les détails de cette implantation font l'objet de la section 6.2.

Une fois le programme C et la propriété logique saisis et traduits en termes algébriques, on procède à la résolution de l'inéquation pour décider si le programme respecte la propriété. À titre d'exemple, considérons le terme “ $a;b;c;d;e$ ”, qui abstrait un programme correspondant à la séquence de cinq actions, et le terme “ $\#;c;\#$ ”, qui abstrait une propriété logique précisant que quelque part dans le programme, l'action “ c ” est effectuée ; le symbole “ $\#$ ” abstrait n'importe quel sous-bloc d'instructions. Lorsqu'on procède à la résolution, par l'intermédiaire du bouton “Solve”, l'outil indique alors que l'inéquation “ $a;b;c;d;e \leq \#;c;\#$ ” est valide (“The inequation is valid”). En effet, il est facile de constater que le programme, abstrait par le terme “ $a;b;c;d;e$ ”, admet une action “ c ”.

Par ailleurs, il est possible d'obtenir les différentes étapes que l'outil a effectuées pour résoudre (valider) une inéquation : il suffit pour cela de cliquer sur le bouton “Resolution steps” : Une troisième et dernière section devient alors disponible : elle énumère les différentes étapes de résolution ; il est possible d'avoir une version “PDF” de ces étapes de résolution.

Solving Equations in Kleene Algebra

Demo



Program verification

Test3

Parcourir... OK

eventually(<encode>eventually(<send>tt))

Program (C code)

```
// Divulge a secret information
int get_number( void );
int read_secret_number( void );
void send_number( int iNumber );

int encode( int iNumber );
void send( int port, int iNumber );
```

Translate

Formula

eventually(<encode>eventually(<send>tt))

Translate

Solving Equations Hide Regular Expressions

Exp1 (program) Select an expression ...

Simplify

a;b;c;d;e

<=

Exp2 (Formula) Select an expression ...

Simplify

#.c.#

Solve

The inequation is valid.

Resolution steps

Format ☐ Text ☒ PDF Text size : 10

genReport

1 / 1 125% Rechercher

$$\begin{aligned}
 & a;b;c;d;e \leq \infty;c;\infty \\
 & \quad \langle \text{Using rule } "a;y \leq z \Leftrightarrow y \leq a \backslash z, \text{ if } a \in G" \rangle \\
 \Leftrightarrow & b;c;d;e \leq a \backslash (\infty;c;\infty) \\
 & \quad \langle \text{Using rule } "a \backslash (\infty;x) = \infty;x + a \backslash x, \text{ if } a \in G" \rangle \\
 \Leftrightarrow & b;c;d;e \leq (\infty;c;\infty) + (a \backslash (c;\infty)) \\
 & \quad \langle \text{Using rule } "b \backslash (a;x) = (b \backslash a);x, \text{ if } a, b \in G" \rangle \\
 \Leftrightarrow & b;c;d;e \leq (\infty;c;\infty) + ((a \backslash c);\infty) \\
 & \quad \langle \text{Using rule } "h \backslash a = 0 \text{ if } a, b \in G \wedge a \neq h" \rangle
 \end{aligned}$$

FIGURE 6.1 – Application Web pour l’analyse de programmes.

En outre, les étapes de résolution sont à interpréter comme de simples étapes de résolution d'inéquations arithmétiques en algèbre classique :

$$\begin{array}{c} \text{Inéquation}_1 \\ \leftrightarrow \quad \langle \text{Justification} \rangle \\ \text{Inéquation}_2 \\ \vdots \end{array}$$

Rappelons qu'en arithmétique élémentaire :

$$\begin{array}{c} (a + b) * X = c * d \\ \leftrightarrow \quad \langle A * X = B \leftrightarrow X = B/A \text{ si } A \neq 0 \rangle \\ X = (c * d) / (a + b) \\ \vdots \end{array}$$

En considérant l'application Web implantée, reprenons la première étape de résolution¹ :

$$\begin{array}{c} a; b; c; d; e \leq \infty; c; \infty \\ \leftrightarrow \quad \langle \text{Using rule} \rangle a; y \leq z \leftrightarrow y \leq a \setminus z \text{ if } a \text{ in } G \rangle \\ b; c; d; e \leq a \setminus (\infty; c; \infty) \\ \vdots \end{array}$$

Elle indique que de l'inéquation initiale “a;b;c;d;e <= #;c;#”, on peut déduire une nouvelle inéquation “b;c;d;e <= a \ (\#;c;#)” en utilisant la règle indiquée. On peut ainsi parcourir toutes les étapes de résolution pour constater que le tout termine par “true” indiquant que l'inéquation de départ est valide.

On peut bien sûr considérer des exemples plus complexes. Par exemple, soit le terme “i;X;(o;d + n);s” qui admet une inconnue “X”. Considérons alors la résolution de l'inéquation “i;X;(o;d + n);s <= #;r;#;s;#”; si elle est valide, ceci indiquerait que le programme “i;X;(o;d + n);s” respecte la propriété même si une partie du programme est inconnue (variable “X” ; dans notre implantation, les variables sont spécifiées par des caractères en majuscules). Donc, on est dans un contexte où on veut vérifier un programme, par rapport à une propriété, même si ce programme comprend une partie

1. Dans la version PDF, le symbole «#» est représenté par «∞».

non encore implantée (cette partie est abstraite, dans cet exemple, par la variable “X”). Il est intéressant de constater que, contrairement aux exemples précédents où la résolution terminait par une valeur booléenne, dans ce cas, la dernière étape spécifie que “ $X \leq \#;r;\#$ ”. Donc la résolution termine par une condition (solution) sur la variable “X”. Cette condition indique que si la variable “X” (qui abstrait un sous-programme) est inférieure ou égale à “ $\#;r;\#$ ”, alors l’inéquation de départ est valide. “ $X \leq \#;r;\#$ ” indique que le sous-programme abstrait par “X” comprend forcément une action “r”, ce qui est cohérent puisque dans l’inéquation de départ, “ $i;X;(o;d + n);s \leq \#;r;\#;s;\#$ ”, on voit clairement que le programme “ $i;X;(o;d + n);s$ ” respecterait la propriété “ $\#;r;\#;s;\#$ ” que dans le cas où “X” contient au moins une action “r”. Donc, en conclusion, notre outil permet de vérifier des programmes avec des parties inconnues et dans le cas où ces programmes respecteraient les propriétés en question, l’outil nous retourne une approximation (un majorant) des valeurs de ces variables.

6.2 Modélisation de code C

L’outil de traduction réalisé permet de mettre en pratique la méthode de transformation du code source d’un programme, écrit en langage C, vers une expression régulière représentative du flot de contrôle du programme. L’analyse d’un programme commence par l’analyse syntaxique du code source, ensuite l’abstraction des instructions et expressions pour ne garder que les appels API, puis l’automate fini représentant le flot de contrôle du programme est construit afin d’aboutir finalement à une représentation algébrique sous forme d’expressions régulières.

Ainsi, l’interface de l’outil de traduction est divisée en quatre onglets, chacun présente le résultat d’une étape de transformation du code source jusqu’à la traduction complète du programme.

Onglet “Source” Le premier onglet de l’outil, tel que le montre la figure 6.2, offre à l’utilisateur de saisir le code source du programme à traduire. Si le programme à traduire est volumineux, il est possible de charger le fichier source directement dans la zone de saisie à travers le bouton de chargement de fichier. L’interface permet aussi de sauvegarder ou d’imprimer le texte du code source du programme, comme il est aussi possible de le faire pour le résultat obtenu après la transformation du programme. La saisie du programme (ou le chargement à partir d’un fichier) entraîne l’analyse syntaxique du code source et par conséquent une validation par rapport à la grammaire du langage C. Si la syntaxe utilisée respecte la grammaire, l’arbre syntaxique du programme est construit. Cette modélisation préliminaire du programme source est assurée par

l'analyseur syntaxique généré à partir de la grammaire BNF du langage C en utilisant GoldParser. Concrètement, le résultat de l'analyse syntaxique est un arbre syntaxique dont la racine est le point d'entrée du programme, généralement la déclaration de la fonction `main()`, et chaque noeud de l'arbre est une des règles de la grammaire du langage présente dans le programme, et les feuilles de l'arbre sont les éléments terminaux de la grammaire.

Par exemple, si le programme contient une expression d'affectation d'une variable `x` à la valeur 0, notée `"x=0;"`, un des noeuds de l'arbre syntaxique généré sera la règle d'affectation (`<assignment-expression>`), et les feuilles seraient les éléments terminaux qui la constituent, à savoir l'identifiant `"x"`, le symbole numérique `"0"` et le symbole d'affectation `"="`. L'arbre syntaxique ainsi construit sera utilisé ultérieurement pour appliquer les règles de transformation énoncées plus haut.

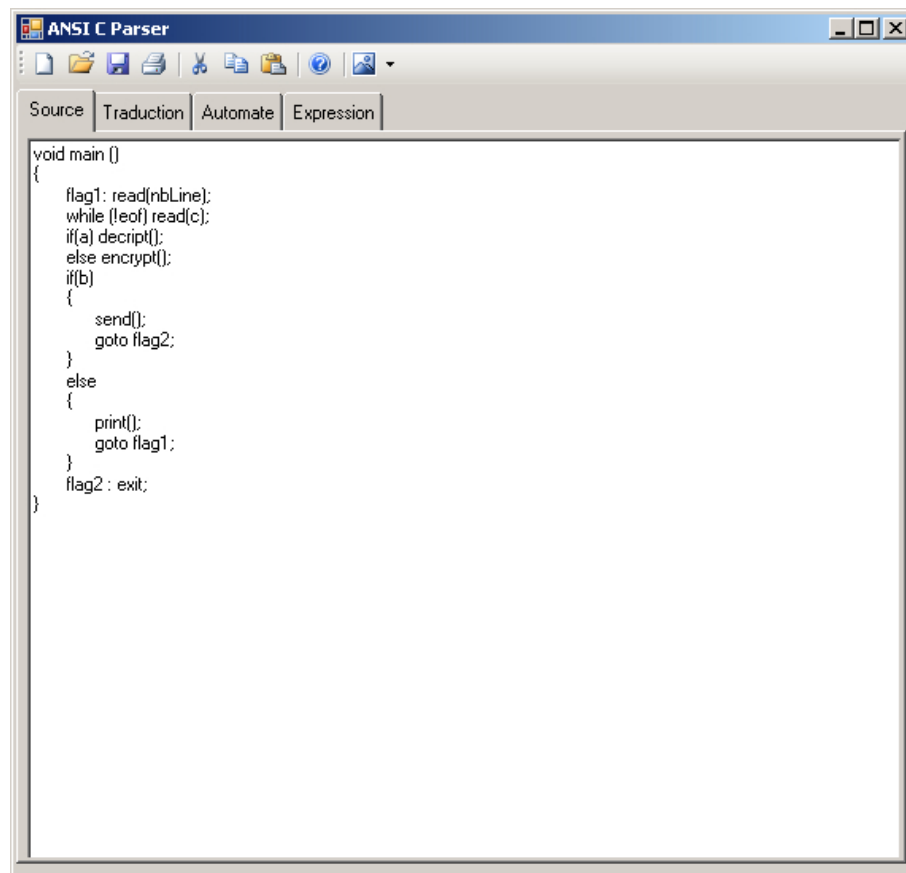


FIGURE 6.2 – Outils de modélisation - onglet "Source".

Onglet "Traduction" Le deuxième onglet présente le résultat de l'analyse syntaxique du programme après y avoir appliqué les règles de transformations présentées dans le chapitre 5. Le résultat de la transformation du programme est un nouveau

programme sémantiquement identique au programme original, mais dépourvu de toutes constructions syntaxiques complexes telles que les structures itératives (`do,while,for`) ou les structures d'aiguillage (`select, case`). En d'autres termes, le programme transformé est une version linéaire du code source original utilisant une syntaxe allégée du langage C. La figure 6.3 montre un exemple d'un programme C après transformation.

Le programme obtenu après la phase de traduction est traité par l'analyseur syntaxique afin d'obtenir le nouvel arbre syntaxique, celui qui correspond au programme transformé. Ce dernier sera le point de départ de la modélisation du programme sous forme d'un automate à états finis.

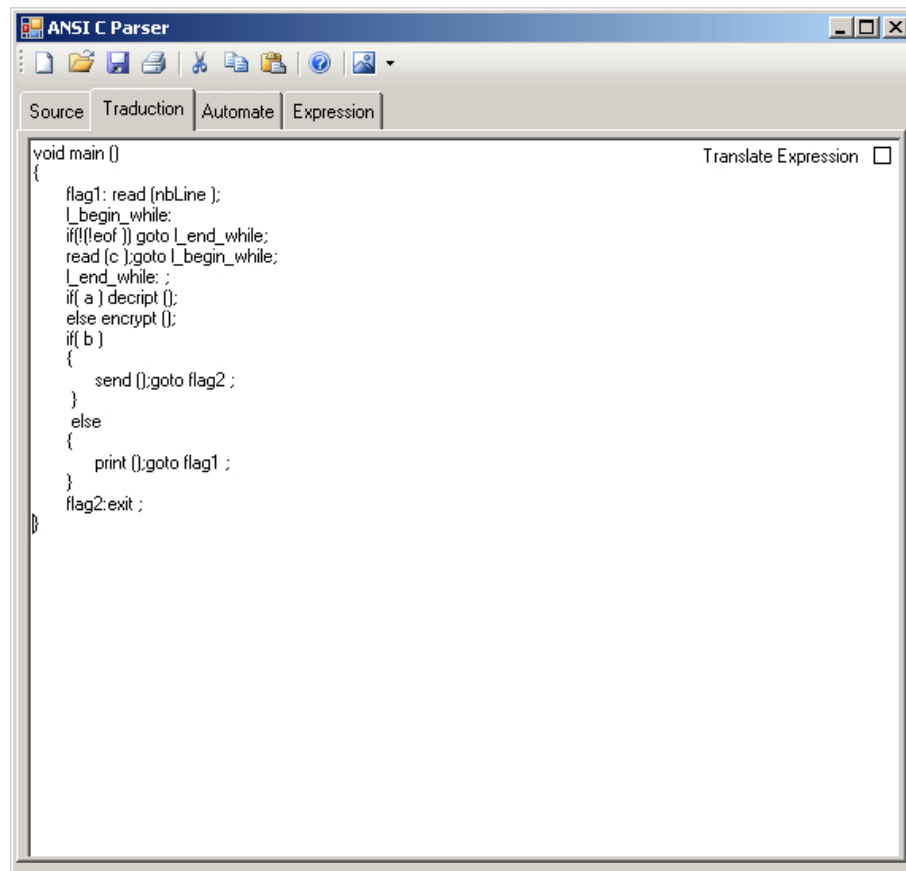


FIGURE 6.3 – Outils de modélisation - onglet “Traduction”.

Onglet “Automate” Le programme obtenu après transformation est composé principalement des appels des fonctions API, des instructions de saut (`goto`) et des instructions de branchement (`if..then..else`). L'enchaînement des divers appels des fonctions API permet de construire un automate fini qui représente le flot de contrôle du programme. Pour cela, l'abstraction des appels de fonctions est traduite par des transitions qui mènent d'un état du système vers un autre, tandis que les branchements

et les sauts sont représentés par des transitions du type ϵ tel que détaillé précédemment (chapitre 5). Le résultat, tel que présenté dans l'interface sous l'onglet "Automate", est un automate composé des éléments suivants :

- un état initial qui représente le point d'entrée du programme ;
- des transitions étiquetées avec les noms des fonctions API du programme ;
- des transitions du type ϵ ;
- plusieurs états qui représentent les différentes situations du système.
- un état final qui représente la sortie du programme.

La modélisation initiale du programme sous forme d'un automate, est susceptible de contenir beaucoup de transitions inutiles et encombrantes pour le modèle. L'élimination des ces transitions notées ϵ permet d'aboutir finalement au flot de contrôle souhaité du programme original, et qui modélise uniquement l'enchaînement exact des appels des fonctions API du programme. Telle que présentée dans les figures 6.4 et 6.5, l'interface de l'outil contient une "case à cocher" qui permet de basculer entre les deux versions du modèle : avec et sans ϵ -transitions.

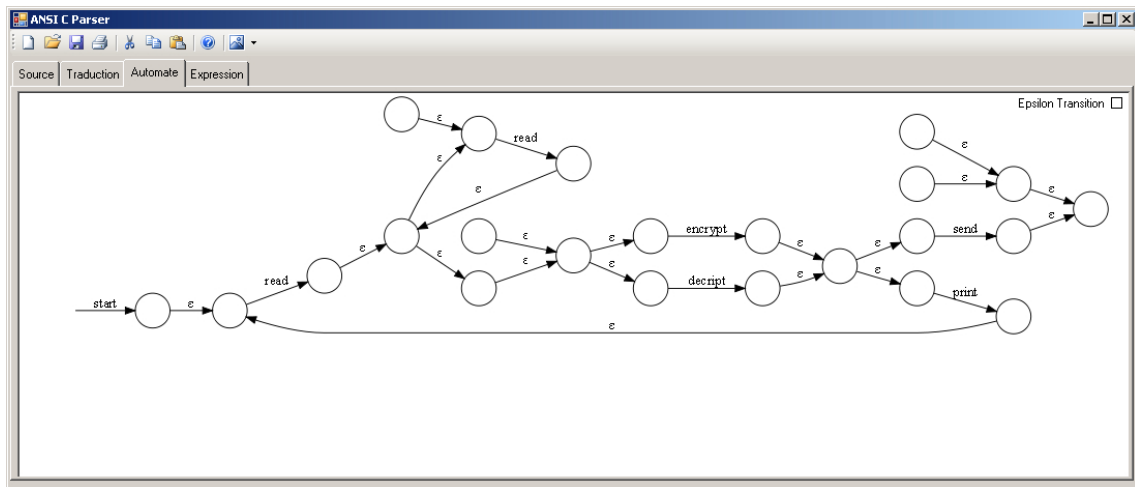


FIGURE 6.4 – Outils de modélisation - onglet "Automate" sans ϵ -réduction.

Onglet "Expression" Finalement, le dernier onglet de l'interface, figure 6.6, affiche la formulation algébrique représentant le flot de contrôle du programme source avec le nom des fonctions abstraites. L'expression régulière est obtenue à partir de la table de transitions de l'automate épuré des transitions ϵ encombrantes.

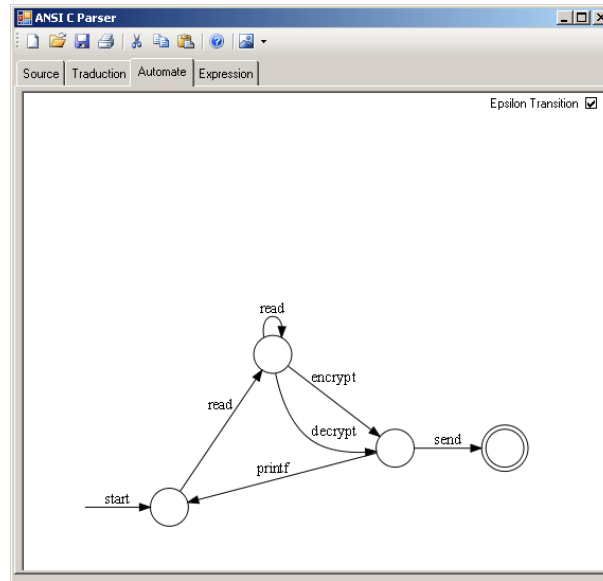
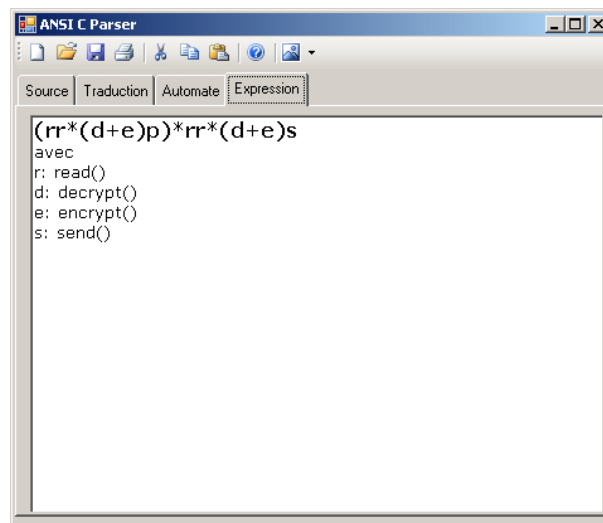
FIGURE 6.5 – Outils de modélisation - onglet “Automate” avec ϵ -réduction.

FIGURE 6.6 – Outils de modélisation - onglet “Expression”.

6.3 Conclusion

Le processus de traduction énoncé dans cette section concerne une partie de la syntaxe du langage C. En raison de la complexité de ce langage, il n'est pas certain qu'une traduction, basée uniquement sur la syntaxe, puisse fournir une modélisation de n'importe quel programme informatique écrit en langage C. La méthode que nous avons présentée se restreint à la traduction de programmes simples ne comprenant pas

d'appels récursifs, ni de tableaux et moins encore l'usage des pointeurs. Pour pouvoir traiter des programmes complexes, il faudrait en plus user d'outils plus puissants pour la traduction, en prenant en considération le type des expressions et l'arithmétique des pointeurs.

Chapitre 7

Conclusion

Les méthodes formelles pour l'analyse des programmes permettent de confronter un modèle du programme par rapport à une politique de sécurité. Cependant, le point de départ de ces techniques consiste généralement en une abstraction d'un programme concret. Dans ce mémoire, nous nous sommes intéressés à combler cette brèche en proposant un outil informatique qui permet de traiter un "vrai" programme écrit en langage C, et à le transformer en une expression régulière, aspect sous lequel l'application de la vérification par évaluation de modèle selon une approche algébrique peut avoir lieu. Cette démarche algébrique permet d'aboutir à des manipulations originales dans le cadre de l'analyse de programmes, et le fait d'utiliser un outil informatique offre la possibilité de traiter des programmes déjà existants, d'où l'intérêt du développement réalisé.

La transformation de programmes écrits en langage C n'est pas tout à fait évidente, surtout à cause de la complexité de la grammaire du langage. Ceci nous a obligés à nous concentrer que sur certains aspects de la transformation, et par conséquent limiter la classe des programmes pouvant être analysés par la version actuelle de l'outil. En effet cette version actuelle permet de simplifier les programmes qui contiennent des instructions, des structures itératives, et des instructions du type `case` seulement. Une mise à jour future devrait se pencher sur des problèmes plus complexes qui existent dans les programmes C les plus courants, à savoir, la récursivité, les pointeurs et les tableaux de fonctions. Plusieurs recherches dans ce sens existent actuellement et comme perspective au travail accompli, il serait intéressant de se pencher sur les travaux futurs susceptibles de compléter la version actuelle pour qu'elle prenne en considération ce genre de difficultés.

Travaux futurs

Ce travail peut être étendu de différentes façons. Premièrement, il faudrait ajouter à l'analyse de flot de contrôle, une analyse de flot de données de programmes écrits en C. En effet, dans le cadre du travail effectué, nous avons ignoré, entre autres, les pointeurs de fonctions ainsi que les fonctions déclarées externes. Il serait pertinent d'effectuer une analyse de flot de données, et particulièrement une analyse de pointeurs, afin d'abstraire de type de fonctions par des variables. Considérons par exemple le code C présenté dans la figure 7.1 :

```
extern int openFile(char*);
extern void closeFile(int);
extern int send(int, char*);

int main()
{
    int socket;
    int file;
    char fileName[1000];
    int (*f)(char *);

    f = openFile;
    f("f1.ex");
    send( socket, "done" );
    f("f2.ex");
    return 0;
}
```

TABLE 7.1 – Exemple de code C.

Il serait intéressant d'abstraire les deux appels «f("f1.ex")» et «f("f2.ex")» par la même variable «X» puisque par analyse statique, il sera possible d'identifier que les deux appels réfèrent la même fonction externe. Comme mentionné dans la section 6.1 (page 101), l'approche algébrique développée par François Lajeunesse-Robert [31], membre du groupe de recherche *LSFM* dont je fais partie, supporte la résolution d'inéquations admettant des inconnues ; l'application Web développée, et présentée à la section 6.1 (page 101), est donc capable de traiter ce type de programmes.

Une autre extension du travail consiste à analyser des programmes rékursifs. Plus précisément, il s'agit de traduire du code `C` en termes d'une algèbre permettant de représenter à la fois des programmes ayant des appels rékursifs et des propriétés non régulières tout en ayant une procédure de décision automatique. En se basant sur le travail entrepris par Claude Bolduc [6], membre du groupe de recherche *LSFM*, il serait intéressant de reconsidérer la méthode présentée au chapitre 5 pour produire des termes de cette algèbre, appelée *algèbre de Kleene à pile visible*.

Bibliographie

- [1] Martin Abadi, Mihai Budiu, Úlfar Erlingsson, and Jay Ligatti. Control-flow integrity principles, implementations, and applications. *ACM Trans. Inf. Syst. Secur.*, 13(1) :1–40, 2009.
- [2] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers : Principles, Techniques, and Tools (2nd Edition)*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2006.
- [3] Zahira Ammarguellat. A control-flow normalization algorithm and its complexity. *IEEE Transactions on Software Engineering*, 18 :237–251, 1992.
- [4] Jean Bergeron, Mourad Debbabi, Jules Desharnais, Béchir Ktari, Martin Salois, and Nadia Tawbi. Detection of malicious code in COTS Software : A short survey. In *International Software Assurance Certification Conference, ISACC'99*, pages 214–230. IEEE Press, Mars 1999.
- [5] Jean Bergeron, Mourad Debbabi, Jules Desharnais, M.Erhioui, Y.Lavoie, and Nadia Tawbi. Static detection of malicious code in executable programs. *Int. J. of Req. Eng.*, 2001.
- [6] Claude Bolduc. *Vers une vérification interprocédurale de programmes impératifs contre des spécifications tenant compte du contexte des procédures dans un formalisme algébrique*. Proposition de recherche, Université Laval, Département d'informatique et de génie logiciel, 2008.
- [7] Cristina Cifuentes and Mike Van Emmerik. Recovery of jump table case statements from binary code. In *Science of Computer Programming*, pages 2–3, 1999.
- [8] Devin D. Cook. [http ://www.devincook.com/goldparser/](http://www.devincook.com/goldparser/).
- [9] Devin D. Cook. Design and development of a grammar oriented parsing system. Master's thesis, California State University, Sacramento, 2004.
- [10] Daniel Damian and Olivier Danvy. Static transition compression, 2001.
- [11] Olivier Danvy and Andrzej Filinski. Representing control : a study of the cps transformation. *Mathematical Structures in Computer Science*, 2(361 - 391), 1992.

- [12] Mourad Debbabi, E. Giasson, Béchir Ktari, Frédéric Michaud, and Nadia Tawbi. Secure self-certified cots. In *WETICE '00 : Proceedings of the 9th IEEE International Workshops on Enabling Technologies*, pages 183–188, Washington, DC, USA, 2000. IEEE Computer Society.
- [13] Franklin Lewin DeRemer. Practical translators for LR(k) languages. Technical report, Massachusetts Institute of Technology, Cambridge, MA, USA, 1969.
- [14] Jules Desharnais. *Informatique théorique*. Département d’informatique et de génie logiciel, Université Laval, 2004.
- [15] Edsger Wybe Dijkstra. GOTO Statement Considered Harmful. *Comm. ACM*, 11(3) :147–148, Mars 1968.
- [16] R. Kent Dybvig, Robert Hieb, and Tom Butler. Destination-driven code generation. Technical report, Indiana University Computer Science Department, Février 1990.
- [17] Ana Maria Erosa. A goto-elimination method and its implementation for the McCat C Compiler. In *In Proc. of the 5th Intl. Work*, 1995.
- [18] François Yvon et Akim Demaille. *Théorie des langages*. EPITA Research and Development Laboratory, EPITA, Janvier 2010.
- [19] Therrezina Fernandes. *Algèbres de Kleene pour l’analyse statique des programmes*. Thèse de doctorat, Université Laval, 2008.
- [20] Robert W. Floyd. Assigning meanings to programs. In J. T. Schwartz, editor, *Mathematical Aspects of Computer Science, Proceedings of Symposia in Applied Mathematics 19*, volume 19, pages 19–31, Providence, 1967. American Mathematical Society.
- [21] Galen Hunt and Doug Brubacher. Detours : binary interception of win32 functions. In *Proceedings of the 3rd conference on USENIX Windows NT Symposium - Volume 3*, pages 14–14, Berkeley, CA, USA, 1999. USENIX Association.
- [22] Petr Jancar, Frantisek Mraz, and Martin Platek. Forgetting automata and context-free languages. *ACTA INFORMATICA*, 33(5) :409 – 420, 1996.
- [23] Thomas J. Pennello and Frank DeRemer. Efficient computation of LALR(1) look-ahead sets. *SIGPLAN Not.*, 39(4) :14–27, 2004.
- [24] Joost-Pieter Katoen. *Principle of Model Checking*. Formal Method ans Tools Group, University of Twente, 2001/2002.
- [25] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*, chapter Annexe 13. Prentice Hall Professional Technical Reference, 1988.
- [26] Brian W. Kernighan and Dennis M. Ritchie, editors. *The C Programming Language*. Prentice Hall Professional Technical Reference, 1988.
- [27] Takahiro Kosakai, Toshiyuki Maeda, and Akinori Yonezawa. Compiling C programs into a strongly typed assembly language. In *ASIAN’07 : Proceedings of the 12th*

- Asian computing science conference on Advances in computer science*, pages 17–32, Berlin, Heidelberg, 2007. Springer-Verlag.
- [28] Dexter Kozen. A Completeness Theorem for Kleene Algebras and the Algebra of Regular Events. *Information and Computation*, 110 :366–390, mai 1994.
- [29] Béchir Ktari, François LaJeunesse-Robert, and Claude Bolduc. Solving linear equations in $*$ -continuous action lattices. In *Relation and Kleene Algebra in Computer Science, Proceedings of the 10th Relation and Kleene Algebra in Computer Science, Proceedings of the 10th International Conference on Relational Methods in Computer Science and 5th International Conference on Applications of Kleene Algebra(RelMiCS/AKA)*, volume 4988, pages 289–303, Frauenwörth, Allemagne, avril 2008. LNCS, Springer.
- [30] Béchir Ktari, François LaJeunesse-Robert, and Claude Bolduc. Solving linear equations in $*$ -continuous action lattices (extended version). Rapport de recherche diul-rr-0801, Université Laval, Département d’informatique et de génie logiciel, Juin 2008.
- [31] François LaJeunesse-Robert. Résolution d’équations en algèbre de Kleene applications à l’analyse de programmes. Mémoire de maîtrise, Université Laval, 2009.
- [32] François LaJeunesse-Robert and Béchir Ktari. Toward solving equations in Kleene algebras. In IOS Press, editor, *New Trends in Software Methodologies, Tools and Techniques, Proceedings of the 6th SoMeT International Conferenc*, volume 161, pages 285–304, Rome, Italie, 2007.
- [33] Wayne Madsen. Crypto ag : The NSA’s trojan whore? *CovertAction Quarterly*, 1(63), Hiver 1998.
- [34] Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3) :348–375, 1978.
- [35] Paul Morris, Ronald A.Gray, and Robert E.Filman. Goto removal based on regular expressions. *Journal of Software Maintenance*, 1997.
- [36] Greg Morrisett, David Walker, Karl Crary, and Neal Glew. From system F to typed assembly language. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 21(3) :527–568, Mai 1999.
- [37] George C. Necula. Proof-carrying code. In *POPL ’97 : Proceedings of the 24th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 106–119, New York, NY, USA, 1997. ACM.
- [38] George C. Necula, Scott McPeak, Shree Prakash Rahul, and Westley Weimer. Cil : Intermediate language and tools for analysis and transformation of c programs. In *CC ’02 : Proceedings of the 11th International Conference on Compiler Construction*, pages 213–228, London, UK, 2002. Springer-Verlag.

- [39] Wesley W. Peterson, Tadao Kasami, and Nobuki Tokura. On the capabilities of while, repeat, and exit statements. *Communication of the ACM*, 16(8) :503–512, 1973.
- [40] Frank Pfenning. The practice of logical frameworks. In *CAAP '96 : Proceedings of the 21st International Colloquium on Trees in Algebra and Programming*, pages 119–134, London, UK, 1996. Springer-Verlag.
- [41] Amir Pnueli, M. Siegel, and F. Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 1384, pages 151–166. LNCS, Springer, 1998.
- [42] Matt Rosoff. Virus alert ! how to avoid getting one, Janvier 1997.
- [43] Andrew R. Twyman. Flexible code safety for Win32. Mémoire de maîtrise, MIT, 1999.
- [44] David Walker. A type system for expressive security policies. In *POPL '00 : Proceedings of the 27th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 254–267, New York, NY, USA, 2000. ACM.
- [45] Fubo Zhang and Erik H. D'holl. Using hammock graphs to structure programs. *IEEE Transactions on Software Engineering*, 30 :231–245, 2004.
- [46] Wei Zhao and Rainer Hauser. Compiling business processes : Untangle unstructured loops in irreducible flow graphs. *Int. J. Web and Grid Services*, 2(1) :68 – 91, 2006.

Annexe A

Étoile d'une matrice de dimension $n \times n, \quad n \geq 2$

Soit A une matrice de dimensions $n \times n$, où $n \geq 2$. L'algorithme de construction de A^* est le suivant :

1. Partitionner la matrice en quatre sous-matrices B, C, D, E de telle sorte que B et E soit des matrices carrées non vides :

$$A = \left[\begin{array}{c|c} B & C \\ \hline D & E \end{array} \right]$$

2. Calculer récursivement E^* .
3. Calculer

$$F = B + CE^*D.$$

4. Calculer récursivement F^* .
5. Calculer

$$A^* = \left[\begin{array}{c|c} F^* & F^*CE^* \\ \hline E^*DF^* & E^* + E^*DF^*CE^* \end{array} \right]$$

Annexe B

Grammaire du langage C

Cette grammaire est la grammaire originale telle que décrite dans l'annexe 13 de la deuxième édition du livre *The C Programming Language* de Brian W. Kernighan. and Dennis M. Ritchie (Prentice Hall Professional Technical Reference, 1988).

```
<translation-unit>  : := {<external-declaration>}*
<external-declaration>  : := <function-definition>
                        | <declaration>
<function-definition>  : :=
    {<declaration-specifier>}* <declarator> {<declaration>}* <compound-statement>
<declaration-specifier>  : := <storage-class-specifier>
                        | <type-specifier>
                        | <type-qualifier>
<storage-class-specifier>  : := "auto"
                        | "register"
                        | "static"
                        | "extern"
                        | "typedef"
<type-specifier>  : := "void"
                | "char"
                | "short"
                | "int"
                | "long"
                | "float"
                | "double"
                | "signed"
                | "unsigned"
                | <struct-or-union-specifier>
                | <enum-specifier>
                | <typedef-name>
<struct-or-union-specifier>  : :=
```

```

<struct-or-union> <identifier> "{" {<struct-declaration>}+ "}"
                  | <struct-or-union> "{" {<struct-declaration>}+ "}"
                  | <struct-or-union> <identifier>
<struct-or-union>  := "struct"
                  | "union"
<struct-declaration>  := {<specifier-qualifier>}* <struct-declarator-list>
<specifier-qualifier> := <type-specifier>
                  | <type-qualifier>
<struct-declarator-list> := <struct-declarator>
                  | <struct-declarator-list> "," <struct-declarator>
<struct-declarator>  := <declarator>
                  | <declarator> " :" <constant-expression>
                  | " :" <constant-expression>
<declarator>         := {<pointer>}? <direct-declarator>
<pointer>            := "*" {<type-qualifier>}* {<pointer>}?
<type-qualifier>     := "const"
                  | "volatile"
<direct-declarator>  := <identifier>
                  | "(" <declarator> ")"
                  | <direct-declarator> "[" {<constant-expression>}? "]"
                  | <direct-declarator> "(" <parameter-type-list> ")"
                  | <direct-declarator> "(" {<identifier>}* ")"
<constant-expression> := <conditional-expression>
<conditional-expression> := <logical-or-expression>
                  | <logical-or-expression> "?" <expression> " :" <conditional-expression>
<logical-or-expression> := <logical-and-expression>
                  | <logical-or-expression> "||" <logical-and-expression>
<logical-and-expression> := <inclusive-or-expression>
                  | <logical-and-expression> "&&" <inclusive-or-expression>
<inclusive-or-expression> := <exclusive-or-expression>
                  | <inclusive-or-expression> "|" <exclusive-or-expression>
<exclusive-or-expression> := <and-expression>
                  | <exclusive-or-expression> "^" <and-expression>
<and-expression> := <equality-expression>
                  | <and-expression> "&" <equality-expression>
<equality-expression> := <relational-expression>
                  | <equality-expression> "==" <relational-expression>
                  | <equality-expression> "!=" <relational-expression>
<relational-expression> := <shift-expression>
                  | <relational-expression> "<" <shift-expression>
                  | <relational-expression> ">" <shift-expression>
                  | <relational-expression> "<=" <shift-expression>
                  | <relational-expression> ">=" <shift-expression>
<shift-expression> := <additive-expression>
                  | <shift-expression> "<<" <additive-expression>
                  | <shift-expression> ">>" <additive-expression>
<additive-expression> := <multiplicative-expression>
                  | <additive-expression> "+" <multiplicative-expression>
                  | <additive-expression> "-" <multiplicative-expression>

```

```

<multiplicative-expression>  : := <cast-expression>
                                | <multiplicative-expression> "*" <cast-expression>
                                | <multiplicative-expression> "/" <cast-expression>
                                | <multiplicative-expression> "%" <cast-expression>

<cast-expression>           : := <unary-expression>
                                | "(" <type-name> ")" <cast-expression>

<unary-expression>          : := <postfix-expression>
                                | "++" <unary-expression>
                                | "--" <unary-expression>
                                | <unary-operator> <cast-expression>
                                | "sizeof" <unary-expression>
                                | "sizeof" <type-name>

<postfix-expression>         : := <primary-expression>
                                | <postfix-expression> "[" <expression> "]"
                                | <postfix-expression> "(" {<assignment-expression>}* ")"
                                | <postfix-expression> "." <identifier>
                                | <postfix-expression> "->" <identifier>
                                | <postfix-expression> "++"
                                | <postfix-expression> "--"

<primary-expression>         : := <identifier>
                                | <constant>
                                | <string>
                                | "(" <expression> ")"

<constant>                   : := <integer-constant>
                                | <character-constant>
                                | <floating-constant>
                                | <enumeration-constant>

<expression>                  : := <assignment-expression>
                                | <expression> "," <assignment-expression>

<assignment-expression>      : := <conditional-expression>
                                | <unary-expression> <assignment-operator> <assignment-expression>

<assignment-operator>        : := "="
                                | "!="
                                | "/="
                                | "%="
                                | "+="
                                | "-="
                                | "<="
                                | ">="
                                | "&="
                                | "^="
                                | "|="

<unary-operator>              : := "&"
                                | "*"
                                | "+"
                                | "_"
                                | "~"
                                | "!"

<type-name>                   : := {<specifier-qualifier>}+ {<abstract-declarator>}?

```



```

<parameter-type-list>  : := <parameter-list>
                        | <parameter-list> "," ...
<parameter-list>      : := <parameter-declaration>
                        | <parameter-list> "," <parameter-declaration>
<parameter-declaration> : := {<declaration-specifier>}+ <declarator>
                        | {<declaration-specifier>}+ <abstract-declarator>
                        | {<declaration-specifier>}+
<abstract-declarator>  : := <pointer>
                        | <pointer> <direct-abstract-declarator>
                        | <direct-abstract-declarator>
<direct-abstract-declarator> : := ( <abstract-declarator> )
                        | {<direct-abstract-declarator>}? "[" {<constant-expression>}? "]"
                        | {<direct-abstract-declarator>}? "(" {<parameter-type-list>|? "]"
<enum-specifier>       : := "enum" <identifier> "{" <enumerator-list> "}"
                        | "enum" "{" <enumerator-list> "}"
                        | "enum" <identifier>
<enumerator-list>      : := <enumerator>
                        | <enumerator-list> "," <enumerator>
<enumerator>           : := <identifier>
                        | <identifier> "=" <constant-expression>
<typedef-name>          : := <identifier>
<declaration>           : := {<declaration-specifier>}+ {<init-declarator>}*
<init-declarator>       : := <declarator>
                        | <declarator> "=" <initializer>
<initializer>           : := <assignment-expression>
                        | "{" <initializer-list> "}"
                        | "{" <initializer-list> "," "}"
<initializer-list>      : := <initializer>
                        | <initializer-list> "," <initializer>
<compound-statement>    : := "{" {<declaration>}* {<statement>}* "}"
<statement>             : := <labeled-statement>
                        | <expression-statement>
                        | <compound-statement>
                        | <selection-statement>
                        | <iteration-statement>
                        | <jump-statement>
<labeled-statement>      : := <identifier> " : " <statement>
                        | "case" <constant-expression> " : " <statement>
                        | "default" " : " <statement>
<expression-statement>  : := {<expression>}? ";"
<selection-statement>    : := "if" "(" <expression> ")" <statement>
                        | "if" "(" <expression> ")" <statement> "else" <statement>
                        | "switch" "(" <expression> ")" <statement>
<iteration-statement>     : := "while" "(" <expression> ")" <statement>
                        | "do" <statement> "while" "(" <expression> ")" ";"
                        | "for" "(" {<expression>}? ";" {<expression>}? ";" {<expression>}? ")" <statement>
<jump-statement>         : := "goto" <identifier> ";"
                        | "continue" ";"
                        | "break" ";"

```

```
| "return" {<expression>} ? " ;"
```